

Top Skills de Claude Code en producción

Catálogo curado de las seis skills que aguantan en producción real, con criterio explícito para evaluar lo que venga después.

Libro técnico del curso · 8 módulos · 36 capítulos · ~59.186 palabras
Edición de trabajo · 29 de mayo de 2026

BORRADOR

Contenido base sin completar el ciclo `/revisar`. Este PDF es trabajo interno de producción; no es la edición publicable. Los datos volátiles (versiones de herramientas, comandos exactos, cifras de adopción) se re-verifican antes de cada módulo de grabación.

Contenido

M0 — Por qué este curso (criterio y filtros)

- M0.1 · De qué va este curso, en una frase
- M0.2 · Skills generales vs skills personalizadas
- M0.3 · El criterio que aplicamos — cuatro filtros
- M0.4 · Avisos antes de empezar

M1 — Skill Creator · la skill oficial de Anthropic

- M1.1 · Qué es Skill Creator
- M1.2 · Cómo se instala y cómo se invoca
- M1.3 · Tu primera skill personalizada en 5 minutos
- M1.4 · Por qué Skill Creator entra primero al curso

M2 — Superpowers · pensar como senior

- M2.1 · El problema número uno de Claude Code — la prisa
- M2.2 · Qué es Superpowers
- M2.3 · Las sub-skills clave de Superpowers
- M2.4 · Instalación y configuración de Superpowers
- M2.5 · Demo — el mismo endpoint con y sin Superpowers

M3 — GSD · sesiones que sobreviven entre chats

- M3.1 · Qué es GSD y por qué nació
- M3.2 · Qué es GSD operativamente y la historia del rug-pull del token \$GSD
- M3.3 · Las primitivas clave del protocolo
- M3.4 · Instalación y configuración de GSD desde el fork
- M3.5 · Demo — la misma pieza retomada con GSD y sin GSD

M4 — /review y /ultrareview · revisión asistida

- M4.1 · Por qué la revisión humana no escala con Claude Code
- M4.2 · /review — el comando estándar
- M4.3 · /ultrareview — la versión profunda
- M4.4 · Demo — un PR real revisado por humano, /review y /ultrareview

M5 — Context Mode · comprimir para durar más

- M5.1 · El problema de la saturación del contexto
- M5.2 · Qué es Context Mode operativamente
- M5.3 · Instalación y primer uso de Context Mode
- M5.4 · Demo — la misma sesión con y sin compresión

M6 — claude-mem · memoria persistente entre proyectos

M6.1 · El problema de la memoria que no se transfiere entre proyectos

M6.2 · Qué es claude-mem operativamente

M6.3 · La mecánica de captura e inyección

M6.4 · Instalación y configuración de claude-mem

M6.5 · Demo — el arranque del mismo proyecto nuevo con y sin claude-mem

M7 — Tu flujo y tu próxima skill

M7.1 · El flujo combinado de las seis skills

M7.2 · Demo end-to-end — un sprint con las seis skills activas

M7.3 · Cómo decidir qué skill personalizada construir tú a continuación

M7.4 · Errores comunes al adoptar el catálogo entero

M7.5 · Guía de próximos pasos a tres meses

PARTE 0

Por qué este curso (criterio y filtros)

M0.1 · De qué va este curso, en una frase

Bloque 0 · Con qué llegas

Si te has apuntado a este curso, probablemente ya tocas Claude Code. Sabes lo que es una sesión, qué pasa cuando escribes `/plan`, has tocado un `CLAUDE.md` al menos para probar. No estás buscando una toma de contacto. Estás buscando dejar de probar y empezar a tener un catálogo de skills sobre el que de verdad puedes apoyarte.

Si entras pensando "a ver qué skills nuevas me cuentan", aquí no va a haber muchas sorpresas en la lista. Las seis que vamos a ver están en YouTube y en GitHub desde hace meses; algunas tienen decenas de miles de estrellas y las habrás visto pasar. La diferencia que aporta este curso no es presentártelas como descubrimientos. Es **decir por qué estas seis y no las otras**, cómo encajan entre sí, y qué criterio aplicar a la siguiente que aparezca el mes que viene cuando la moda haya cambiado.

Y la otra mitad del curso, la que no se ve en el título, es **cómo construir las tuyas**. Porque por bien curadas que estén las seis, ninguna sabe nada de tu trabajo concreto. Las skills personalizadas las pones tú, y lo bueno es que la primera de las seis es justo la que te ayuda a ponerlas.

Bloque 1 · El problema real

A fecha de mayo de 2026 hay más de mil skills disponibles para Claude Code repartidas entre el repositorio oficial de Anthropic, los plugins de la comunidad y mil repos sueltos en GitHub. Mil. Y eso era ayer; la semana que viene serán más.

[!CAPTURA] **ID:** TOPSKL-CAP-M0.1-01 **Qué mostrar:** captura del registry comunitario de skills (`claude-skill-registry` en GitHub o `skills-registry-web.vercel.app`) mostrando el contador "1000+ skills" o equivalente. **Por qué aquí:** el alumno necesita ver con sus ojos que la cifra "más de mil" no es retórica — es contador real visible en la web. **Tipo:** pantallazo de navegador. **Recorte sugerido:** cabecera del registry con cifra total + primera fila de categorías.

Si te has metido en ese catálogo más de cinco minutos, conoces el patrón. Un creador sube un vídeo enseñando una skill que parece resolver un problema real. Vas al repositorio, tiene mil estrellas, README aceptable, comando de instalación. Te la instalas. Funciona en la demo del

vídeo. Y dos semanas después, cuando vuelves a intentar algo parecido en un proyecto real, descubres tres cosas: el repositorio lleva tres meses sin commits, hay seis issues abiertos sin respuesta, y la skill se rompe cuando le pasas un caso que el creador no había previsto.

[!CAPTURA] **ID:** TOPSKL-CAP-Mo.1-02 **Qué mostrar:** ejemplo real (anonimizado o ficticio) de la página de un repo con las tres señales de abandono: barra "Last commit 3 months ago", contador "23 open issues" con flag visible, README con instrucciones desfasadas. **Por qué aquí:** anclar la escena descrita en el párrafo con una imagen que el alumno reconozca visualmente. Refuerza el "tres cosas" del texto. **Tipo:** pantallazo de navegador con anotaciones (3 flechas señalando las 3 señales). **Recorte sugerido:** bloque superior del repo con stats + lista de issues.

No es mala fe del creador. Es la economía del ecosistema. Una skill se publica en un fin de semana, gana atención durante una semana, y muere en silencio cuando el creador pasa a la siguiente cosa que le hace ilusión. Eso es el ochenta o noventa por ciento del catálogo. Y mientras tú intentas distinguir las que sí van a aguantar de las que no, pierdes tiempo en instalaciones, configuraciones, intentos de reparar lo que ya no se mantiene.

El problema, en una frase: **no es la falta de skills, es el exceso**. Cualquier persona puede encontrar veinte vídeos en YouTube esta tarde sobre skills "imprescindibles" de Claude Code. Lo que cuesta encontrar es el filtro.

Bloque 2 · Lo que NO es este curso

Antes de decir qué es, toca cerrar tres puertas. Si esperas alguna de estas tres cosas, este curso no las trae:

No es un repaso enciclopédico. No vamos a recorrer las mil skills disponibles ni siquiera las cien más estrelladas. Vamos a seis. La elección de seis está hecha a propósito para que cada una se vea con detalle, se entienda por qué entra y se pueda combinar con las demás. Si querías un catálogo amplio, hay listas en GitHub que cumplen mejor esa función. Aquí vas a tener selección estrecha y profunda.

No es una lista de YouTube en vídeo largo. No vamos a abrir un repositorio detrás de otro contándote en qué se diferencia cada uno. Cada skill que entra al curso lleva su submódulo con qué problema resuelve, cómo se instala, una demostración, errores típicos, y combinación con las demás. No es ranking; es estudio.

No es un curso sobre cómo programar skills desde cero. Asumimos que Skill Creator existe y que en el primer módulo aprendes a usarlo. No vamos a meternos en cómo escribir el

archivo `skill.md` a mano, ni en la sintaxis de los activadores, ni en el formato exacto que Anthropic espera. Esa información está en su documentación oficial y dura más en pie que cualquier cosa que yo escriba aquí.

Tres cosas que el curso no es. Si lo que buscas era una de las tres, hay otros sitios mejores para encontrarlo.

Bloque 3 · Lo que Sí es

Lo que el curso sí promete cabe en una frase que vamos a repetir varias veces:

Selección curada de las seis skills de Claude Code que aguantan en producción real, criterio explícito para evaluar la siguiente, y un patrón para construir las tuyas propias con la misma cabeza.

Léela con calma porque cada parte importa.

Selección curada. No son las seis más estrelladas, ni las más recientes, ni las que más vídeos generan en YouTube. Son las seis que pasan un filtro concreto que vamos a hacer explícito en Mo.3. Si una skill no pasa los cuatro criterios del filtro, fuera. Si una mejor candidata apareciera mañana, entraría — pero eso ya sería la versión 2 del curso. La de hoy se queda con estas seis y se compromete a justificar cada una.

Aguantan en producción real. No "funcionan en demo", no "el creador las usa en su canal". Aguantan en proyectos donde el código se va a mantener durante meses, donde otros miembros del equipo van a tocar lo que el agente generó, donde un bug en silencio cuesta dinero o reputación. Por ahí va el filtro.

Criterio explícito para la siguiente. Esta es la parte que envejece mejor del curso. Las seis skills concretas pueden cambiar de nombre, de mantenedor o de comando entre que grabamos este vídeo y tú lo ves. Lo que no cambia es el método con el que se seleccionaron. En Mo.3 te entrego los cuatro filtros para que apliques el mismo criterio a las skills nuevas que aparezcan en los próximos meses. Cuando el ecosistema mute, sabrás qué hacer.

Patrón para construir las tuyas. Las seis generales sirven a cualquiera. Pero el cincuenta por ciento del valor de Claude Code en tu trabajo concreto está en las skills que sabes hacer para tu día a día, no para el del vecino. Para eso entra Skill Creator en M1, que es **la pieza que multiplica las demás**. Sin Skill Creator, el catálogo es el catálogo. Con Skill Creator, tu catálogo es el general + el que tú construyas.

Bloque 4 · La promesa de las seis, en cuatro líneas

Antes de cerrar este primer submódulo, te dejo el mapa de las seis para que sepas dónde vamos:

- **Skill Creator** (M1) — la skill oficial de Anthropic que crea skills nuevas desde lenguaje natural. La que habilita el resto.
- **Superpowers** (M2) — plugin con muchas sub-skills que fuerza al agente a planificar, aislar, escribir tests y revisarse en dos fases. Lo que cambia un dev junior por un dev senior dentro del agente.
- **GSD** (M3) — combate el *context rot* lanzando subagentes con contexto limpio para cada tarea. Tu sesión principal queda como orquestador.
- **/review** + **/ultrareview** (M4) — los dos comandos integrados que ya vienen en Claude Code y que la mayoría no sabe que existen.
- **Context Mode** (M5) — filtra el ruido que las herramientas escupen al contexto y reconstruye el estado exacto cuando Claude tiene que compactar.
- **claude-mem** (M6) — memoria persistente entre sesiones, para que abrir Claude Code el lunes no signifique explicarle el proyecto otra vez.

En M7 las combinamos en un flujo y te dejamos el criterio para la siguiente.

Esa es la promesa.

Bloque 5 · Cierre y enlace a M0.2

Una idea para llevarte de este primer submódulo. Si solo retienes una cosa de los próximos minutos, que sea esta: **filtrar es la parte que cuesta, y es la parte que el catálogo público no hace por ti**. Este curso te ayuda a tener criterio para hacerlo. Saber qué se llama Superpowers o cómo se instala lo encuentras en Google. Lo que cuesta encontrar es por qué Superpowers sí y los otros plugins con nombres parecidos no.

En el siguiente submódulo (M0.2) hacemos una distinción que va a recorrer el curso entero: hay dos tipos de skills, **generales** y **personalizadas**, y si confundes para qué sirve cada una, vas a esperar del catálogo público algo concreto de tu trabajo que el catálogo no puede saber. Esa distinción es la que separa al que se queja del catálogo del que lo usa.

Glosario rápido del bloque

- **Skill** (en Claude Code) — herramienta que se le da al agente: procedimiento + activadores + contexto que se carga cuando se invoca.

- **Skill personalizada** — la que construye cada usuario para sus flujos repetitivos. No la trae el catálogo público.
- **Catálogo público** — repositorio oficial de Anthropic + plugins de la comunidad + repos sueltos de GitHub. A fecha de mayo de 2026, más de mil candidatos.
- **Producción real** — proyecto que vive más de unas semanas, lo mantiene más de una persona, y donde un bug en silencio cuesta dinero o reputación. El listón del curso.

Qué viene en M0.2

Distinción entre skills generales y skills personalizadas. Por qué la confundes y por qué te bloquea. Cómo las del catálogo público no te van a cubrir nunca lo concreto de tu día a día, y por qué eso es bueno, no malo. Cinco minutos antes de meternos en el criterio de selección del M0.3.

M0.2 · Skills generales vs skills personalizadas

Bloque 0 · Una queja que escucho mucho

Hablando con devs que ya llevan unos meses usando Claude Code, una queja se repite. Llega más o menos así: *"He probado un montón de skills del catálogo público y al final ninguna me sirve para lo que de verdad hago. Acabo escribiendo los prompts a mano cada vez igual que antes."*

La queja es honesta y entiendo de dónde sale. Pero esconde una confusión que toca nombrar antes de seguir con el curso: **el catálogo público no está pensado para cubrir lo que tú haces concretamente**. Está pensado para cubrir lo que cualquier persona usando Claude Code puede necesitar. Son dos cosas distintas. Si esperas del catálogo lo segundo y te encuentras con lo primero, vas a salir frustrado siempre — no porque el catálogo sea malo, sino porque esperabas de él algo que por diseño no puede dar.

Para que el resto del curso encaje, los próximos diez minutos los dedicamos a separar las dos categorías y a dejar claro el reparto entre lo que te traen y lo que tienes que poner tú.

Bloque 1 · Qué es una skill general

Una skill general es la que sirve igual a cualquier persona que use Claude Code, sin importar a qué se dedique, en qué empresa trabaje o qué stack maneje. Cubren problemas transversales del uso del agente, no del trabajo concreto.

Tres ejemplos para que la idea quede sujeta:

Revisar código antes de cerrar una pieza es algo que cualquiera necesita, da igual si el código es Python, C#, Go o TypeScript. Para eso valen `/review` y `/ultrareview` del M4. La utilidad no depende de tu dominio.

Mantener viva la sesión cuando llevas tres horas trabajando es problema universal de Claude Code. Pasarte de los noventa minutos sin que el agente empiece a olvidar lo del principio es algo que todos sufrimos. Para eso entran GSD (en su versión Redux, lo verás en el M3) y Context Mode del M5. De nuevo, da igual tu proyecto.

No perder el contexto entre sesiones de días distintos es otra cosa que pasa a todo el mundo. Para eso está `claude-mem` del M6. El problema no es específico de tu negocio: es de cómo está pensado el ciclo de Claude Code.

Cinco de las seis skills del curso son **generales en este sentido**. Cubren dolores que toda persona usando Claude Code de forma seria va a tener en algún momento. Por eso entran al catálogo curado: no porque sean las que más me gustan a mí, sino porque son las que aguantan el filtro de "cualquiera se beneficia".

La sexta — Skill Creator del M1, oficial de Anthropic — es distinta. No es una skill general más; es la pieza con la que tú vas a construir tus skills personalizadas. Cubre el otro lado de la frontera, no este. Por eso la tratamos aparte cuando hablamos del catálogo.

La consecuencia importante: las skills generales son las únicas que el catálogo público va a poder mantener. Tienen base de usuarios amplia, tienen mantenedores con incentivo para no abandonarlas, tienen feedback constante. Una skill general bien hecha vive años; las que están mal hechas mueren pronto, y eso es justo lo que el filtro del Mo.3 detecta.

Bloque 2 · Qué es una skill personalizada

Una skill personalizada es la opuesta. Sirve para algo que solo tú haces, o que hace tu equipo, o que hace tu empresa. Cubre flujos repetitivos del trabajo concreto, no del uso del agente.

Pongamos ejemplos pegados a la tierra para que se vea la diferencia.

Análisis después de cada llamada con cliente. Si grabas tus llamadas con clientes (Fathom, Granola o lo que uses), procesar la transcripción, sacar los pasos siguientes, actualizar tu CRM y meter recordatorios en tu calendario es algo que probablemente haces varias veces por semana. Una skill personalizada toma la grabación o la transcripción, ejecuta el procedimiento tuyo (no el de cualquiera), y deja todo donde tú lo necesitas. Ningún catálogo público te va a traer esto porque depende de tus herramientas, de tu CRM, de tu plantilla de seguimiento.

Generar la propuesta comercial con tu plantilla del equipo. El formato, los precios, los argumentos, los proyectos parecidos que has hecho antes — eso es información tuya. Una skill personalizada toma cuatro datos del cliente nuevo, los cruza con tu base de propuestas anteriores, y genera el documento listo para revisar y enviar. Mismo motivo: nada en el catálogo público sabe nada de tu plantilla.

Plantilla de pull request con las convenciones de tu equipo. Tu equipo seguramente tiene su forma de redactar la descripción del PR, las secciones que esperáis, los reviewers que tocan según el área del código. Una skill personalizada coge el diff actual, identifica el área, propone el título y el cuerpo siguiendo vuestra convención. No va a venir hecho de fuera porque la convención es vuestra.

Checklist pre-deploy específica de tu sistema. Cada sistema en producción tiene su lista: que la migración esté revisada, que los flags estén configurados, que el canal de soporte esté avisado. Una skill personalizada repasa la checklist tuya, no la de un sistema genérico.

Lo que comparten estos cuatro ejemplos: ninguno se puede empaquetar para distribuir al público porque dependen de información que solo tú tienes. Y a la vez, los cuatro son **repetitivos**. Los haces varias veces por semana o por mes. Por eso son candidatos perfectos para convertirse en skill personalizada: el coste de crearla una vez se amortiza en las siguientes diez ejecuciones. Los desarrollamos como casos prácticos en M7.3, cuando ya tengas Skill Creator instalado y entendido.

Bloque 3 · Por qué se confunden los dos tipos

La confusión típica es esperar que el catálogo público te cubra los flujos personalizados. *"Tiene que haber una skill que haga X."* Y X es algo específico de tu trabajo. No la encuentras, te quejas del catálogo, vuelves a hacerlo a mano.

¿Por qué se confunden? Por tres motivos que se mezclan:

Las demos de YouTube enseñan casos que parecen generales pero son personales del creador. Alguien sube un vídeo titulado "Mi skill para automatizar emails". Lo enseña bonito. Tú piensas "yo también quiero esto". Pero esa skill está atada a su Gmail, su flujo de etiquetas, su forma de responder. Cuando la instalas, te falta la mitad del contexto que el creador tenía implícito en su máquina. Lo que el creador llama "skill general" es en realidad su skill personalizada que está enseñando — la diferencia se pierde en el título.

Los catálogos públicos están llenos de skills que se anuncian como universales pero que son demos. Una skill llamada "Procesador de facturas" suena general. Y lo es para el caso del repositorio de ejemplo. En cuanto le pasas tu factura, con tu formato, tu IVA, tu manera de numerar, falla. Era prueba de concepto disfrazada de utilidad.

El propio agente puede dar la falsa impresión de que con la skill general te apaña lo personal. Le sueltas un prompt complejo a una skill genérica de "documento profesional" y a veces sale algo aceptable. Pero hacerlo así, prompt largo a mano cada vez, es no aprovechar lo que el sistema te ofrece. Has hecho a mano lo que una skill personalizada haría sola con tres datos de entrada.

El resultado del enredo es siempre el mismo: frustración con el catálogo, conclusión de que "estas skills son humo", abandono del intento de subir el listón en tu uso de Claude Code. Y el problema no estaba en el catálogo — estaba en lo que esperabas que cubriera.

Bloque 4 · El reparto natural del trabajo

Una vez separadas las dos categorías, queda claro el reparto.

Las skills generales las eliges, no las construyes. Hay decenas de candidatas en el catálogo público y tu trabajo es seleccionar las que entran a tu setup. Para eso vale el criterio del M0.3, y por eso este curso te da cinco ya filtradas como base — más Skill Creator, que es de otra naturaleza. Ahorras meses de probar y descartar. A partir de ahí, cuando aparezca una candidata nueva (y aparecerán, este ecosistema no para), aplicas el mismo criterio.

Las skills personalizadas las construyes, no las eliges. No las hay en el catálogo público y no las habrá nunca, porque lo personalizado por definición no se generaliza. La parte fácil: construirlas se ha vuelto barato gracias a Skill Creator (oficial de Anthropic, M1), que es la pieza que cubre justo este lado. La parte que no se delega: el trabajo lo tienes que hacer tú, no hay atajo.

[!CAPTURA] **ID:** TOPSKL-CAP-M0.2-01 **Qué mostrar:** diagrama dos columnas. Columna izquierda "Generales — las eliges": cinco iconos representando las 5 skills generales del catálogo (Superpowers, GSD, /review+/ultrareview, Context Mode, claude-mem). Columna derecha "Personalizadas — las construyes": cinco rectángulos con los 5 casos del hilo conductor (PR, post-call, propuesta, release notes, pre-deploy). Una flecha desde Skill Creator (centro-arriba) hacia la columna derecha. **Por qué aquí:** la distinción es el eje del submódulo. Una imagen consolida la frontera mejor que mil palabras de texto. **Tipo:** diagrama (puede hacerse en Excalidraw o Mermaid). **Recorte sugerido:** vista completa del diagrama, una página entera.

El reparto se ve mejor con un número. Si trabajas en serio con Claude Code, mi estimación informal es que el cincuenta por ciento del valor de tu setup va a venir de las skills generales bien elegidas, y el otro cincuenta por ciento de las personalizadas que tú construyas. Esa mitad personalizada nadie te la va a dar. Ni este curso, ni el siguiente, ni el catálogo público que veas en el futuro.

Y esa es buena noticia, no mala. Significa que tu trabajo concreto, tu forma de hacer las cosas, tu CRM, tus plantillas, tu checklist de deploy — todo eso queda fuera del agente que viene en una caja y queda dentro del agente que tú ajustas. Tu setup acaba siendo tuyo. Quien no entiende este reparto se queja de que el ecosistema no le da lo que necesita; quien sí lo entiende usa el catálogo para lo que vale y construye lo demás.

Bloque 5 · Cierre y enlace a M0.3

Lo que te llevas de este submódulo es la frontera entre los dos tipos: **lo general lo eliges, lo personalizado lo construyes**. Las cinco generales del curso entran tras pasar un filtro; la sexta, Skill Creator, es la herramienta con la que tú construyes las personalizadas. Es la primera que veremos en detalle precisamente porque habilita el otro lado de la frontera.

En el siguiente submódulo (M0.3) bajamos al criterio concreto. Cuatro filtros que las seis del curso pasan y la mayoría no. Cuatro filtros que tú vas a aplicar a la próxima candidata que aparezca en GitHub en los próximos meses, cuando el ecosistema se mueva otra vez. Ese criterio es lo que envejece bien del curso — las skills concretas pueden renombrarse, los cuatro filtros se quedan.

Glosario rápido del bloque

- **Skill general** — sirve a cualquier persona usando Claude Code, cubre dolores transversales del uso del agente. Vive en el catálogo público; tú la eliges, no la construyes.
- **Skill personalizada** — sirve solo para tus flujos concretos (tu CRM, tu plantilla, tu checklist). No vive en el catálogo público porque depende de tu contexto; tú la construyes, no la eliges.
- **Flujo repetitivo** — tarea digital que haces varias veces por semana o por mes con el mismo patrón. Candidato natural a skill personalizada.
- **Reparto cincuenta-cincuenta** — estimación informal del peso relativo. La mitad del valor de un setup serio viene de seleccionar bien las generales; la otra mitad de construir bien las personalizadas.

Qué viene en M0.3

El criterio concreto para evaluar una skill general. Cuatro filtros con nombre y rúbrica que aplicamos a las cinco del curso para que veas por qué entran. Y te quedas con la herramienta para evaluar la siguiente cuando aparezca, que es lo que de verdad sobrevive cuando el ecosistema mute.

M0.3 · El criterio que aplicamos — cuatro filtros

Bloque 0 · Por qué necesitas tu propio filtro

Llevamos dos submódulos diciendo que el catálogo público tiene mucho ruido y poca señal. Está bien identificar el problema; ahora toca darte la herramienta para resolverlo en cualquier momento, no solo para las seis del curso.

La herramienta es un filtro propio. Cuatro preguntas sencillas que vas a aplicar a cualquier skill antes de instalarla. Las contestas mentalmente en treinta segundos y descartas la gran mayoría del catálogo. Las que pasan los cuatro filtros entran a una segunda categoría que merece tu tiempo: las pruebas durante una semana en un proyecto real y decides.

¿Por qué cuatro y no diez? Por dos cosas que se refuerzan. La primera es práctica: cualquier filtro más largo deja de aplicarse después de la tercera vez. La segunda es de cobertura: estos cuatro cazan la gran mayoría de los problemas que aparecen después. Hay matices que se podrían añadir, pero meterlos en el filtro de entrada los convierte en barrera para todo el mundo cuando el problema solo afecta a casos concretos.

Vamos a ver los cuatro de uno en uno, con el ejemplo del catálogo en la cabeza para que se entiendan en contexto.

Bloque 1 · Filtro uno — viva

¿La skill está viva?

Tres señales para responder en treinta segundos sin entrar a leer el código:

El último commit es reciente. Como regla aproximada, reciente quiere decir tres meses o menos para una skill activa. Si la fecha del último cambio es de hace siete meses, no significa automáticamente que esté muerta, pero es señal de que el mantenedor ha pasado a otras cosas. Si pasa de un año, salvo casos muy concretos, considéralo abandonado.

Los issues abiertos tienen respuesta. Abres la pestaña de issues, miras los cinco últimos. Si están todos sin contestar y llevan semanas así, el repositorio no está activo. Si hay conversación, aunque sea para decir "lo miraré la semana que viene", el proyecto vive. Una skill con cien issues abiertos y cero respuestas es más sospechosa que una con cinco issues bien gestionados.

El README dice lo que el proyecto hace hoy, no lo que decía hace dos años. Lo más sutil de los tres pero importante. Las skills que mutan rápido a veces dejan el README obsoleto: instrucciones que ya no funcionan, comandos renombrados, capturas viejas. Si lo que ves en el README no coincide con lo que ves al instalar, el proyecto cojea aunque tenga estrellas.

La regla informal: si las tres señales fallan, descarta sin probar. Con dos de tres funcionando, anótala para una segunda ronda más calmada. Si las tres salen bien, ya tienes media batalla; pasa al filtro siguiente.

Bloque 2 · Filtro dos — adoptada

¿La skill la usa gente real, no solo bots?

Aquí entra el número de estrellas, pero con matiz. Las estrellas son señal débil porque hay maneras de inflarlas y porque hay gente que estrella todo lo que abre.

El número solo no basta. Una skill de Claude Code con quinientas estrellas ya es proyecto vivo; una con veinte estrellas puede ser una buena idea que necesita más tiempo. La pregunta no es "¿pasa el umbral mágico?" sino "¿la gente ha llegado a este repo y le ha parecido útil?". Quinientas estrellas con conversación en issues vale más que cinco mil estrellas en silencio.

Importa también cómo llegaron esas estrellas. Las herramientas tipo *star-history* enseñan la curva temporal. Una curva limpia que crece despacio durante meses es señal de adopción real: gente que descubre el proyecto, lo prueba, lo recomienda. Una subida brusca en un día concreto, normalmente coincidiendo con un vídeo en YouTube o un post viral, y luego silencio, es señal de moda corta. No descarta el proyecto, pero te dice que mucha de esa atención fue espuma.

[!CAPTURA] **ID:** TOPSKL-CAP-Mo.3-01 **Qué mostrar:** dos curvas de star-history lado a lado. Izquierda: curva sana — crecimiento gradual sostenido durante meses (ejemplo real recomendado: Superpowers). Derecha: curva de moda — subida vertical en un día y luego plana o decreciente (cualquier skill que cumpla el patrón sin nombre comprometedor). **Por qué aquí:** el alumno entiende "curva limpia vs subida brusca" mejor viéndolo. Es el filtro adoptada hecho visual. **Tipo:** pantallazo de star-history.com con dos repos comparados. **Recorte sugerido:** vista de comparativa con eje temporal visible.

Y por último, dónde se usa la skill fuera del propio repo. Si encuentras gente referenciándola en otros proyectos, en hilos de discusión, en respuestas a preguntas reales, eso es adopción seria. Si solo hay menciones del vídeo del creador y tres tuits, la skill todavía no ha llegado a usuarios externos.

La estrella es punto de partida, no punto final. Dos skills con el mismo número de estrellas pueden tener calidad de adopción muy distinta.

Bloque 3 · Filtro tres — reversible

Si la instalo y no me convence, ¿puedo desinstalarla sin daño?

Este es el filtro que más subestimás al principio. Las skills mal pensadas tocan demasiadas cosas: modifican archivos del proyecto, dejan configuraciones residuales, cambian el comportamiento de comandos estándar, instalan dependencias globales que arrastran efectos. Y cuando intentas quitarla, te das cuenta de que el proceso tiene más cabos sueltos de lo que parecía.

Tres preguntas concretas:

¿Toca solo lo que dice tocar? El alcance de una skill bien hecha es explícito: actualiza `CLAUDE.md`, añade unos archivos en una carpeta concreta, registra unos activadores. Una skill mal hecha puede modificar archivos del proyecto, sobrescribir configuraciones existentes, dejar comandos personalizados que no formaban parte de la promesa.

¿La desinstalación está documentada? Mira el README. Si no menciona cómo desinstalar, o la mención se reduce a "elimina la carpeta", es señal de que el creador no se ha planteado el ciclo de vida completo. Las skills serias documentan tanto la instalación como la retirada.

¿Su efecto en `CLAUDE.md` es limitado y reversible? Las skills suelen añadir entradas al `CLAUDE.md` para que el agente sepa cuándo invocarlas. Si lo que añade es una sección concreta etiquetada, perfecto: la borras y queda como antes. Si reescribe partes amplias del archivo o mezcla su contenido con el resto, dejás instalada una huella que cuesta limpiar.

La razón por la que este filtro importa tanto es asimétrica. Probar una skill reversible te cuesta lo que dura la prueba: si no te convence, la quitas y se acabó. Probar una que no es reversible, en cambio, deja rastros que te van a confundir meses después aunque el proyecto parezca el mismo. Ante la duda, exige reversibilidad.

Bloque 4 · Filtro cuatro — compone

¿La skill encaja con otras skills y con tu flujo, o es una isla?

Una skill puede pasar los tres filtros anteriores y aun así no entrar al curso si no compone con el resto de tu setup. La idea de "compone" es importante porque las skills no se usan aisladas: las combinas, las encadenas, las activas en orden. Si una skill solo funciona como pieza única que no habla con nada, está limitando lo que puedes construir.

Una skill que compone bien hace una cosa y la hace clara. Las que intentan cubrir cinco responsabilidades a la vez encajan mal con cualquier otra que quieras meter; las que se centran en un objetivo concreto y delimitado son las que luego se combinan en flujos.

Importa también que sus entradas y salidas sean las que el ecosistema espera. Si una skill genera resultados en un formato propio que ninguna otra entiende, la has aislado de antemano. Las que componen bien toman el contexto de la sesión y dejan resultados en sitios que el resto del agente o de las otras skills pueden consumir.

Y por último, que no se pise con responsabilidades de otras del catálogo. Si dos skills hacen casi lo mismo de manera ligeramente distinta, vas a tener que elegir y descartar una. Las que pasan al curso ocupan su nicho sin pelearse con sus vecinas.

El test mental rápido: si te imaginas tu flujo de trabajo con tres o cuatro skills activas a la vez, ¿esta nueva candidata se encaja sin chocar, o tendrías que rehacer la lógica de las demás? Si es lo segundo, no entra.

Bloque 5 · Cómo se aplica en treinta segundos

Para cerrar, pongamos los cuatro juntos en acción con el patrón mental que te recomiendo seguir cuando aparezca una skill candidata.

Imagínate que mañana ves un vídeo en YouTube de alguien presentando una skill llamada cualquier-cosa-súper-útil. El proceso a aplicar antes de instalarla:

[!CAPTURA] **ID:** TOPSKL-CAP-Mo.3-02 **Qué mostrar:** tabla de los 4 filtros con su pregunta nuclear, tiempo de aplicación y veredicto rápido. Formato: 4 filas × 4 columnas (Filtro / Pregunta / Tiempo / Cómo veo señal verde). **Por qué aquí:** cierra el submódulo con un resumen visual que el alumno puede usar como tarjeta de referencia. Útil también como diapositiva de cierre del módulo Mo. **Tipo:** tabla / infografía. **Recorte sugerido:** tabla completa centrada, una página.

Treinta segundos en el repositorio para el filtro uno: el último commit, los issues abiertos, el README.

Otros treinta segundos en estrellas y menciones para el filtro dos: número, curva temporal si la herramienta te lo deja ver rápido, búsqueda del nombre fuera del propio repositorio.

El filtro tres pide más rato: dos minutos en el README y el archivo principal mirando cómo se instala, cómo se desinstala, qué toca.

El cuarto es el más caro mentalmente — cinco minutos imaginando la skill funcionando en tu setup actual junto a las que ya tienes. ¿Encaja, choca, sobra?

Total: menos de diez minutos antes de instalar nada. Si pasa los cuatro, la pruebas durante una semana en un proyecto real. Si no, descartada y a otra cosa.

La regla práctica final: **si dudas en alguno de los cuatro filtros, descarta**. Hay más skills de las que vas a poder probar; el filtro está hecho para descartar rápido, no para investigar a fondo. Las dudas las resuelves con las próximas que pasen sin dudas.

Bloque 6 · Cierre y enlace a M0.4

Los cuatro filtros aguantan aunque las skills concretas cambien. El catálogo va a mutar; tu criterio se queda igual. Aplícalo a cualquier candidata nueva con la misma cabeza.

Las seis skills del curso pasan los cuatro filtros. En el M1 vas a ver Skill Creator y eso será evidente. Pero quería que el filtro te quedara antes de presentar la primera, no después, para que veas las seis con el criterio en la cabeza y puedas estar de acuerdo o discrepar con razones.

Y recuerda que es asimétrico: descartar algo bueno se arregla en dos minutos cuando alguien más serio que tú lo recomienda. Instalar algo que no aguantaba puede costarte tiempo de limpieza y un proyecto sucio.

En el siguiente submódulo (M0.4) cerramos el módulo cero con cuatro avisos prácticos antes de meternos en la primera skill. Cosas concretas que vale tener claras para que la grabación que estés viendo dure lo más posible sin desfasarse. Después de eso, directos a Skill Creator.

Glosario rápido del bloque

- **Filtro de entrada** — el conjunto de preguntas rápidas que aplicas a cualquier skill antes de invertir tiempo en ella. Cuatro en este curso.
- **Adopción real** — uso visible por parte de personas distintas al creador de la skill. Estrellas no bastan; mejor con curva temporal limpia y menciones externas.
- **Reversibilidad** — propiedad de una skill que se puede desinstalar dejando el proyecto como estaba antes. Insistir en este filtro cuesta poco y ahorra mucho.
- **Composición** — capacidad de la skill de funcionar bien junto a otras y dentro de un flujo. Una skill aislada limita el setup; una que compone lo amplía.

Qué viene en M0.4

Cuatro avisos prácticos antes de meternos en la primera skill: cómo envejece este vídeo concreto que estás viendo, qué hacer cuando un comando que aquí nuestro deje de funcionar, qué hacer cuando una skill cambie de nombre, y cómo se entrega el comando de instalación todo-en-uno (al final del M7, no antes). Dos minutos de preparación y arrancamos M1.

M0.4 · Avisos antes de empezar

Bloque 0 · Por qué este submódulo dura dos minutos

Cierro el Mo con avisos prácticos en lugar de teoría nueva porque después de tres submódulos (la promesa, los dos tipos de skill, los cuatro filtros) lo que queda es bajar a la primera skill. Antes de hacerlo, tres cosas concretas que toca tener claras para que el resto del curso te rinda lo máximo posible aunque pase el tiempo desde la grabación.

Bloque 1 · Aviso uno — el ecosistema muta

Este vídeo está grabado en mayo de 2026 y el ecosistema de Claude Code va a estar distinto dentro de seis meses, posiblemente dentro de tres. Una skill puede cambiar de nombre, de comando de instalación, de mantenedor. Una puede entrar en el catálogo oficial de Anthropic y otra puede pasar a deprecación silenciosa.

¿Qué hacer cuando eso pase con algo que ves en el curso?

Lo primero: el **nombre** de la skill puede mover ficha pero el **problema que resuelve** suele quedarse. Si vas al M2 buscando Superpowers y descubres que ahora se llama de otra forma o ha sido absorbida por una skill más grande, lo que tienes que buscar es lo que hace, no el nombre. La descripción del problema en cada submódulo está pensada para sobrevivir al cambio de etiqueta.

Sobre el **comando de instalación**: lo verifico siempre antes de grabar cada módulo, pero la realidad del catálogo va a su ritmo. Si el comando que mostro no funciona, ve al repositorio oficial de la skill (el enlace está en cada submódulo) y mira el README actual. El comando vigente vive ahí, no en el vídeo.

Y lo más importante: si una de las seis desaparece o muere, el **criterio del Mo.3 te dice qué hacer**. Los cuatro filtros se aplican a cualquier candidata nueva. La skill concreta cambia; tu criterio se queda.

Bloque 2 · Aviso dos — oficial vs comunidad

De las seis skills del curso, no todas vienen del mismo sitio. La diferencia no es de calidad, pero sí cambia las expectativas que toca tener.

Oficiales de Anthropic. En la selección entran piezas que mantiene directamente el equipo de Claude. Tienen ventaja en mantenimiento (no se mueren porque al creador le canse el proyecto) y desventaja en velocidad de evolución (un proyecto oficial cambia menos veces pero cuando cambia tiene impacto). Skill Creator del M1 es el ejemplo claro.

[!CAPTURA] **ID:** TOPSKL-CAP-Mo.4-01 **Qué mostrar:** tabla resumen de las 6 skills del curso clasificadas por origen. Tres columnas: Oficial Anthropic (Skill Creator), De la comunidad (Superpowers, GSD, Context Mode, claude-mem), Integrada en Claude Code (/review + /ultrareview). Cada celda con el nombre canónico y el módulo donde aparece. **Por qué aquí:** el aviso 2 de Mo.4 introduce esta clasificación. Una tabla visual la consolida en una mirada. **Tipo:** tabla / infografía. **Recorte sugerido:** tabla completa, una página.

De la comunidad. Otras vienen de mantenedores individuales o de pequeños equipos que han hecho de su skill una marca. Tienen ventaja en cercanía al uso real (responden rápido a feedback) y desventaja en futuro garantizado (depende de que el mantenedor siga con energía). A fecha de la grabación, Superpowers, GSD, Context Mode y `claude-mem` entran en esta categoría.

Integradas en el propio Claude Code. Hay una que no se instala porque ya viene dentro: los comandos del M4. Es una categoría de una, pero merece mención porque su mantenimiento depende del propio producto Claude Code, no de un repositorio externo.

En cada submódulo verás explícitamente de qué categoría es la skill que toca, porque cambia las expectativas de mantenimiento que conviene tener.

Bloque 3 · Aviso tres — el comando todo-en-uno llega al final

Si has venido al curso esperando que en los primeros cinco minutos te dé un script que instale las seis de golpe, paciencia: el comando todo-en-uno está al final del M7, no antes. La razón es deliberada.

Si te lo entrego ahora, vas a copiarlo, pegarlo, instalar las seis, y empezar a usarlas sin haber visto qué hace cada una ni por qué entra al curso. El resto del material te va a parecer relleno y vas a saltarte la mitad. Cuando una de las seis se comporte de manera rara, no vas a saber distinguir si es la skill, si es tu configuración o si es interacción entre dos de ellas.

Si lo entrego al final, después de haber visto cada una en su submódulo, vas a tener el modelo mental para usar el comando con conocimiento. La instalación va a tardar dos minutos extras de copiar y pegar, pero los próximos meses con esas seis activas van a ser mucho más productivos..

Entre tanto, en cada submódulo tienes el comando de instalación de la skill correspondiente. Si quieres ir instalando una a una mientras avanzas, perfecto. Pero el script consolidado es el cierre, no la apertura.

Bloque 4 · Cierre del M0 y enlace a M1

El catálogo público resuelve un tipo de problema y tú resuelves el otro. Skills generales eliges; skills personalizadas construyes. Si lo confundes, te frustras con el catálogo por algo que no le toca cubrir.

Los cuatro filtros (viva, adoptada, reversible, compone) son la herramienta para cuando el ecosistema mute. Las seis del curso los pasan; aplica los mismos a cualquier candidata nueva.

El curso es honesto sobre qué envejece y qué no. Lo que envejece: nombres, comandos, números concretos. Lo que se queda: el criterio, el reparto entre general y personalizada, la lógica de combinación que veremos en M7.

En el siguiente módulo entramos en Skill Creator, la primera de las seis. No es la más espectacular, pero es la que habilita el resto. Sin ella, este curso te entrega seis skills generales y nada más; con ella, te entrega seis generales y la herramienta para construir todas las personalizadas que necesites.

Nos vemos en M1.1.

Glosario rápido del bloque

- **Skill oficial** — la mantiene directamente el equipo de Anthropic. Estable en el tiempo, evoluciona despacio.
- **Skill de comunidad** — la mantienen individuos o pequeños equipos. Cercana al uso real, futuro menos garantizado.
- **Skill integrada en Claude Code** — viene dentro del producto, no se instala. Su mantenimiento depende de Claude Code mismo.
- **Comando todo-en-uno** — script consolidado que instala las seis skills de golpe. Se entrega al final del M7, no al principio del curso, para que sepas qué estás instalando.

Qué viene en M1

Skill Creator. Qué es y qué la diferencia del resto del catálogo: es la skill oficial de Anthropic que crea nuevas skills desde lenguaje natural. Por qué entra primero al curso (porque habilita la

mitad personalizada del valor que vimos en Mo.2). Cómo se instala, cómo se invoca, qué actualiza por dentro. Una demo de cinco minutos donde construimos una skill personalizada útil partiendo desde un prompt en lenguaje natural. Y por qué, después de instalar Skill Creator, ya estás en una versión distinta de Claude Code aunque las otras cinco del curso no las hayas instalado todavía.

PARTE 1

Skill Creator · la skill oficial de Anthropic

M1.1 · Qué es Skill Creator

Bloque 0 · Por qué entra primero

Si el Mo te convenció de que el cincuenta por ciento del valor de tu configuración va a venir de las skills personalizadas que tú construyas, la pregunta obvia es cómo se construyen. Y hasta hace unos meses la respuesta era "tienes que escribir markdown a mano siguiendo un formato que cambia cada poco, con sus activadores, sus secciones, sus pruebas". Para devs era aburrido; para no-devs era barrera de entrada.

Skill Creator cambia eso. Y por eso entra primero al curso, antes que las cinco generales que vienen detrás. Si ves los siguientes cinco módulos sin tener Skill Creator instalado primero, te llevarás cinco buenas skills generales y no podrás construir las personalizadas que cubren tu trabajo concreto. Si ves M1 primero, sales de aquí con la herramienta que te va a permitir hacer ambas cosas: aprovechar las generales y crear todas las personalizadas que necesites.

Toca ver qué es Skill Creator operativamente. El cómo se invoca queda para M1.2; aquí nos centramos en qué es por dentro y por qué tiene la posición que tiene.

Bloque 1 · El kit completo, no solo Create

Si miras el primer vídeo o el primer post que aparece al buscar "Skill Creator", la definición que recibes es: *"skill oficial de Anthropic que te permite crear nuevas skills describiendo lo que quieres en lenguaje natural"*. Eso es cierto y es la pieza más visible, pero deja fuera buena parte de lo que esta pieza hace.

La definición operativa más fiel: **Skill Creator es un kit completo de desarrollo de skills**, que mantiene Anthropic, que cubre desde la idea inicial hasta la skill afinada y lista para producción. Tiene cuatro modos operativos distintos según en qué fase del ciclo estés, y cuatro agentes internos que colaboran entre sí para llevarte por las cuatro fases.

Cuando el vídeo divulgativo dice "crea skills", está describiendo solo uno de los cuatro modos. Los otros tres están todos disponibles en la versión actual y son lo que diferencia un kit serio de un generador de plantillas.

Esa diferencia importa, y mucho. Si vas a construir skills personalizadas que vivan en tu equipo más de una semana, la primera versión nunca es la última: necesitas mejorarla, evaluarla, comparar versiones — y para eso son los otros tres modos. A esto se suma una segunda ventaja: tener todo el ciclo en una sola pieza, bien viva, evita que cada equipo se monte su propio script

para evaluar skills, su propio script para comparar versiones, su propio script para benchmark. Eso ya lo tienes hecho y lo mantiene el mismo equipo que mantiene Claude Code.

Bloque 2 · Los cuatro modos operativos

Skill Creator tiene cuatro modos. Los nombro y explico para qué sirve cada uno; el detalle de cómo se invoca queda para M1.2.

Create. El modo que la mayoría conoce. Le describes lo que quieres en lenguaje natural y la pieza monta la primera versión de la skill: archivo `skill.md` con la estructura correcta, activadores, secciones, ejemplos. La salida es una skill que ya puedes usar.

Eval. Aquí Skill Creator deja de generar y empieza a probar. Le das una skill (la que acaba de salir del modo Create, o una que ya tenías) y una colección de prompts de prueba que describen casos reales. Ejecuta la skill contra cada prompt y te dice qué hace bien y qué hace mal. El objetivo no son los tests sobre el código sino la evaluación del comportamiento de la skill, más cercana a un *eval* de LLM clásico que a un test unitario.

Improve. Cuando el modo Eval te ha enseñado que la skill falla en algunos casos, el modo Improve toma esa información y propone cambios concretos al `skill.md` para que la skill se comporte mejor en los casos que fallaban. No es magia: el modo Improve usa los resultados del modo Eval como insumo. Sin Eval previo, Improve va a ciegas.

Benchmark. El modo más infrautilizado y el más útil cuando vas en serio. Ejecuta la skill múltiples veces con el mismo prompt y mide la varianza de las salidas. Las skills bien hechas producen salidas similares para la misma entrada; las mal hechas tienen mucha varianza, y eso significa que su comportamiento depende del azar más de lo que debería. Benchmark te dice cuánta confianza puedes tener en la skill antes de meterla en tu flujo diario.

Cuatro modos. La mayoría arranca solo con Create. El curso te recomienda usar los cuatro al menos una vez con tu primera skill personalizada; la diferencia de calidad entre la primera versión y la versión iterada con Eval e Improve es notable.

Bloque 3 · Los cuatro agentes internos

Por debajo de los cuatro modos hay cuatro agentes especializados que se reparten el trabajo. Esto no es decoración: ver cómo trabajan por dentro te ayuda a entender por qué Skill Creator es más fiable que un script casero que hiciera lo mismo.

Executor. Ejecuta la skill contra prompts de evaluación. Es la pieza que en modo Eval lanza la skill una y otra vez con los casos de prueba que le das. Aislado, sin lógica de juicio: solo ejecuta y captura las salidas.

Grader. Evalúa las salidas del Executor contra criterios definidos. Aquí entra la parte de "qué se considera bien" según lo que tú especifiques en los prompts. Una skill bien definida tiene criterios explícitos; el Grader los aplica.

Comparator. Hace comparaciones A/B en ciego entre dos versiones de la misma skill. Le pasas la versión anterior y la versión mejorada (modo Improve), y el Comparator decide cuál se comporta mejor para los casos que has definido. Sin sesgo del orden de presentación.

Analyzer. Mira los resultados del Grader y del Comparator y sugiere mejoras concretas para la próxima iteración. Es la pieza que cierra el bucle: Eval te dice qué falla, Analyzer te dice por qué y qué cambiar, Improve aplica los cambios, Benchmark valida.

Los cuatro agentes están coordinados. Tú no los invocas uno a uno — invocas un modo y el modo activa los agentes que necesite. Pero saber que están ahí te ayuda a confiar en el proceso: el sistema reparte el trabajo en piezas con responsabilidades separadas, y los resultados son verificables paso a paso.

Bloque 4 · Qué actualiza por dentro al crear una skill

Cuando ejecutas el modo Create y la skill nueva queda lista, Skill Creator toca dos cosas en tu proyecto:

El archivo de la skill. Crea el `skill.md` correspondiente en la carpeta de skills del proyecto (`.claude/skills/` o equivalente según tu configuración). Ese archivo contiene la descripción de la skill, los activadores, los ejemplos, lo que el agente necesita para invocarla y aplicarla en futuras sesiones.

Tu `CLAUDE.md`. Este es el detalle que se nota más en el día a día. Skill Creator añade una entrada al `CLAUDE.md` del proyecto explicándole al agente qué hace la nueva skill y en qué circunstancias debería invocarla. Sin esa entrada, la skill existe pero el agente no sabe que existe, así que la tendrías que invocar a mano siempre. Con la entrada, el agente la detecta y la usa cuando el contexto lo pide.

La consecuencia práctica: cada vez que creas una skill personalizada, tu `CLAUDE.md` crece un poco. Eso es buena señal: el agente está aprendiendo qué herramientas tienes disponibles. Lo único que toca vigilar: si en seis meses tienes treinta skills personalizadas y tu `CLAUDE.md` ocupa mil líneas, repasa cuáles sigues usando y cuáles puedes retirar.

Bloque 5 · Cierre y enlace a M1.2

Skill Creator es mucho más que un generador de plantillas: un kit de desarrollo completo con cuatro modos y cuatro agentes que cubren creación, evaluación, mejora y benchmark. La mayoría se queda solo con el modo Create; el curso te recomienda usar los cuatro al menos una vez.

Es oficial de Anthropic, vive en el repositorio público de skills y se mantiene al ritmo del propio producto Claude Code. Pasa los cuatro filtros del MO.3 con holgura: la cuida el mismo equipo que mantiene Claude Code, supera las cien mil instalaciones semanales a mayo de 2026, se desinstala como cualquier plugin oficial sin dejar rastro, y compone bien porque actualiza el `CLAUDE.md` de forma estándar.

Es la que entra primero al curso porque habilita la mitad personalizada del valor que vimos en el MO.2. Sin esta pieza, las otras cinco que vienen son buenas pero no cubren lo que tu trabajo concreto necesita. Con esta pieza, sales del curso con la herramienta para construir todas las personalizadas que necesites.

En el siguiente submódulo (M1.2) entramos en cómo se instala y cómo se invoca, paso a paso. Es la parte mecánica: diez minutos y la tienes funcionando en tu Claude Code. Después, M1.3 será una demo construyendo una skill personalizada real desde un prompt en lenguaje natural.

Glosario rápido del bloque

- **Skill Creator** — kit oficial de Anthropic para crear, evaluar, mejorar y comparar skills de Claude Code. Cuatro modos y cuatro agentes internos.
- **Modo Create** — el más conocido. Genera una skill desde una descripción en lenguaje natural.
- **Modo Eval** — ejecuta la skill contra prompts de prueba y te dice qué hace bien y qué hace mal.
- **Modo Improve** — aplica mejoras concretas al `skill.md` basándose en los resultados de Eval.
- **Modo Benchmark** — ejecuta la skill múltiples veces con la misma entrada y mide la varianza de las salidas. Indicador de confianza.
- **Agentes internos** — Executor (ejecuta), Grader (evalúa), Comparator (compara A/B), Analyzer (sugiere mejoras). Tú invocas un modo, el modo activa los agentes.

Qué viene en M1.2

Instalación paso a paso, comando exacto verificado, y cómo se invoca el modo Create por primera vez. Incluye un truco práctico: si la descripción que vas a dictar es larga, usar una herramienta de dictado por voz (Wispr Flow o similar) es más rápido que teclear. Diez minutos y sales con Skill Creator activo en tu Claude Code, probado con una invocación trivial.

M1.2 · Cómo se instala y cómo se invoca

Bloque 0 · El plan de los próximos diez minutos

Si lo hacemos bien, en diez minutos tendrás Skill Creator instalado, una primera invocación de prueba ejecutada, y la confirmación de que tu `CLAUDE.md` ha quedado correctamente actualizado. La idea es que salgas de este submódulo con algo funcionando delante, antes de seguir con teoría.

Cuatro pasos. Cada uno tarda menos de tres minutos. Si en alguno te atascas más, lo más probable es que sea problema de tu Claude Code base, no del plugin. En cada paso te digo qué mirar si algo no encaja.

Empezamos.

Bloque 1 · El comando exacto

Skill Creator se instala con una sola línea en tu sesión de Claude Code:

```
/plugin install skill-creator@anthropic-agent-skills
```

Eso es todo. El plugin viene del marketplace oficial de plugins de Anthropic (`anthropic-agent-skills` es el identificador del marketplace), y `skill-creator` es el nombre del paquete dentro. Claude Code resuelve la descarga, te pide confirmación una vez, y deja la pieza instalada.

Antes de teclear conviene comprobar que tu Claude Code está al día. El sistema de plugins ha ido cambiando entre versiones del cliente, y un Claude Code de hace meses puede no entender el comando `/plugin install`. Si te suelta "comando no reconocido", actualiza el cliente y vuelve a intentarlo. No es un problema del plugin. Y como en este ecosistema los comandos se reescriben cada pocos meses, si el día que veas este vídeo claude.com/plugins/skill-creator enseña otra sintaxis, fíate de la página oficial: ahí siempre vive la versión vigente. El comando que ponemos aquí está verificado contra esa página a fecha de redactar.

Bloque 2 · Comprobar que la instalación funciona

Tras el `install`, Claude Code reporta éxito. No te fíes y comprueba a mano. Son quince segundos de trabajo y te ahorra explicaciones raras dentro de media hora.

Lo más rápido es teclear `/skill-creator` sin más. Si el plugin está bien instalado, te responde con un saludo o una lista de los modos disponibles (Create, Eval, Improve, Benchmark — los cuatro modos que vimos en M1.1). Si no aparece nada o te dice que no conoce el comando, el plugin no se instaló bien. Repite el `install` o revisa el registro del cliente.

Si quieres una comprobación más completa, escribe `/plugin list` y mira que `skill-creator` aparece en la lista de plugins activos. Esto te confirma además de qué marketplace viene y qué versión tienes. Cuando vayas acumulando plugins, este comando se vuelve tu amigo.

Si las dos comprobaciones pasan, has terminado la instalación. Si alguna falla, no avances al siguiente bloque: el problema vive aquí y no en la invocación.

Bloque 3 · La primera invocación, sin prisa

Vamos a hacer una invocación trivial para que veas el flujo entero. No es la práctica del M1.3, que será una skill útil de verdad; es solo el "hola mundo" para confirmar que todo encaja.

Escribe:

```
/skill-creator
Modo: Create
Quiero una skill llamada saludo-cordial que escriba "Hola, [nombre]" en respuesta
cuando alguien me salude por nombre.
```

Lo primero del prompt es que el modo lo explicitas tú. Skill Creator tiene los cuatro modos que ya conoces de M1.1; si no le dices cuál quieres, intenta adivinarlo del contexto y suele acertar, pero la primera vez conviene decirlo de forma explícita para que no haya sorpresas. El segundo es el nombre de la skill, que también lo eliges tú — aquí `saludo-cordial`. La convención es kebab-case sin acentos, y Skill Creator lo respeta tal cual. El tercero es la descripción: cuanto más concreta, mejor sale el resultado. "Que escriba *Hola, [nombre]*" se entiende y produce algo útil. "Que dé la bienvenida de forma profesional y amigable" es vago y te devuelve una skill peor. Lo aprendes rápido en cuanto ves un par de salidas.

Le das al enter y Skill Creator hace su trabajo: monta el `skill.md`, lo coloca en la carpeta de skills, te enseña un resumen de lo que ha hecho. Tarda entre quince y treinta segundos. Si tarda más de un minuto, algo no está bien.

Al terminar te dice dos cosas: dónde quedó el archivo de la skill, y qué entrada ha añadido a tu `CLAUDE.md`. Anota ambas; el siguiente bloque las usa.

Bloque 4 · Comprobar que tu `CLAUDE.md` quedó bien

Este es el paso que muchísima gente se salta — y luego no entiende por qué su nueva skill no se activa sola. No tengo una cifra medida; sí tengo el patrón visto suficientes veces como para que merezca un bloque entero.

Abre tu `CLAUDE.md` del proyecto. Busca la sección que Skill Creator ha añadido — suele ir al final, con una cabecera que menciona la skill que acabas de crear. Comprueba primero que la entrada existe; si no aparece, la skill está creada pero el agente no sabe que existe, y vas a tener que invocarla siempre a mano. Comprueba después que la entrada describe bien cuándo activar la skill. Skill Creator hace un trabajo razonable con la primera versión, pero a veces el trigger queda demasiado abierto o demasiado estrecho. Si dice algo tipo "cuando el usuario salude", lee y decide si lo dejas como está o lo afinas.

Si tienes que afinar, lo haces editando el `CLAUDE.md` a mano. Ese trabajo es parte normal de tener skills personalizadas: Skill Creator te da el primer borrador, tú lo ajustas según tu contexto real. Cuando llegues aquí con instalación verificada, invocación trivial ejecutada y `CLAUDE.md` revisado, Skill Creator está vivo en tu Claude Code, en bastante menos de diez minutos.

Bloque 5 · El truco para descripciones largas

Cuando uses Skill Creator más allá de un "hola mundo" como el del bloque anterior, te vas a encontrar describiendo flujos largos. *"Quiero una skill que tome el correo del cliente, busque sus pedidos anteriores en el CRM, identifique si hay alguno con incidencia abierta, genere un borrador de respuesta empática, y deje un recordatorio para mí en mi agenda si la incidencia lleva más de cinco días sin tocar."* Eso son ochenta palabras y solo era un ejemplo de calentamiento. Si lo tecleas, te va a doler la espalda.

El autor del vídeo origen recomienda **Wispr Flow** para esto, y la recomendación se sostiene. Es una app de dictado por voz que vive en Mac, Windows, iPhone y Android (web oficial: `wisprflow.ai`). Atajo de teclado, hablas, y mientras hablas va transcribiendo a texto en cualquier aplicación, incluida tu sesión de Claude Code. La propia empresa publicita un factor cuatro respecto a teclear; mi experiencia coincide en ese rango para descripciones largas.

Antes de instalarla hay dos cosas que conviene saber, porque no son obvias y se descubren tarde. La primera es que el procesamiento de voz va a la nube. Wispr Flow no transcribe en tu máquina; envía el audio a servidores externos. Sus propias páginas de subprocesadores citan a Baseten y Soniox para la transcripción y a Anthropic, OpenAI y Cerebras para el procesamiento de texto. Si trabajas con clientes que tienen acuerdos de confidencialidad estrictos, o estás en un sector

regulado (sanidad, banca, sector público), dictar prompts que mencionen datos sensibles no es buena idea. Lee la política de privacidad antes de meterla en tu flujo profesional.

La segunda es el consumo de memoria. Hablamos de unos 800 MB reservados cuando no la estás usando activamente, según lo observado en mayo de 2026 — la cifra puede cambiar con versiones. En una máquina con dieciséis gigas o más no se nota; en una con ocho ya empieza a competir con otras cosas. Para máquinas modestas o con muchas pestañas y herramientas abiertas, es algo a sopesar.

Si esas dos consideraciones te encajan, Wispr Flow te va a ahorrar tiempo. Si te bloquea alguna, hay alternativas: el dictado nativo del sistema operativo (Mac y Windows lo traen ya — menos refinado, pero local y gratis), o tecleas y aguantas. Lo que pesa es la calidad de la descripción que escribes; el cómo la metes en el campo es secundario.

Bloque 6 · Cierre y enlace a M1.3

Si has seguido los cuatro pasos, Skill Creator queda instalado, verificado, invocado una vez, y su entrada en tu `CLAUDE.md` revisada. Wispr Flow es opcional — aparece aquí porque las descripciones largas son la fricción real cuando trabajas en serio con Skill Creator, no porque sea parte del producto.

En el siguiente submódulo (M1.3) toca práctica útil: una skill personalizada con valor para tu día a día. Elegirás entre tres candidatas reales — generador de pull request con tu plantilla, análisis post-llamada, o lista de comprobación previa al despliegue — y le aplicaremos el ciclo completo Create → Eval → Improve. Sales del módulo con una skill funcionando en tu proyecto.

Glosario rápido del bloque

- `/plugin install` — comando estándar de Claude Code para instalar plugins desde un marketplace.
- `anthropic-agent-skills` — identificador del marketplace oficial de Anthropic para skills. Es el sufijo `@marketplace` del comando.
- `/plugin list` — comando de verificación que muestra todos los plugins instalados y de qué marketplace vienen.
- **Wispr Flow** — app de dictado por voz para Mac, Windows, iPhone y Android. Atajo de teclado, transcripción en cualquier aplicación. Procesamiento en la nube, ~800 MB de memoria reservada cuando está abierta sin uso activo (mayo de 2026).

Qué viene en M1.3

Tu primera skill personalizada útil — una que vayas a usar de verdad, no un "hola mundo". Eliges entre tres candidatas reales (generador de pull request, análisis post-llamada, o lista de comprobación previa al despliegue) y le aplicas el ciclo Create → Eval → Improve completo. El submódulo cierra con la skill funcionando en tu proyecto, en unos diez minutos.

M1.3 · Tu primera skill personalizada en 5 minutos

Bloque 0 · Por qué este caso y no otro

Llegado a este punto tienes Skill Creator instalado (M1.2), sabes invocarlo y entiendes los cuatro modos (M1.1). Toca construir algo real. La pregunta es qué.

Para una primera vez busco un caso que cumpla cuatro condiciones: que sea genuinamente repetitivo (lo haces varias veces por semana), que el output sea fácil de evaluar a ojo, que el contexto se entienda sin explicaciones largas, y que la mejora frente a hacerlo a mano se note el primer día. Tres candidatos me cumplen las cuatro: generador de pull requests con la plantilla del equipo, análisis post-llamada (si grabas reuniones con Fathom o similar), y checklist pre-deploy.

Vamos con el primero. Razón principal: si trabajas en cualquier equipo de devs serio, el PR semanal con su descripción bien escrita es de las cosas que más das por hechas y que peor sueles hacer cuando vas con prisa. Una skill que te lo deja redactado en treinta segundos es ganancia neta inmediata, y el resultado lo evalúas en un solo vistazo.

Si tu día a día no es código y los PR no encajan, los otros dos casos los tienes en `recursos/skills-de-ejemplo.md` con la misma estructura. Adapta el patrón.

Bloque 1 · El problema concreto

Imagina un PR típico de tu equipo. Probablemente sigue una convención más o menos parecida a esta:

- Título corto en imperativo (tipo "feat: añade endpoint de búsqueda de facturas").
- Descripción con tres secciones: qué cambia, por qué cambia, cómo probarlo.
- Una checklist de cosas a verificar antes de fusionar.
- Mención a reviewers según el área afectada del código.
- Links a tickets o discusiones relacionadas.

Lo haces a mano cada vez. Cuando vas con prisa o estás cansado un viernes a las seis, te saltas la sección "por qué cambia" o pones "ver código" como descripción. Y cuando esa pieza la revisa el del equipo dentro de tres meses, no encuentra contexto.

La skill que vamos a construir toma el diff actual de tu rama, genera el PR siguiendo la convención del equipo, y lo deja listo para que tú revises y publiques. Te ahorra dos cosas: el

tiempo de redactar (cinco a diez minutos típicos por PR), y la inconsistencia entre los buenos y los malos días.

Bloque 2 · El prompt para el modo Create

Invocamos Skill Creator en modo Create y le pasamos la descripción. Si te has instalado Wispr Flow del M1.2, ahora es buen momento de probarlo; tecleado se hace largo. Si vas a teclado, copia esto y adáptalo:

```
/skill-creator
Modo: Create

Quiero una skill llamada generar-pr que tome el diff de la rama actual
contra main y genere la descripción de un Pull Request siguiendo la
convención del equipo.

Convención del equipo:
- Título: imperativo, prefijo según tipo (feat, fix, refactor, docs, chore).
  Máximo 60 caracteres.
- Cuerpo con tres secciones obligatorias:
  - ## Qué cambia (lista de los principales cambios funcionales)
  - ## Por qué (el problema que resuelve, link al ticket si lo hay)
  - ## Cómo probarlo (pasos concretos, no genéricos)
- Checklist al final con cuatro items fijos: tests añadidos/actualizados,
  documentación actualizada, breaking changes documentados, revisado lint.
- Si el diff toca src/Facturas/, sugerir reviewer @maria.
- Si el diff toca src/Pagos/, sugerir reviewer @pedro.
- No inventes información que no esté en el diff. Si falta algo
  (por ejemplo el link al ticket), deja "(pendiente)" en el sitio.
```

Cuatro detalles del prompt que importan:

El nombre es concreto y descriptivo. `generar-pr` está bien; `helper` está mal. El nombre lo verás en `CLAUDE.md` y al invocar la skill, así que un nombre claro te ahorra confusión al cabo de semanas.

Las reglas son explícitas, no aspiracionales. "Título máximo 60 caracteres" es ejecutable; "título profesional" es opinión. Cuando una regla se puede contar (caracteres, secciones, items), cuéntala.

El mapeo área del código → reviewer es del equipo, no genérico. Es exactamente la parte que ninguna skill pública del catálogo te va a traer porque depende de tu organigrama. Por eso esta es una skill personalizada.

La instrucción "no inventes" es importante. Sin ella, el modelo va a rellenar los huecos con texto plausible pero falso. Esa es de las pocas reglas que merece la pena repetir en cada skill

personalizada que escribas.

Le das al enter. Skill Creator tarda entre veinte y cuarenta segundos. Cuando termina, te confirma dónde dejó el `skill.md`, qué entrada añadió a tu `CLAUDE.md`, y un resumen de lo que la skill hace.

Bloque 3 · Lo que sale a la primera versión

Esto es importante para gestionar expectativas: la primera versión casi nunca está lista para producción. Suele estar entre el setenta y el ochenta por ciento bien. Lo que falta no es errores graves; son matices que aparecen al usarla con casos reales.

Cuando pruebes la skill recién creada contra un PR real, vas a notar tres patrones típicos:

Cosas que clavan a la primera. La estructura general (título, tres secciones, checklist). El mapeo a reviewers si el diff toca las carpetas que mencionaste. El uso del prefijo correcto si el cambio es obvio. Esto sale bien casi siempre.

Cosas que necesitan ajuste. La sección "Cómo probarlo" suele salir demasiado genérica (cosas tipo "ejecutar los tests"). El prefijo del título puede equivocarse cuando el cambio es mixto (una refactorización que de paso añade algo nuevo). La detección de "breaking changes" en la checklist a veces da falsos positivos.

Cosas que la skill no puede saber. El link al ticket si tu equipo no lo pone en el nombre de la rama. El contexto histórico ("esto es el segundo intento, el primero estaba aquí"). Decisiones de diseño no obvias del diff.

Los tres patrones son normales. La gracia es no quedarse en la primera versión y aplicar los modos Eval e Improve para mover la skill del setenta por ciento al noventa.

Bloque 4 · Modo Eval — probar con casos reales

Antes de improvisar mejoras, sirve más medir qué falla concretamente. Para eso entra el modo Eval.

Le das tres o cuatro casos de prueba reales. No casos sintéticos: PRs reales que hayas hecho en las últimas semanas. Le dices a Skill Creator:

```
/skill-creator
Modo: Eval
Skill: generar-pr
```

Casos de prueba:

1. PR #341 – refactorización del módulo de facturación. Diff disponible en la rama feature/refactor-billing.
Espero: prefijo "refactor", reviewer @maria, mención a issue #320.
2. PR #344 – fix de bug crítico en el pago con tarjeta. Rama hotfix/payment-error.
Espero: prefijo "fix", reviewer @pedro, mención a breaking changes en la checklist marcada como "no".
3. PR #348 – actualización de documentación de la API pública.
Rama docs/update-api.
Espero: prefijo "docs", sin reviewer específico, tests no aplica.

Skill Creator ejecuta la skill contra cada caso, captura el resultado, y te entrega un informe con qué hace bien, qué hace mal, y qué hace dudoso. No es magia: lo que está haciendo por dentro es el ciclo del M1.1 (Executor + Grader del modo Eval).

Para nuestro caso típico, los hallazgos van a ser parecidos a esto:

- Caso 1: bien al noventa por ciento. Reviewer correcto, prefijo correcto, pero la sección "Cómo probarlo" dice "ejecutar tests del módulo" sin especificar cuáles. Mejorable.
- Caso 2: mal en un detalle. La skill marca "breaking changes: sí" porque el diff toca código público, pero el cambio es interno y no rompe nada. Falso positivo a corregir.
- Caso 3: bien. Detecta que es `docs`, no sugiere reviewer porque no hay regla para `docs/`, marca "tests no aplica" sin que se lo digas.

Dos de tres bien, uno con un fallo concreto y reproducible. Eso es justo lo que el modo Improve necesita.

Bloque 5 · Modo Improve — cerrar el bucle

Con el informe del Eval delante, invocas Improve:

```
/skill-creator
Modo: Improve
Skill: generar-pr
```

Cambios concretos pedidos:

1. Para "Cómo probarlo", instruir a la skill que liste los tests afectados específicos (extraer del diff), no genéricos.
2. La detección de "breaking changes" debe basarse en cambios reales de signature pública, no solo en "toca archivo público". Si no está seguro, dejar la checklist sin marcar y añadir nota "(revisar manualmente)".

Skill Creator analiza la skill existente, aplica los cambios al `skill.md`, y te entrega la versión actualizada. La parte interesante: actualiza también la entrada del `CLAUDE.md` si los triggers han cambiado. No tienes que tocar nada a mano.

Vuelves a correr Eval con los mismos tres casos. Si los cambios funcionaron, ahora los tres salen bien al noventa por ciento. Si alguno empeoró (a veces pasa, el modelo a veces sobre-corrige), vuelves a iterar.

En la práctica, dos ciclos Eval → Improve te suelen llevar de "primera versión a un setenta y cinco por ciento" a "versión madura para usar en producción al noventa por ciento". Más allá del noventa rinde menos: la mejora marginal cuesta más iteraciones de las que vale.

Bloque 6 · El resultado y dónde queda todo

Al cerrar el ciclo tienes tres cosas en tu repo:

Tu archivo `skill.md`, en la carpeta de skills (típicamente `.claude/skills/generar-pr/skill.md`). Si lo abres, ves el procedimiento que Skill Creator generó. Es markdown plano; puedes editarlo a mano si más adelante necesitas un retoque puntual que no merece pasar por Improve.

Una entrada en tu `CLAUDE.md` que le dice al agente cuándo invocar la skill. Suele ser algo así como *"cuando el usuario pida generar la descripción de un PR, abrir un PR nuevo, o redactar un mensaje de commit largo, invocar la skill generar-pr"*. Esto es lo que hace que la próxima vez digas simplemente "voy a abrir un PR" y el agente proponga usar la skill por su cuenta.

El registro de los Evals e Improves, si Skill Creator lo guarda en tu proyecto (por defecto sí). Es histórico de cómo la skill llegó a su versión actual. Útil dentro de seis meses si quieres entender por qué la skill funciona así.

Punto de checkpoint: si has hecho la práctica entera, ahora mismo deberías poder escribir en Claude Code "voy a abrir un PR para esta rama" y ver que el agente invoca `generar-pr` sin que tú

se lo pidas explícitamente. Si no lo hace, lo más probable es que la entrada del `CLAUDE.md` no esté bien afinada. Lee el `CLAUDE.md`, ajusta el trigger a mano, y vuelve a intentar.

Bloque 7 · Cierre y enlace a M1.4

Dos ideas para llevarte de esta práctica.

La primera versión nunca está lista, pero pasa de no estar lista a estar lista en dos ciclos cortos de Eval e Improve. Si te quedas en Create y nunca iteras, las skills personalizadas se quedan a un setenta por ciento y eso es justo lo que termina causando la frustración con las skills personalizadas que la gente cuenta tres meses después.

El valor de la skill no está en lo que escribe por ti, está en lo que estandariza. La descripción del PR la podías escribir tú; la diferencia es que ahora la escribes igual de bien el viernes a las seis que el lunes a las nueve. Esa consistencia es lo que tu equipo va a notar antes que el ahorro de tiempo.

En el siguiente submódulo (M1.4) cerramos el módulo con una vuelta al porqué Skill Creator entra primero al curso, ahora que ya tienes una skill funcionando construida. Dos minutos para fijar la idea y arrancar M2 con Superpowers.

Glosario rápido del bloque

- **Skill personalizada** — la que construyes con Skill Creator para tu trabajo concreto. No vive en el catálogo público.
- **Modo Create** — genera la primera versión de la skill desde una descripción en lenguaje natural.
- **Modo Eval** — ejecuta la skill contra casos de prueba reales y reporta qué hace bien y qué hace mal.
- **Modo Improve** — aplica cambios al `skill.md` basándose en los hallazgos del Eval. Mantiene `CLAUDE.md` sincronizado.
- **Ciclo Eval → Improve** — la combinación que mueve una skill del setenta por ciento al noventa. Dos vueltas suelen bastar.

Qué viene en M1.4

Cierre del módulo en dos minutos: por qué Skill Creator entra primero al curso, ahora que ya has visto el ciclo completo en acción. Después, arrancamos M2 con Superpowers, que es la pieza

grande del catálogo curado.

M1.4 · Por qué Skill Creator entra primero al curso

Bloque 0 · Lo que vamos a hacer en dos minutos

Ya tienes Skill Creator instalado (M1.2), has visto su anatomía (M1.1), y has construido una skill personalizada útil pasando por los cuatro modos (M1.3). Antes de cambiar de pieza y meternos con Superpowers en el M2, dos minutos para fijar la pregunta que igual te ha quedado en la cabeza: **¿por qué entra al curso antes que las skills más vistosas?**

Bloque 1 · La razón principal

Si miras el catálogo público de skills, Skill Creator no es la que más estrellas tiene, no es la que más vídeos genera en YouTube, no es la que más promete. Otras cinco del curso son más llamativas en demo. Y aun así abre el módulo. La razón cabe en una idea: **es la única del catálogo que multiplica todo lo demás.**

Las otras cinco son piezas terminadas. Las instalas, te dan lo que dan, y el techo es ese. Skill Creator no: instalarla no añade una capacidad concreta, añade la posibilidad de fabricar todas las capacidades que tu trabajo necesite. Esa diferencia es la que justifica la posición.

Vuelve al Mo.2 un segundo y recupera la distinción entre skills generales y personalizadas. El cincuenta por ciento de tu valor diario con Claude Code va a venir de las personalizadas — las que cubren tu trabajo concreto, tu plantilla, tu flujo. Esas no las trae nadie hecho. Las tienes que poder construir tú. Skill Creator es la pieza que te lo hace barato.

Sin esta primera skill, las otras cinco del curso son buenas piezas pero incompletas. Cubren la mitad general bien y dejan la mitad personalizada en el aire. Con esta primera, cubres ambas mitades — y eso explica el orden del temario.

Bloque 2 · Lo que cambia ahora que ya la tienes

Acabas el M1 con la skill `generar-pr` funcionando en tu repo, con el `CLAUDE.md` correctamente actualizado, y con un patrón en la cabeza para construir las próximas. La consecuencia operativa es que las próximas skills personalizadas te van a salir más rápido que la primera.

La primera te costó diez minutos porque ibas aprendiendo el ciclo Eval → Improve a la vez que construías. La segunda te va a costar tres o cuatro porque ya tienes el patrón. La tercera ya

entrará en cinco. Es como cualquier herramienta nueva: la curva al inicio es la que paga la velocidad de después.

En el M7 vamos a ver una caja de cinco skills personalizadas más como inspiración para que vayas montando la tuya propia. No es trabajo de una tarde; es trabajo de unas semanas durante las cuales identificas qué tareas repites y las vas convirtiendo. Skill Creator es la pieza que hace que ese proceso sea liviano en lugar de aburrido.

Bloque 3 · Cierre y enlace a M2

Para llevarte: las cinco skills generales que vienen a partir de aquí son buenas y útiles cada una por separado, pero solo cubren su mitad del problema. Con Skill Creator detrás, el catálogo entero de tu Claude Code se completa con todas las piezas personalizadas que tu trabajo necesita.

En el siguiente módulo entramos en **Superpowers**. Es el más extenso del curso, porque no es una skill suelta sino un plugin con más de veinte sub-skills activables que cambian la forma en que el agente trabaja. El cambio se nota en cinco minutos de uso. Pasamos directos a M2.1.

PARTE 2

Superpowers · pensar como senior

M2.1 · El problema número uno de Claude Code — la prisa

Bloque 0 · Una escena que te suena

Le pides a Claude Code que añada un endpoint nuevo a tu API. Algo aparentemente sencillo: validar el input, hacer una consulta a base de datos, devolver el resultado. El agente arranca, escribe el código en treinta segundos, ejecuta los tests que él mismo se ha hecho, sale verde, y te dice "listo".

Lo miras. Está bien. Compila. Los tests pasan. Lo merge.

Tres días después aparece el problema en producción: el endpoint funciona con la mayoría de inputs pero revienta cuando el cliente le pasa un campo opcional vacío en lugar de no enviarlo. El test que el agente escribió no contemplaba ese caso. Tú no lo viste porque confiaste en que los tests pasaban. El caso límite estaba ahí desde el primer día.

Si esta escena te ha pasado más de tres veces, este módulo va dirigido a ti. Y si te ha pasado más de diez, probablemente te culpaste a ti mismo cuando en realidad estabas peleando contra el comportamiento por defecto del agente.

Bloque 1 · El diagnóstico, sin metáforas

El agente por defecto **va con prisa**. Esa es la descripción más fiel del problema. No es que no sepa programar bien; sabe. No es que no entienda los casos límite; los entiende si los miras a la cara. El problema es que cuando le pides algo, el comportamiento por defecto del agente es entregar rápido, no entregar bien.

Tres síntomas concretos que verás en cuanto los busques:

Tests escritos solo para la salida feliz. El agente escribe el test que demuestra que el código funciona en el caso normal. Si tú no le pides explícitamente que cubra casos límite, no los cubre. Y como los tests pasan, parece que está todo bien.

Decisiones de diseño tomadas sin avisar. El agente elige entre dos formas de resolver el problema y se queda con una. La elección puede ser razonable, pero no la has visto, no la has discutido, y dentro de dos meses cuando alguien lea el código no sabrá por qué se hizo así.

Saltos sin red de seguridad. El agente refactoriza una función auxiliar mientras trabaja en otra cosa "porque se le ve la limitación". La refactorización parece inocente, no tiene tests

específicos, y la próxima ejecución revienta porque hay otra parte del código que dependía del comportamiento viejo.

Los tres síntomas comparten una raíz: el agente toma atajos para entregar antes. No es maldad y no es incompetencia; es comportamiento por defecto. Y como cualquier comportamiento por defecto, se cambia cambiando el default, no recordándoselo en cada prompt.

Bloque 2 · Por qué "piensa antes" no es la solución

La primera reacción de mucha gente cuando ve esto es añadir instrucciones del tipo *"piensa bien antes de empezar"*, *"considera todos los casos límite"*, *"escribe tests para los bordes también"*. Lo prueban, lo dejan, lo abandonan. La razón es simple: **funciona una vez de cada cinco**.

El recordatorio textual depende de que el agente lo procese, lo retenga durante toda la sesión, y lo aplique en el momento correcto. Lo procesa, sí. Lo retiene los primeros dos turnos, normalmente. Lo aplica cuando se acuerda, que con sesiones largas es menos de lo que parece. A las cinco respuestas, ya estás de vuelta en el comportamiento por defecto sin haberte dado cuenta.

Lo que de verdad funciona es cambiar el procedimiento, no el recordatorio. Si el agente, por procedimiento fijo, planifica antes de tocar código, abre un entorno aislado, escribe el test antes que la implementación, y revisa su trabajo en dos pases distintos antes de declarar la tarea terminada, los tres síntomas del bloque anterior se desactivan solos. Ya no dependen de que el agente "se acuerde": dependen del procedimiento que el plugin instala.

Esa es la idea de Superpowers en una frase. No te da una skill mejor: te instala un procedimiento entero que hace que el agente trabaje como un dev senior por procedimiento, no por buena voluntad.

Bloque 3 · Lo que viene en el módulo

Superpowers es plugin, no skill suelta. Por dentro lleva más de veinte sub-skills coordinadas que cubren el ciclo entero de desarrollo: brainstorming inicial, trabajo en rama aislada con worktree, planificación en micro-tareas, TDD red-verde de verdad, ejecución por subagentes con revisión en dos fases, y manejo de code review. Cada una hace una cosa concreta; juntas, cambian cómo se siente trabajar con el agente.

En los próximos cuatro submódulos:

- M2.2 entra en qué es Superpowers exactamente, quién lo mantiene, y por qué la versión oficial del marketplace de Anthropic es la que recomendamos instalar.

- M2.3 desgrana las sub-skills clave una a una. No las veinte; las cinco que de verdad cambian el comportamiento. El resto se cargan automáticamente cuando hace falta.
- M2.4 es la instalación con el comando exacto verificado y la comprobación de que el procedimiento está activo.
- M2.5 cierra el módulo con la misma comparativa que hicimos en M0.1 con el caso de la factura, ahora aplicada a un proyecto pequeño: el mismo problema, con y sin Superpowers, para que veas el cambio de procedimiento en vivo.

Cuarenta y cinco minutos en total. Es el módulo más extenso del curso por una razón: si Superpowers solo cubriera la mitad de lo que cubre, ya merecería un módulo propio. Lo que hace es lo que separa "Claude Code en pruebas" de "Claude Code en producción".

Bloque 4 · Cierre y enlace a M2.2

El problema con el comportamiento por defecto del agente no es que falle en lo difícil. Es que parece terminar bien en lo fácil — y eso es justo lo que esconde los casos límite que aparecen en producción.

Recordarle al agente que piense antes funciona mal. Lo que funciona es cambiar el procedimiento por defecto. Esa es la diferencia entre meter una instrucción en el prompt y meter un plugin que instala un procedimiento.

Superpowers es plugin de procedimiento, no skill puntual. Lo que cambia en tu Claude Code después de instalarlo no es lo que sabe el agente; es cómo lo aborda. Y eso se nota en cinco minutos de uso real.

En el siguiente submódulo (M2.2) entramos en qué es Superpowers operativamente: quién lo creó, qué hace por dentro, y por qué la versión oficial del marketplace de Anthropic es la que tiene sentido instalar en lugar de la comunitaria.

Glosario rápido del bloque

- **Comportamiento por defecto** — la forma en que el agente aborda una tarea cuando no le has dado un procedimiento específico. En Claude Code, el default tiende a la entrega rápida.
- **Plugin de procedimiento** — pieza que no añade una capacidad concreta sino que cambia el cómo el agente aborda las tareas. Superpowers es el ejemplo de referencia.
- **Saltos sin red de seguridad** — refactorizaciones o cambios laterales que el agente hace por iniciativa propia sin estar previstos. La causa principal de regresiones en producción atribuidas al agente.

- **Procedimiento vs recordatorio** — la diferencia entre instalar un comportamiento por defecto nuevo y recordar a cada prompt que aplique buenas prácticas. El primero funciona; el segundo funciona una vez de cada cinco.

Qué viene en M2.2

Qué es Superpowers operativamente. Quién lo creó (Jesse Vincent, Prime Radiant). Por qué hay dos formas de instalarlo (marketplace oficial de Anthropic y marketplace comunitario propio) y cuál tiene más sentido para alguien que está empezando. Y la cifra de adopción que importa, con la salvedad honesta de cómo leer las cifras de estrellas sin caer en la trampa del titular.

M2.2 · Qué es Superpowers

Bloque 0 · La promesa del submódulo

Si el módulo anterior te convenció de que el problema con Claude Code no es lo que sabe sino cómo lo aborda por defecto, la pregunta siguiente es qué pieza concreta instala un comportamiento por defecto distinto. La respuesta en este curso es Superpowers. En este submódulo vamos a ver qué es exactamente, quién está detrás, y cómo se instala con las dos opciones que hay sobre la mesa.

Lo que no vamos a tocar todavía es el catálogo de sub-skills una a una; ese desglose entra en M2.3. Aquí dejamos fijado el qué y el quién para que cuando veamos las piezas concretas en el siguiente submódulo, sepas en qué marco encajan.

Bloque 1 · Qué es operativamente

Superpowers es un **plugin** para Claude Code que se instala desde el sistema de plugins estándar (`/plugin install`) y trae más de veinte **sub-skills** coordinadas. Tres detalles del párrafo anterior importan más de lo que parecen:

Plugin, no skill suelta. Esto cambia cómo lo evalúas. Una skill suelta hace una cosa concreta y la haces invocando manualmente. Un plugin trae un conjunto de skills que se cargan automáticamente según el contexto. Cuando tu Claude Code detecta que vas a empezar una pieza nueva, el plugin activa el skill de brainstorming. Cuando aprueba el diseño, activa el skill de planificación. Cuando el plan está listo, activa el skill de ejecución por subagentes. Tú no tienes que orquestar el flujo — el plugin lo hace por ti.

Más de veinte sub-skills. El número exacto va creciendo porque el repositorio recibe contribuciones, pero la cifra cualitativa importa más que la exacta: no son tres o cuatro, son veinte y pico. Eso explica por qué se llama plugin y no skill: hay piezas pequeñas y especializadas que juntas componen una metodología completa.

Coordinadas, no sueltas. Las sub-skills no son herramientas separadas que tú activas cuando te acuerdas. Forman un flujo: el output de una alimenta la entrada de la siguiente. Brainstorming produce un documento de diseño, el documento alimenta a Writing-Plans, el plan alimenta a Subagent-Driven-Development, el resultado de los subagentes pasa por revisión en dos fases. Si te pidieran resumir el plugin en una palabra sería **metodología**, no herramienta.

Si has hecho SDDCC en este vault, o si has trabajado con Spec-Driven Development en cualquier forma, vas a reconocer la columna vertebral inmediatamente. Brainstorm → spec → plan → ejecución con verificación es el mismo patrón. Superpowers es una de las implementaciones más maduras de esa columna vertebral aplicada a Claude Code.

Bloque 2 · Quién está detrás

El plugin lo mantiene **Jesse Vincent**, que opera en GitHub con el handle `obra` (por eso el repositorio se llama `obra/superpowers`). Es fundador y CEO de **Prime Radiant**, una empresa que se dedica a metodologías y herramientas para agentes de IA aplicados a desarrollo.

¿Por qué importa saber esto?

Importa porque uno de los cuatro filtros del MO.3 era *viva*, y "viva" responde mejor a una pregunta concreta: ¿quién mantendrá esto dentro de seis meses? En Superpowers la respuesta es estable: hay una empresa pequeña pero con foco específico (Prime Radiant) cuyo producto principal gira alrededor del plugin. No es un proyecto de fin de semana de alguien que después se aburrirá. Hay incentivo económico y profesional para que siga vivo.

Importa también porque Jesse escribe activamente sobre el proyecto en su blog (`blog.fsck.com`) y en podcasts del nicho. Si quieres entender por qué algunas decisiones de diseño son las que son, hay material en abundancia para leer y escuchar. No tienes que adivinar la filosofía del proyecto.

Y un detalle pequeño pero relevante para tu evaluación: aunque Superpowers nació pensado para Claude Code, hoy soporta también Codex CLI, Codex App, Factory Droid, Gemini CLI, OpenCode, Cursor y GitHub Copilot CLI. Si dentro de un año cambias de cliente agente, el procedimiento que has interiorizado con Superpowers no se queda inservible. Es bonus del filtro *compone*.

Bloque 3 · Las dos formas de instalarlo

Aquí hay un detalle operativo que merece atención. Superpowers se puede instalar por dos caminos distintos. Las dos llevan al mismo plugin, pero las consecuencias para ti son ligeramente diferentes.

Camino oficial — desde el marketplace de Anthropic. Comando:

```
/plugin install superpowers@claude-plugins-official
```

El sufijo `@claude-plugins-official` apunta al marketplace que mantiene directamente Anthropic. Es donde viven los plugins curados que pasan revisión interna de Anthropic antes de

aparecer. Para ti significa una capa extra de garantía: si está ahí, está suficientemente probado para que Anthropic no quiera líos.

Camino comunitario — desde el marketplace propio del proyecto. Dos pasos:

```
/plugin marketplace add obra/superpowers-marketplace
/plugin install superpowers@superpowers-marketplace
```

El primero registra el marketplace que mantiene Jesse Vincent directamente. El segundo instala el plugin desde ese marketplace.

Las dos versiones son **el mismo plugin**. El código viene del mismo repositorio (`obra/superpowers`) y el comportamiento es idéntico una vez instalado. La diferencia está en la cadena de distribución.

¿Cuál elegir? Para esta primera vez, **la oficial**. Tres razones:

Es un comando, no dos. Menos ceremonia, menos cosas que pueden salir mal cuando llevas media hora intentando configurar tu Claude Code.

Si algo va mal con la versión oficial, el debugging es más estándar. La versión comunitaria depende también de la configuración del marketplace registrado a mano.

Para usuarios avanzados que quieren versiones pre-release o sub-skills experimentales antes de que pasen por el filtro oficial, el camino comunitario tiene sentido. Para empezar, no.

La excepción razonable: si en algún momento ves que la versión oficial está varias semanas por detrás de la del marketplace comunitario y necesitas algo específico que solo está en la versión nueva, cambia. Mientras tanto, oficial.

Bloque 4 · La cifra de adopción y cómo leerla

Vamos con un tema que el vídeo origen tocaba a brochazos. Superpowers tiene una cifra de adopción muy alta — Jesse Vincent y Prime Radiant lo llaman, con autoridad de quien lo mantiene, "el plugin más usado de Claude Code en el mundo". A fecha de redactar este curso, el repositorio cuenta con más de ciento cincuenta mil estrellas en GitHub, con crecimiento sostenido en los últimos meses.

Tres avisos honestos antes de quedarte solo con el titular.

Las estrellas son señal, no garantía. Cumple el filtro *adoptada* del Mo.3, pero como vimos en ese módulo, una skill con quinientas estrellas y conversación viva puede valer más que una con

cinco mil en silencio. Superpowers cumple las dos: estrellas altas y conversación viva (issues respondidos, releases frecuentes, blogs del creador). La combinación es la que cuenta.

Las cifras concretas mueven ficha rápido. El vídeo origen de este curso, grabado unas semanas antes, citaba ~190 mil estrellas. Otra fuente de la misma época citaba 208 mil. La realidad oscila según fuente y fecha. Lo importante no es el número exacto sino el orden de magnitud, que es "decenas a centenas de miles" — claramente dentro del rango del catálogo serio.

Cuidado con la lectura "si tantos lo usan, será bueno". Cumple para descartar lo abandonado, no para validar lo bueno. Hay plugins muy estrellados que están bien para una cosa y mal para otra. Las estrellas dicen que mucha gente al menos lo probó; no dicen que sea lo que tu setup necesita. Para eso necesitas los otros tres filtros (viva, reversible, compone). Superpowers los cumple los cuatro, pero la cifra alta sola no habría bastado para que entrara al curso.

Bloque 5 · Por qué pasa los cuatro filtros del M0.3

Cerramos el submódulo aplicando el criterio del curso para que veas explícitamente por qué Superpowers entra a la selección.

Viva: sí. Repositorio activo con commits semanales, issues respondidos, releases frecuentes. Lo mantiene una empresa con foco específico en el producto (Prime Radiant), no un voluntario que puede aburrirse.

Adoptada: sí, sobradamente. Más de ciento cincuenta mil estrellas con crecimiento sostenido, blog del creador con tráfico significativo, conversación viva en el nicho.

Reversible: sí. Es plugin estándar. Se instala con `/plugin install`, se desinstala con `/plugin uninstall` o equivalente, deja trazas mínimas y predecibles. Si lo pruebas una semana y no te convence, te lo quitas sin pelearte con la configuración.

Compone: sí, y aquí destaca. No solo trabaja bien con otras skills de Claude Code; soporta media docena de plataformas agente distintas. El procedimiento que aprendes con Superpowers se traslada si en algún momento cambias de cliente. Pocas piezas del catálogo cumplen esto.

Cuatro de cuatro. Por eso es la primera de las cinco skills generales del curso después de Skill Creator, y por eso ocupa el módulo más largo.

Bloque 6 · Cierre y enlace a M2.3

Superpowers es plugin de procedimiento con más de veinte sub-skills coordinadas, que mantiene Jesse Vincent desde Prime Radiant. Lo que cambia al instalarlo no es lo que el agente sabe; es

cómo aborda las tareas. La distinción de M2.1 entre recordatorio y procedimiento sigue valiendo aquí.

Hay dos formas de instalarlo y para empezar la oficial gana. La comunitaria tiene sentido para usuarios avanzados o para necesidades específicas; mientras tanto, un comando y a otra cosa.

Las cifras de adopción son altas pero hay que leerlas con criterio. Los cuatro filtros del Mo.3 se cumplen sin discusión, y por eso entra al curso — no por el titular de las estrellas.

En el siguiente submódulo (M2.3) entramos en las sub-skills concretas. No las veinte; las cinco que de verdad mueven la aguja. Brainstorming, Using-Git-Worktrees, Writing-Plans, Test-Driven-Development y Subagent-Driven-Development. El resto se cargan automáticamente cuando el contexto lo pide; estas son las que merece la pena entender una a una.

Glosario rápido del bloque

- **Plugin (en Claude Code)** — paquete que agrupa varias skills coordinadas y se instala desde el sistema `/plugin install`. Diferente de una skill suelta.
- **Marketplace** — fuente desde la que se descargan plugins. El oficial lo mantiene Anthropic; los proyectos grandes pueden tener marketplaces propios.
- **Sub-skill** — pieza dentro de un plugin que hace una cosa concreta. Superpowers tiene más de veinte; se activan automáticamente según el contexto.
- **Cadena de distribución** — el camino que sigue un plugin desde el código fuente hasta tu Claude Code. El curador (Anthropic o mantenedor) añade una capa de revisión.
- **Cifra cualitativa de adopción** — lo que importa cuando hablamos de estrellas. No el número exacto sino el orden de magnitud y la calidad de la conversación alrededor.

Qué viene en M2.3

Las cinco sub-skills que mueven la aguja: Brainstorming (qué activa antes de tocar código), Using-Git-Worktrees (el aislamiento que evita romper main), Writing-Plans (la planificación en micro-tareas de 2-5 minutos), Test-Driven-Development (TDD red-verde de verdad), y Subagent-Driven-Development (ejecución con revisión en dos fases). Quince minutos para entender cómo se encadenan y por qué la cadena entera funciona como funciona.

M2.3 · Las sub-skills clave de Superpowers

Bloque 0 · Por qué cinco y no las veinte

Superpowers trae más de veinte sub-skills. Si vamos pieza por pieza, este submódulo dura cuarenta minutos y al alumno se le olvidan diecisiete antes de llegar a la primera práctica. La aproximación que funciona mejor: **cinco que entiendes a fondo, el resto que cargas y dejas trabajar en segundo plano.**

Esas cinco son las que de verdad cambian el comportamiento del agente cuando empieza una pieza nueva. Las demás (revisar PRs, manejar el final de una rama, debugging sistemático, dispatch paralelo y compañía) son herramientas auxiliares — buenas, pero no donde se gana o pierde la batalla del módulo anterior contra la prisa.

Las cinco que vamos a ver, en el orden en que se activan dentro de una pieza típica:

1. **Brainstorming:** antes de tocar código.
2. **Using-Git-Worktrees:** antes de tocar nada en `main`.
3. **Writing-Plans:** antes de escribir tests o lógica.
4. **Test-Driven-Development:** durante la ejecución de cada micro-tarea.
5. **Subagent-Driven-Development:** orquestando el ciclo y cerrando con revisión.

Las cinco se encadenan: el output de una alimenta a la siguiente. Si entiendes la cadena, entiendes la metodología completa. Vamos por orden.

Bloque 1 · Brainstorming

Cuando le pides al agente algo nuevo y este plugin está activo, **Brainstorming entra antes de que el agente toque una línea de código.** Su trabajo: refinar la idea que has tirado, hacer las preguntas que faltan, explorar alternativas, y producir un documento de diseño que el resto de sub-skills van a usar como entrada.

¿En qué se nota?

Imagina que sueltas: *"añade un endpoint para que el cliente pueda anular su suscripción"*. Sin Brainstorming, el agente arranca y escribe el endpoint con las asunciones por defecto. Con Brainstorming, te devuelve algo como: *"Antes de implementarlo, tres preguntas: ¿la anulación es inmediata o al final del período facturado? ¿Hay reembolso prorrateado? ¿Quién recibe*

notificación de la cancelación, solo el cliente o también facturación?". Y después: "Aquí están dos diseños posibles; el A es más simple, el B respeta el modelo de eventos que ya tienes. ¿Cuál prefieres?".

Tres detalles del comportamiento:

Pregunta hasta que las decisiones implícitas se vuelven explícitas. Eso es lo que evita el síntoma del módulo anterior de "decisiones de diseño tomadas sin avisar". Lo que el agente decidiría solo, aquí lo decides tú con la información delante.

Explora alternativas, no se queda con la primera idea. Mira tu código existente para entender qué patrones ya usas y propone el diseño que respeta esos patrones. Útil sobre todo en proyectos con cierta historia: el agente nuevo tiende a reinventar lo que ya está hecho de otra forma.

Guarda el documento de diseño como artefacto. El output no se queda en la conversación; queda como archivo en tu proyecto que las siguientes sub-skills van a leer. Eso permite que Writing-Plans tenga input estructurado, no que dependa de que el agente "se acuerde" de lo que se discutió.

Cuándo se activa: al inicio de cualquier pieza no trivial. Para tareas pequeñas (renombrar una variable, ajustar un mensaje) el plugin no la dispara, no merece la ceremonia. Para piezas que tocan diseño, activación automática.

Bloque 2 · Using-Git-Worktrees

Cuando el diseño está aprobado y vais a empezar a tocar código, **Using-Git-Worktrees crea un entorno aislado en una rama nueva antes de mover un solo carácter.**

Si no conoces el comando `git worktree`, va el contexto rápido: te permite tener varias copias de tu repositorio en distintos directorios, cada una en una rama distinta, todas trabajando sobre el mismo histórico de git. La sub-skill lo usa para que el trabajo del agente nunca toque tu `main` ni tu rama activa actual. Si algo sale mal, el desastre vive en su propio directorio y lo borras sin secuelas.

Tres cosas que hace por procedimiento:

Crea la rama nueva con un nombre derivado del diseño que Brainstorming guardó. Algo del estilo `feature/cancel-subscription`.

Ejecuta el setup del proyecto en el worktree nuevo. Instala dependencias si hace falta, compila, prepara el entorno. Cuando termina, el entorno aislado está listo para trabajar.

Verifica que el baseline de tests está en verde antes de empezar. Esto es importante: si tus tests están rotos antes de que el agente toque nada, el agente debe saberlo antes de implementar. Si no lo hace, podría arreglar un test que ya estaba roto, atribuirse el mérito, y dejarte con la duda de qué arregló y qué seguía roto.

Cuándo se activa: tras aprobar el diseño en Brainstorming, antes de Writing-Plans. Si tu proyecto no soporta worktrees por alguna razón (configuración rara, monorepo con tooling que no le sienta bien), el plugin tiene un modo alternativo que trabaja en branch normal con más precauciones. No es ideal pero funciona.

Bloque 3 · Writing-Plans

Esta es la sub-skill que probablemente más cambia cómo se siente trabajar con el agente. **Writing-Plans toma el diseño aprobado y lo descompone en micro-tareas de dos a cinco minutos cada una, con rutas de archivo exactas, código completo a escribir, y pasos de verificación.**

La diferencia con la planificación que el agente hace por defecto sin esta sub-skill es enorme. Un plan típico del agente por defecto: *"Voy a crear el endpoint, añadir validación, conectar con base de datos, y escribir tests"*. Cuatro pasos vagos de quince minutos cada uno.

Un plan típico con Writing-Plans:

1. Crear archivo `src/endpoints/subscription/cancel.ts` con la firma de la función `cancelSubscription(userId, reason?)` y throw de `NotImplementedError`. (2 min)
2. Añadir validación de `userId` en `src/endpoints/subscription/cancel.ts` líneas 12-18. Test esperado: `cancel-validation.test.ts`. (3 min)
3. Añadir test `cancel-validation.test.ts` con tres casos: `userId` válido, `userId` inválido, `userId` inexistente. (4 min)
4. ...

Veintitrés pasos así, granulares, verificables uno a uno.

Tres efectos directos de esta granularidad:

Cada paso es testeable. Si el paso siete falla, sabes exactamente qué falló y por qué. No tienes que reconstruir media implementación para entender dónde se torció algo.

El agente no improvisa entre pasos. Como cada paso tiene su instrucción exacta, no hay margen para que el agente decida hacer algo "extra que se le ocurrió". Lo que el plan dice se hace; lo que no, no.

Tú puedes intervenir entre pasos. Si en el paso doce el agente entrega algo que no te convence, paras el ciclo, ajustas, y continúas. No hay rollback masivo; hay control fino.

Cuándo se activa: con el diseño aprobado y el worktree listo. Genera el plan, te lo enseña, esperas tu aprobación, y solo cuando dices "adelante" pasa a la siguiente sub-skill.

Bloque 4 · Test-Driven-Development

Aquí es donde Superpowers va a contracorriente de cómo trabajan muchos devs con Claude Code en su día a día. **Test-Driven-Development hace TDD red-verde de verdad, no la versión light que el agente suele hacer por defecto.**

El procedimiento, para cada micro-tarea del plan:

1. Escribir el test primero. Antes de la implementación.
2. Ejecutar el test y confirmar que falla (rojo). Esto verifica que el test está midiendo lo correcto.
3. Implementar lo mínimo para que el test pase (verde).
4. Refactorizar si hace falta, manteniendo verde.

Si has hecho TDD en serio alguna vez, esto te resulta familiar. Si no, lo más notable: el agente nunca escribe código que no esté justificado por un test. La tendencia del agente a añadir de más — código extra que nadie pidió, "por si acaso" — se elimina por procedimiento.

Y aquí viene la parte que cuesta interiorizar: junto al TDD aplica **YAGNI** (*You Aren't Gonna Need It*, no lo vas a necesitar) y **DRY** (*Don't Repeat Yourself*, no te repitas) como principios duros. YAGNI bloquea al agente cuando intenta añadir abstracciones genéricas "para el futuro". DRY le obliga a refactorizar cuando ve duplicación. Las dos son fricciones positivas: te quitan código que no merece estar ahí.

Honestidad sobre la fricción: el primer día con este procedimiento activado, el agente se va a sentir más lento. Va a tardar más en producir el mismo número de líneas. Es la sensación esperada. Lo que vas a notar después es que tres días después tienes menos código y menos bugs. La velocidad sin esta sub-skill es ilusión; la velocidad con ella es la real.

Bloque 5 · Subagent-Driven-Development

La quinta es la sub-skill que orquesta todo lo anterior. **Subagent-Driven-Development toma el plan generado, dispatcha un subagente fresco para cada micro-tarea, y aplica revisión en dos fases tras cada una.**

Tres detalles del comportamiento:

Un subagente fresco por tarea. No es la misma sesión arrastrando contexto de quince pasos atrás. Cada micro-tarea recibe un subagente nuevo con el contexto justo: el plan, el código actual de la rama, y la tarea concreta. Esto evita lo que en el M3 vamos a llamar *context rot* — la degradación de las sesiones largas. Si Superpowers no usara subagentes frescos, las últimas micro-tareas de un plan grande saldrían peor que las primeras. Con subagentes frescos, salen igual.

Revisión en dos fases tras cada subagente. La primera fase comprueba **spec compliance**: ¿lo entregado cumple lo que el plan decía? La segunda fase comprueba **code quality**: ¿el código está bien escrito, sigue las convenciones, no introduce smell? Solo cuando las dos pasan, la micro-tarea se considera terminada. Si alguna falla, vuelve al subagente con feedback concreto.

Checkpoints humanos opcionales. Por defecto el procedimiento ejecuta tarea tras tarea sin pararte. Para piezas grandes o sensibles, puedes configurar que pare cada cierto número de tareas para que tú revises antes de seguir. Es la forma de tener control fino sin convertir el flujo en un infierno de interrupciones.

La consecuencia operativa: cuando Subagent-Driven-Development termina, te entrega una pieza completa, testada, con el código habiendo pasado revisión en dos fases tarea por tarea. La probabilidad de bug latente en producción cae bastante respecto al comportamiento por defecto.

Bloque 6 · La cadena entera vista junta

Si has llegado hasta aquí con las cinco en la cabeza, toca verlas como cadena en lugar de como piezas sueltas:

Tú dices qué quieres → **Brainstorming** te hace preguntas hasta que el diseño está cerrado → guarda el documento → **Using-Git-Worktrees** crea entorno aislado con baseline limpio → **Writing-Plans** toma el diseño y lo descompone en micro-tareas verificables → tú apruebas el plan → **Subagent-Driven-Development** dispatcha subagentes frescos por tarea → cada subagente aplica **Test-Driven-Development** → cada tarea cierra con revisión en dos fases → al final, pieza completa y testada.

El output de cada eslabón alimenta al siguiente. Tú entras al inicio para describir lo que quieres, al final para validar el resultado, y opcionalmente en los checkpoints intermedios para mantener el control. Lo demás corre solo, pero corre con procedimiento, no con improvisación.

¿Lo notas en cinco minutos de uso? Probablemente no en cinco; sí en treinta. El tiempo de la primera pieza con Superpowers activo es más largo que sin él, porque hay ceremonia de

Brainstorming y Writing-Plans que antes no había. La segunda pieza es igual de larga que sin Superpowers. La tercera es más corta. Y a partir de la cuarta entras en velocidad de crucero con menos bugs y más confianza.

Bloque 7 · Las extras que no entran al detalle

Para completitud, las sub-skills extra que el plugin trae y se activan automáticamente sin que tengas que entenderlas a fondo:

- **dispatching-parallel-agents**: cuando una pieza permite paralelismo (tareas independientes entre sí), reparte a varios subagentes simultáneos en lugar de secuenciales.
- **executing-plans**: el motor que ejecuta un plan generado por Writing-Plans. Lo orquesta Subagent-Driven-Development pero la pieza vive aparte.
- **finishing-a-development-branch**: cierra la rama cuando todo está listo. Prepara el commit final, sugiere el mensaje, deja el branch listo para PR.
- **requesting-code-review / receiving-code-review** — manejo del ciclo de revisión externa, cuando otro humano va a revisar el código.
- **systematic-debugging** — cuando algo falla en producción, procedimiento sistemático para encontrar la causa raíz sin caer en el "prueba y a ver".

No hace falta entenderlas todas para usar Superpowers. Se activan cuando el contexto las pide y hacen su trabajo. Si en algún momento ves que el plugin hace algo que no esperas, mira en el repositorio cuál se activó y por qué — son markdown legibles, no código opaco.

Bloque 8 · Cierre y enlace a M2.4

Las cinco sub-skills clave forman una cadena, no un menú. Brainstorming → Using-Git-Worktrees → Writing-Plans → TDD → Subagent-Driven-Development. Entender la cadena entera importa más que recordar las cinco una a una.

La fricción que sientes el primer día es síntoma de que el procedimiento está activo. Si no la sientes, el plugin no está cambiando nada. Aceptar la fricción del primer día es lo que da las velocidades de crucero del tercer día en adelante.

Las quince sub-skills restantes están ahí para cuando hagan falta, no para que las aprendas en frío. Se activan automáticamente, hacen su trabajo, y siguen. Si en algún momento te entra curiosidad de cómo funciona alguna, el repositorio es legible y está bien documentado.

En el siguiente submódulo (M2.4) entramos en lo operativo: comandos exactos de instalación con la opción que recomendamos (la oficial del M2.2), cómo verificar que el procedimiento está activo, y un par de gotchas que pueden aparecer la primera vez y que el README no menciona explícitamente.

Glosario rápido del bloque

- **Brainstorming** — sub-skill que activa antes de tocar código; refina la idea con preguntas y guarda documento de diseño.
- **Using-Git-Worktrees** — sub-skill que crea entorno aislado en branch nuevo, con setup limpio y baseline de tests verde antes de empezar.
- **Writing-Plans** — sub-skill que toma el diseño y lo descompone en micro-tareas de 2-5 minutos con rutas exactas, código completo y verificaciones.
- **Test-Driven-Development** — TDD red-verde real + YAGNI + DRY como principios duros. Bloquea al agente cuando intenta añadir código no justificado.
- **Subagent-Driven-Development** — orquesta el ciclo: dispatcha subagente fresco por tarea, aplica revisión en dos fases (spec compliance + code quality).
- **Two-stage review** — la revisión en dos fases que cada subagente pasa antes de cerrar su tarea. Primera fase: cumple el plan. Segunda: el código está bien escrito.
- **YAGNI / DRY** — principios duros que TDD aplica. YAGNI bloquea abstracciones especulativas; DRY obliga a evitar duplicación.

Qué viene en M2.4

Instalación verificada con el comando del marketplace oficial, comprobación de que el procedimiento está activo en tu Claude Code, y dos gotchas que aparecen la primera vez (qué hacer si tu proyecto no soporta git worktrees y qué pasa la primera vez que el plan se siente demasiado granular). Diez minutos y sales con Superpowers operativo en tu setup.

M2.4 · Instalación y configuración de Superpowers

Bloque 0 · Qué cubrimos en cinco minutos

Tres cosas y se termina. La instalación con el comando exacto del marketplace oficial (la opción que en M2.2 dejamos como recomendada). La forma rápida de comprobar que el procedimiento está activo en tu Claude Code y no solo que el comando devolvió OK. Y los dos detalles que aparecen casi siempre la primera vez y que el README del proyecto no explica: qué hacer si tu proyecto no soporta git worktrees y qué pasa la primera vez que el plan que genera te suena demasiado granular.

Si vas a seguir el módulo en directo, abre tu Claude Code antes de seguir. Lo siguiente se ejecuta sobre la marcha.

Bloque 1 · La instalación

Un comando:

```
/plugin install superpowers@claude-plugins-official
```

Eso es todo. El sufijo `@claude-plugins-official` apunta al marketplace que mantiene Anthropic, que es la opción que justificamos en M2.2. Si por alguna razón decides ir por la versión comunitaria, recuerda que son dos pasos en lugar de uno y que la diferencia operativa empieza a importar en casos específicos.

Tres detalles a tener delante mientras corre:

Tarda más que un plugin pequeño. Superpowers trae más de veinte sub-skills, cada una con su carpeta de archivos. La instalación descarga todo y deja todo cargado. En conexiones lentas pueden ser uno o dos minutos.

Confirma con un mensaje de OK. Si todo va bien, ves un mensaje del estilo *"Plugin superpowers installed"* con el número de sub-skills cargadas. Si el número no aparece, revisa el siguiente bloque antes de seguir.

No tienes que reiniciar Claude Code. El plugin queda activo en la sesión actual. Las sesiones siguientes lo cargan automáticamente porque queda registrado en tu configuración.

Si el comando falla con un error de permisos, lo más probable es que tu Claude Code esté en un modo restringido. Cambia a modo normal con `/permissions` y vuelve a intentarlo.

Bloque 2 · Cómo saber que está activo

Que el comando termine con OK no garantiza que el procedimiento se active en tu próximo prompt. El plugin puede estar instalado pero no enganchado al flujo. La comprobación rápida:

Lanza una tarea de programación nueva. Cualquiera que no sea trivial. Algo del estilo:

"Añade un endpoint nuevo a la API que devuelva el historial de pagos del usuario."

Si Superpowers está activo, **no vas a ver código en la respuesta inmediata**. Vas a ver al agente entrando en modo Brainstorming: preguntas para refinar el diseño, propuesta de alternativas, mención de un documento de diseño que va a guardar. Esa es la señal.

Si en cambio el agente arranca a escribir código directamente sin preguntas, el plugin está instalado pero no se está activando. Causas posibles:

- **El prompt es demasiado pequeño para disparar Brainstorming.** El plugin no activa el procedimiento completo para tareas triviales. Si pides "renombra esta función", no espera Brainstorming. Asegúrate de probar con algo que justifique el ceremonial.
- **Tu Claude Code no está aplicando los plugins cargados.** Lanza `/plugins list` y comprueba que `superpowers` aparece como activo. Si aparece pero no se dispara, reinicia la sesión.
- **Hay un plugin conflictivo cargado.** Si tienes otro plugin que también intercepta el inicio de tareas, pueden chocar. Mira en la lista qué otros plugins están activos y desactiva temporalmente los que dudes.

Cuando el agente entra en Brainstorming sin que se lo pidas, ya está. El procedimiento corre.

Bloque 3 · Gotcha 1 — Si tu proyecto no soporta worktrees

Using-Git-Worktrees (M2.3, sub-skill número dos) intenta crear un worktree en cuanto el diseño queda aprobado. En proyectos estándar funciona transparente. En estos otros, no:

- **Monorepos con tooling propio** que asume que todo el código vive en una sola raíz (algunos setups de Nx, Turborepo o Bazel viejos). El worktree confunde a las herramientas.
- **Proyectos con submódulos git no inicializados** en el sitio donde crea el worktree.

- **Repositorios montados sobre filesystems remotos** (NFS, SMB) que no soportan algunas operaciones de worktree.

La señal de que estás en uno de estos casos: el plugin te dice algo del estilo *"No se pudo crear el worktree, paso a modo branch normal"*. No es un error grave; es el plan B integrado.

Qué hacer:

Si el modo branch normal te vale, sigue. La sub-skill trabaja con más precauciones (te avisa antes de cada commit grande, te pide confirmación cuando va a tocar archivos críticos), pero el resto del procedimiento funciona igual.

Si quieres worktrees a toda costa, configura un directorio externo dedicado para worktrees y configúraselo al plugin. La documentación oficial del repositorio cubre cómo hacerlo. En la mayoría de casos no compensa la complicación; en monorepos grandes con cambios paralelos sí.

Bloque 4 · Gotcha 2 — Cuando el plan se siente demasiado granular

La primera vez que veas un plan generado por Writing-Plans, la reacción común es: *"esto son demasiados pasos para algo tan tonto"*. Plan de veintitrés micro-tareas para añadir un endpoint que tú habrías escrito en quince minutos.

Es la reacción esperada y normalmente equivocada. Dos detalles que ayudan a leerla bien:

El plan está pensado para que cada paso sea verificable por separado, no para que sea eficiente en líneas escritas. Veintitrés pasos verificables es mejor que cinco pasos que pueden fallar en cualquier sitio sin que sepas dónde. La granularidad es la garantía de control.

El primer plan siempre se siente más largo que la pieza necesaria. A medida que el plugin aprende los patrones de tu proyecto (mira lo que ya hay, lee tu estilo de tests, ve tus convenciones), los siguientes planes son más densos. El primero es el que parece exagerado; del tercero en adelante, encaja.

¿Qué hacer si aun así un plan concreto te parece desproporcionado para la tarea?

Tienes la opción de pedir un plan más corto: *"este plan es demasiado largo para lo que pido, hazlo en menos pasos sin perder verificabilidad"*. El plugin lo rehace. No es ideal cogerlo como costumbre (estás luchando contra el procedimiento que instalaste a propósito), pero como excepción para tareas pequeñas funciona.

La regla razonable: las primeras cinco piezas, aguanta el plan tal como sale. Si llegado ese punto sigues pensando que es exagerado, ajusta. Habitualmente para entonces ya ha hecho clic.

Bloque 5 · Cierre y enlace a M2.5

Cinco minutos, plugin operativo, y dos gotchas en la mochila para cuando aparezcan. En el siguiente submódulo (M2.5) cerramos el módulo más largo del curso como cerramos el Mo: con la comparativa del mismo problema con y sin la skill activa. Ahí vas a ver en vivo lo que en este módulo hemos explicado: cómo cambia el procedimiento del agente, cuánto tarda la primera pieza, cuándo aparece la velocidad de crucero.

Si has llegado hasta aquí con Superpowers ya instalado y la comprobación pasada, vas con la pieza lista para la demo del siguiente submódulo.

Glosario rápido del bloque

- `/plugin install` — comando estándar de Claude Code para instalar un plugin desde un marketplace registrado.
- `@claude-plugins-official` — sufijo que apunta al marketplace oficial de Anthropic. La opción recomendada.
- `/plugins list` — comando de comprobación; muestra qué plugins están instalados y activos en la sesión actual.
- **Modo branch normal** — plan B de Using-Git-Worktrees cuando no puede crear worktree. Funciona con más precauciones; suficiente en la mayoría de casos.
- **Plan denso** — el plan que generas a partir de la pieza número tres en adelante, una vez Writing-Plans ha aprendido los patrones del proyecto. Lo notas porque deja de sentirse exagerado.

Qué viene en M2.5

Demo en vivo del mismo problema con y sin Superpowers. Tarea pequeña pero no trivial (añadir una ruta a la API con validación, persistencia y test). Comparativa de cinco dimensiones: tiempo total, número de turnos, código entregado, bugs latentes detectados a posteriori, y sensación de control. Siete minutos que cierran el módulo más largo del curso con la evidencia que sostiene todo lo dicho.

M2.5 · Demo — el mismo endpoint con y sin Superpowers

Bloque 0 · Lo que vamos a hacer

Tarea pequeña pero no trivial. La planteamos al agente en los dos escenarios:

- **Escenario A:** Claude Code limpio, sin Superpowers. Comportamiento por defecto.
- **Escenario B:** el mismo Claude Code, con Superpowers instalado y activo (M2.4).

El mismo prompt en los dos. El mismo proyecto. La misma rama de partida. Comparativa al final en cinco dimensiones que dejarán claro qué cambia y qué no.

Si grabas esta demo tú mismo, partimos del mismo punto: una mini-API en .NET 10 con ASP.NET Core, EF Core sobre SQLite, y xUnit v3 para tests. Es el stack del vault y aplica como ejemplo concreto.

Bloque 1 · La tarea

El prompt que vamos a soltar es el mismo en los dos escenarios:

"Añade un endpoint nuevo a la API. Recurso: `/customers/{id}/subscriptions/cancel`. Método: POST. Debe verificar que el usuario tiene una suscripción activa, marcarla como cancelada con la fecha actual, y devolver el estado actualizado. Si no hay suscripción activa, error 404."

La tarea tiene tres trampas que un dev senior huele a la legua y un agente con prisa no:

1. **Idempotencia.** Si el cliente llama dos veces seguidas, ¿qué pasa? La segunda debería ser un no-op limpio, no un error tipo "ya está cancelada".
2. **Concurrencia.** ¿Qué pasa si dos llamadas entran a la vez? Sin lock o validación optimista, la segunda puede sobrescribir el estado de forma inconsistente.
3. **Notificación de eventos.** Si la app tiene un sistema de eventos (común en .NET con MediatR o similar), la cancelación debería emitir un evento. Sin Brainstorming preguntando, el agente lo va a ignorar.

Las tres trampas son lo que separa una solución que "funciona" de una que sobrevive en producción. La comparativa del bloque 4 va a girar alrededor de cuál de los dos escenarios las pilla.

Bloque 2 · Escenario A — sin Superpowers

Soltamos el prompt. Cronometramos.

Lo que pasa en pantalla:

El agente lee el prompt, mira la estructura del proyecto, identifica que hay un `CustomersController`, y arranca a escribir. En unos cincuenta segundos te entrega:

- El método `CancelSubscription` añadido al controlador.
- Una llamada al servicio `SubscriptionService` con el método `CancelAsync(int customerId)`.
- Implementación del método en el servicio: busca la suscripción activa, si no hay devuelve 404, si hay actualiza el estado y devuelve.
- Un test que verifica que cancelar una suscripción activa devuelve 200, y otro que verifica que sin suscripción activa devuelve 404.

Los tests pasan. El código compila. Lo lees y parece razonable. Si tu reflejo es merge, ya estás dentro del problema que M2.1 describía.

Las tres trampas, una a una:

- **Idempotencia:** no contemplada. Si llamas dos veces, la segunda devuelve 404 porque ya no hay suscripción "activa". Comportamiento confuso para el cliente.
- **Concurrencia:** no contemplada. No hay lock optimista (`xmin` en EF Core o equivalente). Dos llamadas simultáneas pueden generar estado inconsistente sin que nadie se entere.
- **Notificación de eventos:** ignorada. El agente no preguntó si la app emite eventos al cancelar suscripciones. Si los emite, la integración con facturación va a fallar en silencio cuando el evento que se espera nunca llegue.

Tiempo de la primera entrega: **52 segundos**. Tiempo hasta detectar las tres trampas en code review humano posterior: **17 minutos**. Tiempo total real para una pieza merge-ready: **24 minutos** entre la primera entrega, la detección de bugs y los tres turnos extra de fix.

Bloque 3 · Escenario B — con Superpowers

Mismo prompt. Mismo proyecto. Reset de la rama. Plugin activo.

Lo que pasa en pantalla es distinto desde la primera línea.

El agente entra en Brainstorming. No escribe código. Te devuelve:

"Antes de implementarlo, cuatro preguntas. Una: ¿qué pasa si el cliente llama dos veces? ¿Idempotencia (200 con estado actual) o error explícito de 'ya cancelada'? Dos: ¿hay concurrencia esperable o estás cubierto con un lock optimista? Veo que el modelo `Subscription` no tiene campo de versión. Tres: ¿la cancelación debe emitir un evento de dominio? He visto que tienes `MediatR` cargado y los demás cambios de estado emiten eventos. Cuatro: ¿la fecha de cancelación es la de hoy o se acepta una fecha enviada por el cliente?"

Contestas las cuatro:

- Idempotencia, 200 con estado actual.
- Lock optimista; sí, añadir campo de versión al modelo.
- Sí, emitir evento `SubscriptionCancelled`.
- Fecha actual del servidor, no del cliente.

El agente guarda el documento de diseño. Pasa a Using-Git-Worktrees, crea la rama nueva, verifica baseline de tests verde. Pasa a Writing-Plans y te genera un plan de catorce micro-tareas. Lo apruebas. Subagent-Driven-Development ejecuta tarea por tarea con TDD red-verde y revisión en dos fases.

Cuando termina, la pieza entregada cubre:

- Endpoint con idempotencia explícita (verificado por test).
- Lock optimista con campo `RowVersion` en el modelo (verificado por test de concurrencia).
- Evento `SubscriptionCancelled` emitido por MediatR (verificado por test de integración con handler dummy).
- Fecha del servidor (verificado por test que usa `IClock` mockeable).

Tiempo de la pieza: **6 minutos 30 segundos**, con ceremonia de Brainstorming incluida. Las tres trampas pilladas en la primera entrega. Tiempo extra de fix posterior: **cero**.

Bloque 4 · La comparativa lado a lado

Dimensión	Sin Superpowers	Con Superpowers
Tiempo de la primera entrega	52 segundos	6 min 30 seg
Turnos hasta merge-ready	4 (primera + 3 fix)	1 (Brainstorming responde + 1)
Trampas pilladas en la entrega	0 / 3	3 / 3
Tests escritos por el agente	2 (caso feliz + 404)	7 (incluidos idempotencia, concurrencia, evento, fecha)
Tiempo total real merge-ready	24 minutos	6 min 30 seg
Sensación de control sobre las decisiones de diseño	Baja (las tomó el agente solo)	Alta (las tomaste tú con la info delante)

Cinco datos para llevarte:

El primer momento es engañoso. A los 52 segundos parece que el escenario A es cuatro veces más rápido. Sin contexto del code review posterior, ganaría. Con el contexto, el escenario B es casi cuatro veces más rápido en el tiempo real.

El número de tests cuádruplica. No por relleno; cada test extra cubre una de las trampas. La cobertura no es presunción de calidad: es la calidad medible que te ahorra el bug en producción.

Los turnos de fix posteriores desaparecen. En el escenario A, los tres turnos de fix son tiempo emocional caro: descubres el bug, te frustras, vuelves al agente, vuelves a revisar. En el B, cero.

La sensación de control es la dimensión menos cuantificable y la que más importa a largo plazo. En el escenario A, cuando termina, no sabes qué decisiones tomó el agente. En el B, las tomaste tú; las dejaste registradas en el documento de diseño; dentro de tres meses, cuando vuelvas, vas a saber por qué.

El extra de tiempo en Brainstorming se amortiza en la segunda pieza. La cuarta línea de la tabla (turnos hasta merge-ready) es donde se ve el efecto compuesto. Si haces ocho piezas al día, la del A te cuesta dos horas; la del B, una.

Bloque 5 · Cuándo no compensa Superpowers

Honestidad para cerrar bien el módulo. Hay casos donde el procedimiento es demasiado pesado para lo que la tarea pide:

- **Cambios cosméticos:** renombrar variables, ajustar mensajes de log, corregir un typo. Superpowers se activa con criterio (no dispara Brainstorming para tareas triviales), pero si te pasa puedes desactivarlo temporalmente.
- **Prototipos exploratorios donde no te importa el resultado.** Si estás probando si una librería sirve para algo y todo lo que escribes va a la papelera mañana, el procedimiento sobra.
- **Reproducciones de bug donde solo quieres ver el error.** Necesitas velocidad, no calidad.

La regla razonable: si la pieza va a vivir más de una semana en producción, el procedimiento compensa. Si va a vivir menos de un día, probablemente no.

Bloque 6 · Cierre del módulo M2 y enlace al M3

Hemos cerrado el módulo más largo del curso con la evidencia que justifica los cuarenta y cinco minutos invertidos. El procedimiento que Superpowers instala no es una moda; es la diferencia entre código que parece terminado y código que de verdad lo está.

El comportamiento por defecto de Claude Code no es malo: es rápido. La diferencia entre rápido y bueno se cierra cambiando el procedimiento, no recordándoselo en cada prompt.

Las cinco sub-skills clave de Superpowers (Brainstorming, Using-Git-Worktrees, Writing-Plans, Test-Driven-Development, Subagent-Driven-Development) forman una cadena que cubre el ciclo entero de desarrollo. El resto de sub-skills trabaja en segundo plano cuando hace falta.

El primer día sientes fricción; del tercero en adelante sientes velocidad real, no aparente. La fricción del primer día es síntoma de que el procedimiento está instalado, no de que sobre.

En el módulo siguiente (M3) entramos en GSD, la pieza que aborda otro problema distinto y complementario: cuando la tarea es demasiado grande para una sola sesión y necesitas que el agente la termine en varios chats sin perder el hilo. Vamos a ver tanto GSD original como su fork GSD Redux, y la situación de gobernanza que motivó la migración a este último.

Glosario rápido del bloque

- **Tarea no trivial** — pieza con al menos una trampa de diseño que un agente con prisa va a ignorar. La unidad de evaluación útil para comparativas de procedimiento.
- **Idempotencia** — propiedad de una operación que, ejecutada varias veces seguidas, produce el mismo resultado que ejecutada una sola.

- **Lock optimista** — técnica de concurrencia basada en versionar el registro. La actualización falla si la versión leída no coincide con la actual. En EF Core, `[ConcurrencyCheck]` o `RowVersion`.
- **Evento de dominio** — mensaje que se emite cuando un cambio de estado relevante ocurre. En .NET con MediatR, el patrón estándar.
- **Tiempo total real** — el tiempo desde el primer prompt hasta la pieza merge-ready, incluyendo todos los turnos de fix. La métrica honesta para comparar procedimientos.

Qué viene en M3

GSD (Get Shit Done) y su fork GSD Redux. Quinta sesión del agente sin contexto previo y termina la pieza igual. Cinco submódulos: el problema que aborda, la situación de gobernanza original con notificación honesta y redirección al fork con vida, las primitivas clave, instalación, y demo de continuidad entre sesiones largas.

PARTE 3

GSD · sesiones que sobreviven entre chats

M3.1 · Qué es GSD y por qué nació

Bloque 0 · La sesión que se acaba en mal momento

Llevas dos horas con Claude Code metido en una pieza compleja. Migración de un módulo de pagos, refactorización de una capa de servicios, lo que sea. La sesión ha generado un contexto enorme: archivos analizados, decisiones tomadas, un par de bugs descubiertos y arreglados sobre la marcha, tres ramas de discusión sobre cómo enfocar una parte.

Y entonces pasa algo. El servicio se cae, tu portátil se queda sin batería, te tienes que ir a una reunión que dura hora y media, o el contexto del agente llega al límite y te avisa de que la sesión necesita reiniciar.

Cuando vuelves, te encuentras ante una pantalla nueva. El agente no sabe nada. Las dos horas de trabajo viven en tu cabeza y en el código a medias. Tienes que decidir: o le explicas todo el contexto desde cero (cuarenta y cinco minutos de prompt) o te resignas a que la siguiente sesión empiece sin parte de la información buena, lo que probablemente meta inconsistencias en las decisiones que tome.

Si esta escena te ha pasado más de cuatro veces, este módulo va dirigido a ti. Y si has empezado a evitar las piezas grandes por miedo a este punto, estás cambiando lo que haces para acomodarte al límite de la herramienta. Mala señal.

Bloque 1 · El diagnóstico — el problema número dos

En M2.1 vimos el problema número uno: la prisa. La forma en que el agente, por defecto, prioriza entregar rápido sobre entregar bien. Aquí tenemos el segundo. **El contexto del agente no sobrevive a la sesión, y los apaños caseros para que sí lo haga no terminan de funcionar.**

Dos diferencias importantes con el problema de la prisa:

Este es un problema técnico real, no de comportamiento. El agente no decide olvidar; la conversación se trunca cuando el contexto llega al límite o cuando la sesión se reinicia. No hay procedimiento que evite que la sesión se acabe.

El daño no aparece en la sesión rota; aparece en la siguiente. Cuando reanudas el trabajo en una sesión nueva sin el contexto entero, las decisiones que el agente toma en esos

primeros turnos contradicen las que tomasteis en la sesión anterior. Inconsistencias sutiles que se cuelean en el código y aparecen en code review tres días después.

Y un detalle no obvio: cuanto más senior es tu uso de Claude Code, más sufres este problema. Porque las piezas que aboradas son más grandes, los contextos que construyes son más densos, y el coste de perder el hilo es mayor que para alguien que solo le pide al agente snippets sueltos. Para un usuario casual, el problema número dos es invisible. Para alguien que está metiendo a Claude Code en flujos serios, es el techo de cristal de lo que puede abordar.

Bloque 2 · Por qué los apaños caseros no llegan

La primera reacción de mucha gente cuando se da cuenta del problema es montarse un sistema casero. Las dos versiones más comunes:

Versión uno: un archivo `notas-sesion.md` que el agente actualiza al final de cada sesión con el estado, las decisiones y los pendientes. Al empezar la siguiente, le pides que lo lea.

Versión dos: confiar en el sistema de memoria del propio Claude Code (los archivos `~/.claude/projects/<hash>/memory/`). Le pides que escriba ahí lo importante y que recupere de ahí cuando empiece de nuevo.

Los dos funcionan a medias. Tres problemas concretos que aparecen pronto:

El agente no sabe qué guardar. Cuando le pides al final de la sesión "deja anotado lo importante para mañana", el agente guarda un resumen que parece bueno hasta que la siguiente sesión lo lee y descubre que falta justo el matiz que necesitaba. No es maldad: es que decidir qué es importante para mañana requiere haber tenido ya el problema de mañana.

La sesión nueva no recupera con criterio. El agente nuevo lee el archivo de notas, lo procesa, y arranca. Pero no aplica el filtro de "esto sí me importa para lo que estoy haciendo / esto no". Trabaja con todo o con parte arbitraria, dependiendo del orden en que lo encuentre.

No hay verificación de coherencia. Si la sesión vieja tomó una decisión y la sesión nueva la contradice sin darse cuenta, no hay nadie comprobando. El humano se entera cuando el código revienta o el code review levanta la mano.

Esto es lo mismo que pasaba en M2.1 con los recordatorios de "piensa antes de empezar": funcionan algunas veces, pero el comportamiento por defecto vuelve a aparecer cuando bajas la guardia.

La diferencia con el caso de la prisa es que aquí ni siquiera estamos cambiando el comportamiento del agente; estamos pidiéndole que asuma una responsabilidad (gestionar

continuidad entre sesiones) que no es propia del agente sino del flujo. La pieza que de verdad arregla esto tiene que vivir fuera del agente y orquestar la conversación entre sesiones con reglas. Eso es GSD.

Bloque 3 · Qué es GSD operativamente

GSD significa **Get Shit Done**. El nombre describe la promesa: que el trabajo se termine, no que la sesión se mantenga viva.

Es una skill instalable en Claude Code que añade un **protocolo de continuidad entre sesiones**. La idea es simple en la descripción y rica en las consecuencias: la skill define una estructura de artefactos persistentes en tu proyecto (no en el agente) que la sesión activa actualiza durante el trabajo y la sesión siguiente carga al arrancar. Cuatro detalles del protocolo importan:

Los artefactos son archivos concretos en tu proyecto, no estado opaco. Tú los puedes abrir, leer, editar a mano, versionarlos en git. Eso quita el miedo a que algo se quede colgado en un sitio inaccesible.

La actualización es continua, no solo al final. La skill empuja al agente a actualizar el estado del trabajo mientras avanza, no como tarea final. Eso elimina el problema de "se me olvidó guardar antes de que pasara X".

El handoff incluye lo que la siguiente sesión necesita decidir, no solo lo que pasó. La diferencia con un log es que el handoff está orientado al futuro: contiene los pendientes con suficiente contexto para que el agente nuevo retome con criterio.

El protocolo es agnóstico del modelo. Funciona con el mismo Claude del que partió la sesión vieja, con otro Claude, o con cualquier agente capaz de leer y seguir instrucciones de un archivo markdown. Eso le da longevidad: no depende de que ningún modelo específico siga existiendo.

En los siguientes submódulos entramos en los detalles operativos: las primitivas concretas que define la skill (M3.3), cómo se instala con el comando exacto verificado (M3.4), y la demo de continuidad entre dos sesiones reales (M3.5).

Bloque 4 · Lo que viene en el módulo (incluida una aclaración de gobernanza)

El módulo M3 son cinco submódulos:

- M3.1 (este): el problema que aborda GSD y por qué los apaños caseros no llegan.

- M3.2: qué es GSD y un aviso sobre la situación del proyecto original. Hay una historia de gobernanza que cambió cuál es el repo recomendado. Lo abordamos con honestidad, sin entrar en juicio largo, y dejamos claro a qué repo apuntar a fecha de hoy.
- M3.3: las primitivas clave del protocolo. Los artefactos que crea, cómo se actualizan, qué pinta tienen.
- M3.4: instalación, con el comando del repo recomendado y la comprobación de que el protocolo está activo.
- M3.5: demo de continuidad. Una pieza grande iniciada en una sesión, interrumpida a propósito, y retomada en otra sesión. Comparativa con la versión casera para que se vea la diferencia.

Cuarenta minutos en total. Es el segundo módulo más largo del curso, justo detrás del de Superpowers, porque GSD es la skill que más se nota en el día a día cuando empiezas a usar Claude Code para piezas serias. Sin Superpowers vas más lento; sin GSD, hay piezas que simplemente no terminas.

Bloque 5 · Cierre y enlace a M3.2

El problema número dos de Claude Code no es de comportamiento: es de protocolo. El agente no decide olvidar; la sesión se termina. Hace falta una pieza fuera del agente que gestione lo que pasa entre sesiones.

Los apaños caseros (archivo de notas, memoria del propio Claude) funcionan a medias porque el agente no sabe qué guardar con criterio y la sesión nueva no recupera con criterio. Hace falta un protocolo, no un recordatorio.

GSD es ese protocolo. Es plugin de continuidad: artefactos concretos en tu proyecto que la sesión activa actualiza y la siguiente carga. Antes de entrar en los detalles operativos en M3.3-M3.5, en el siguiente submódulo (M3.2) toca la aclaración sobre gobernanza que decide a qué repo de GSD apuntamos a fecha de hoy. No es largo, pero sin esa aclaración la instalación del M3.4 quedaría descontextualizada.

Glosario rápido del bloque

- **Problema número dos** — la imposibilidad de mantener el contexto del agente entre sesiones, y el daño que esto causa en piezas grandes. Lo que GSD aborda.
- **Protocolo de continuidad** — conjunto de reglas y artefactos que permite a una sesión retomar el trabajo de otra anterior sin perder información clave.

- **Handoff orientado al futuro** — la diferencia entre un log de lo que pasó y un documento que dice lo que la siguiente sesión necesita saber para decidir bien.
- **Apaño casero** — solución improvisada con notas markdown o memoria del agente. Funciona a medias por falta de criterio sobre qué guardar y qué recuperar.

Qué viene en M3.2

Qué es GSD operativamente, y la aclaración de gobernanza que define a qué repo apuntar. El proyecto original tuvo una situación que motivó la migración a un fork que la comunidad mantiene. Lo cubrimos con honestidad, sin alargar, y dejamos claro cuál es la opción recomendada para empezar.

M3.2 · Qué es GSD operativamente y la historia del rug-pull del token \$GSD

Bloque 0 · La promesa del submódulo

En M3.1 dejamos el diagnóstico: el problema número dos de Claude Code es la sesión que se acaba con la pieza a medias, y los apaños caseros no llegan. GSD es la skill que aborda ese problema y mucho más. En este submódulo vamos a hacer dos cosas:

Primero, ver qué es GSD operativamente. La explicación de fondo, no la versión simplista.

Segundo, contarte una historia. La de cómo GSD pasó en cuatro meses de ser el framework spec-driven más adoptado del ecosistema a tener al creador desaparecido tras un rug-pull del token cripto asociado, y a cómo la comunidad técnica respondió forkeando bajo gobernanza abierta. Es una historia real, ocurrida entre diciembre de 2025 y mayo de 2026, con fuentes públicas verificables.

No la contamos por chisme; la contamos porque es el ejemplo más limpio que tienes del valor de los cuatro filtros del Mo.3 aplicados al ecosistema real. Cuando termines este submódulo, vas a saber exactamente a qué repo apuntar hoy y vas a entender por qué tener criterio propio no es académico.

Bloque 1 · Qué es GSD operativamente

GSD significa **Get Shit Done**. La descripción oficial: "a light-weight and powerful meta-prompting, context engineering and spec-driven development system for Claude Code". Tres palabras técnicas que merecen desempaquetarse.

Meta-prompting. La skill no le añade conocimiento al agente; le añade un patrón de cómo trabajar. El agente sigue siendo el mismo Claude; lo que cambia es la estructura del diálogo que mantiene contigo, con sus subagentes y consigo mismo.

Context engineering. La capa que organiza qué información se mantiene en el contexto activo y qué se exterioriza a artefactos persistentes. El concepto central del problema número dos del M3.1: el agente no debería tener que recordarlo todo; lo importante vive fuera y se carga con criterio.

Spec-driven development. El patrón estructural. El trabajo se descompone en cinco fases ordenadas: **discuss** → **plan** → **execute** → **verify** → **ship**. Cada fase tiene entradas, salidas, y

artefactos verificables. Si has hecho SDDCC en este vault, vas a reconocer la columna vertebral inmediatamente; GSD es una de las implementaciones más maduras del patrón aplicado a Claude Code.

A alto nivel, las cuatro propiedades operativas que GSD instala:

Artefactos persistentes en tu proyecto. Archivos markdown concretos en una carpeta dedicada. Tú los puedes abrir, leer, editar a mano, versionarlos en git. El nombre y la cantidad exactos los veremos en M3.3, pero la idea es esa: el estado vive en disco, no en la cabeza del agente.

Subagentes paralelos con contexto limpio. Cada subtarea grande se delega a un subagente fresco que arranca con una ventana de hasta 200.000 tokens, sin arrastrar la conversación previa. El antídoto al context rot por construcción.

Multi-runtime. GSD no solo funciona con Claude Code. Soporta también Codex, Gemini CLI, Cursor, Windsurf, Augment, OpenCode, Kilo y media docena más. Catorce runtimes en total a fecha de hoy. Lo que aprendes con GSD se traslada si en algún momento cambias de cliente.

Auditabilidad. Los archivos del proyecto cuentan la historia entera. Code review puede tocar las decisiones igual que toca un endpoint. El estado del agente deja de ser opaco.

Eso es GSD en su forma técnica. Si solo me preocupara explicarte la skill, este bloque sería suficiente y el submódulo ya estaría cerrado. Pero hay una historia que necesitas conocer para decidir bien a qué repo apuntas hoy. Vamos a ella.

Bloque 2 · El origen — diciembre de 2025

GSD nace en diciembre de 2025. Lo publica un desarrollador con handle público **TÂCHES** (así escrito, con acento circunflejo) en GitHub bajo el owner `gsd-build`. URL del repositorio: github.com/gsd-build/get-shit-done.

La estructura técnica del proyecto desde el inicio:

- Aproximadamente 29 skills internas y 12 agentes especializados (`gsd-executor`, `gsd-planner`, `gsd-phase-researcher`, entre otros).
- CLI en Node.js con hooks para enganchar la skill al ciclo de vida de Claude Code.
- Aproximadamente 50 archivos markdown que definen las plantillas, los prompts y las convenciones del sistema.
- Instalación desde npm: `npx get-shit-done-cc@latest`.

El proyecto es técnicamente sólido desde el día uno. TÂCHES escribe documentación, responde issues, publica releases semanales. Hay un canal de comunidad activo donde la gente comparte casos de uso y reporta bugs. Todo aparenta normalidad de proyecto open source serio.

Hay un detalle que en su momento parece secundario y que más tarde explicará el desenlace. TÂCHES lanza también un **token cripto** asociado al proyecto, llamado `$GSD`, con su propia "foundation" (`@gsd_foundation`) que mantiene presencia en redes sociales y badges en el repo. La narrativa pública: el token sostendrá la gobernanza descentralizada del ecosistema GSD a futuro. El proyecto técnico y el proyecto cripto están imbricados desde el inicio.

Bloque 3 · La explosión de adopción — enero a marzo de 2026

GSD se vuelve viral muy rápido en el ecosistema Claude Code. Las cifras de los primeros cuatro meses, verificadas hoy contra el repositorio:

- **62.500 estrellas** en GitHub (a fecha de mayo de 2026).
- 5.300 forks.
- 138 contribuidores distintos.
- Más de 2.800 commits.
- 57 releases publicadas.
- 316 ramas activas, 228 tags de versión.

Para un proyecto de cuatro meses, son cifras de absoluto referente del nicho. Cuando alguien pregunta "¿qué uso para spec-driven en Claude Code?", la respuesta por defecto es GSD. Cuando un youtuber del nicho graba un vídeo de top skills (como el CC6SK que inspira este curso), GSD aparece sí o sí. Cuando una empresa pequeña adopta Claude Code para sus flujos, lo más probable es que termine probando GSD en algún momento.

La adopción no es solo en Claude Code. Como GSD soporta catorce runtimes, gente de Codex, Cursor, Windsurf y Gemini CLI también lo incorpora. Es uno de esos proyectos que define cómo se hace la cosa en el ecosistema entero, no solo en su nicho original.

A finales de marzo de 2026, GSD es probablemente el framework spec-driven más adoptado del mundo agente.

Bloque 4 · La señal de alerta — 1 de abril de 2026

A primeros de abril, algo cambia. TÂCHES, el creador y owner del repositorio, **deja de aparecer**:

- Para de responder a issues.
- Para de hacer commits.
- Sus cuentas en redes sociales (X/Twitter y similares) se eliminan o se quedan inaccesibles.
- La cuenta `@gsd_foundation` también cesa la actividad pública.
- Última actividad confirmada del autor: **1 de abril de 2026**.

Para un proyecto open source pequeño, una semana sin actividad del mantenedor es normal. Para un proyecto con 60.000 estrellas, 138 contribuidores y miles de devs que dependen de él en producción, una semana ya empieza a generar conversación. Dos semanas, alarma. Cuatro, búsqueda activa de respuestas.

Los colaboradores del proyecto intentan contactar. Sin éxito. Los issues abiertos se acumulan sin respuesta del owner. Los PRs no se mergean. La gente empieza a preguntar en públicamente: "¿qué pasa con GSD?".

Bloque 5 · La conexión con el token \$GSD

Aquí está el matiz que el vídeo origen del curso no cubre y que explica por qué el silencio. El silencio de TÂCHES no es coincidencia con un episodio cripto. Es **el efecto** de un episodio cripto.

Lo que ocurrió, según fuentes públicas verificables (AI Weekly, OurCryptoTalk, varios analistas del nicho):

Aproximadamente entre finales de marzo y primeros de abril de 2026, TÂCHES **vendió de golpe todas sus tenencias del token \$GSD** en el mercado donde cotizaba. Esto es lo que en el argot cripto se conoce como **rug-pull**: el creador del proyecto liquida su posición concentrada en un movimiento que hunde el valor para quien quedó con el token comprado.

Estimaciones de la comunidad cripto: aproximadamente **500.000 dólares estadounidenses** en valor extraído de la gente que había comprado `$GSD` confiando en que era inversión en el ecosistema técnico.

Y entonces TÂCHES desaparece. Cuentas borradas, cero comunicación pública, cero respuesta a contacto privado de colaboradores.

Tres apuntes honestos para entender bien lo que significa esto:

El proyecto técnico GSD sigue siendo técnicamente sólido. El rug-pull es del token cripto asociado, no de la calidad del software. Cualquiera que clone el repo el 2 de abril tiene en sus manos el mismo sistema que el 31 de marzo.

La gobernanza del repositorio sí queda comprometida. El owner del repo en GitHub sigue siendo TÂCHES, ahora ausente. Los paquetes npm (`get-shit-done-cc`, `@gsd-build/sdk`) siguen bajo cuentas controladas por una persona que ha demostrado mala fe y de la que no se sabe nada. Si esas cuentas se vieran comprometidas por hackeo, o si TÂCHES decidiera publicar una versión maliciosa más tarde, miles de proyectos podrían verse afectados.

El problema no es técnico; es de cadena de suministro. La pregunta no es "¿funciona GSD?". La pregunta es "¿confías en que el siguiente release no será un caballo de Troya?".

Bloque 6 · La respuesta de la comunidad — 22 de mayo de 2026

Aquí entra **trek-e**, uno de los colaboradores activos del proyecto desde sus primeros meses. trek-e tiene un problema gordo enfrente:

- El código técnico sigue siendo bueno y útil para miles de personas.
- El owner ha desaparecido tras un rug-pull público.
- La cadena de distribución (npm packages, GitHub repo) es insegura por construcción mientras TÂCHES sea el único con control formal.
- La comunidad técnica necesita un sitio claro al que migrar.

El **22 de mayo de 2026** (hace exactamente siete días desde la fecha en que este submódulo está siendo escrito), trek-e abre github.com/open-gsd/get-shit-done-redux. Es un fork comunitario con cuatro decisiones clave:

Preservación de la atribución técnica. Todo el código, commits, ramas, tags y créditos a TÂCHES se mantienen idénticos. El fork no esconde el origen ni reescribe la historia; la respeta. trek-e deja claro en el README que el trabajo técnico original es de TÂCHES y que la decisión de forkear es para preservar la continuidad del proyecto, no para apropiárselo.

Renombrado de paquetes npm para que la cadena de suministro sea segura desde cero. El paquete antiguo `get-shit-done-cc` queda obsoleto y se reemplaza por `get-shit-done-redux` bajo el scope `@opengsd`. El SDK pasa de `@gsd-build/sdk` a `@gsd-redux/sdk`. Cualquier proyecto que actualice los nombres pasa a depender de paquetes bajo control de la nueva gobernanza.

Eliminación de los elementos del token cripto. Los badges de `@gsd_foundation` y las referencias al token `$GSD` se quitan del fork. El proyecto técnico se separa formalmente del proyecto cripto. La narrativa del fork es: "esto es software open source, no especulación".

Auditoría de seguridad pública. El equipo nuevo realiza una revisión de seguridad del código heredado y publica el resultado en abierto, para que cualquier usuario pueda verificar que no hay

puertas traseras dejadas por el autor original.

El comando oficial de instalación del fork, verificado a fecha de hoy:

```
npx @opengsd/get-shit-done-redux@latest
```

Primera release publicada bajo la gobernanza nueva: **v1.1.0, el 24 de mayo de 2026**. Renumeración limpia desde el fork. La discusión que documenta la decisión y la migración es pública en GitHub bajo el título *"Welcome to get-shit-done-redux — why the fork, what changed, what's next"*.

Bloque 7 · El estado raro del repo original a fecha de hoy

Aquí viene un detalle que despista a quien mira el repositorio original ahora mismo y por el que es fácil pensar que nada ha pasado. **El repo `gsd-build/get-shit-done` SIGUE TENIENDO ACTIVIDAD**. Si lo abres a fecha de hoy, te encuentras:

- 62.500 estrellas (la cifra sigue creciendo porque mucha gente desconoce la historia).
- Commits recientes, de hace pocas horas.
- Releases nuevas con números de versión.
- About con el texto original "by TÂCHES".

¿Es esto contradictorio con todo lo que acabas de leer?

No, y tiene una explicación operativa elegante. **trek-e también mantiene el repo original en modo "transición"**, añadiendo a cada release notas que avisan a los usuarios que llegan ahí: "el proyecto se ha movido, el desarrollo activo está en `open-gsd/get-shit-done-redux`, los paquetes npm a usar son los nuevos". Los commits que ves de hace 4 horas son de trek-e haciendo precisamente eso: documentar la migración y enlazar al fork para que nadie llegue al original y se quede sin saber lo que pasó.

El About sigue diciendo "by TÂCHES" por dos razones: una, porque el código en origen es de TÂCHES y la atribución es correcta. Dos, porque trek-e no es el owner formal del repo (el owner sigue siendo `gsd-build`); está manteniéndolo en cooperación temporal.

Para alguien que llega al repo original sin contexto y lee solo el About y las estrellas, todo parece normal. Para alguien que lee las release notes recientes y entra en los issues, queda claro que el proyecto activo es el fork.

Esta es la trampa que vamos a ayudarte a esquivar.

Bloque 8 · Qué significa esto para quien ve el vídeo origen hoy

El vídeo origen del curso CC6SK (la inspiración de TOPSKL) se grabó antes del 22 de mayo de 2026. El presentador no podía saber lo que iba a pasar. En su vídeo enseña en pantalla el repo `gsd-build/get-shit-done` con sus 60.000 estrellas y lo recomienda como instalación. En su momento, esa recomendación era razonable; era el repo de referencia del ecosistema.

Si alguien ve ese mismo vídeo hoy y sigue la recomendación tal cual, va a:

- Llegar al repo original.
- Posiblemente leer el aviso de migración o posiblemente saltárselo (mucha gente no lee READMEs enteros).
- Instalar el paquete antiguo `get-shit-done-cc` con `npm get-shit-done-cc@latest`.
- Quedarse en una versión que no recibe updates ni auditoría de seguridad del fork.
- Quedar fuera de la comunidad técnica que ahora vive en `open-gsd`.

El vídeo no se equivoca; quedó obsoleto en cuestión de días. Es un riesgo intrínseco de cualquier curso o tutorial sobre ecosistemas vivos: la realidad cambia más rápido que el material grabado.

Y aquí entra la pieza que sí podemos aportar nosotros desde TOPSKL.

Bloque 9 · Por qué esta historia justifica el curso entero

En Mo.3 te dimos cuatro filtros para evaluar cualquier skill antes de incorporarla a tu flujo: **viva, adoptada, reversible, compone**. En ese momento eran abstractos; podían sonar a teoría académica.

La historia de GSD es la demostración viva de que esos cuatro filtros no son académicos. Vamos a aplicarlos al caso real para que veas cómo el criterio te salva.

Filtro "viva". Pregunta operativa: "¿quién mantendrá esto dentro de seis meses?". Si tu única referencia es el vídeo origen, la respuesta parecería "TÂCHES, el creador, claramente mantiene esto, lleva meses respondiendo". Si aplicas el filtro hoy mirando datos actuales (commits del owner formal, presencia del autor en redes, actividad de issues), te das cuenta de que el owner formal está ausente, el último commit del propio TÂCHES es de hace casi dos meses, y los commits recientes son todos de trek-e. El filtro "viva" del repo original falla. El del fork pasa con holgura.

Filtro "adoptada". Lecturas honestas que aprendiste en M2.2: las estrellas son señal débil; importa la actividad de la conversación. El repo original tiene 60.000 estrellas pero su

conversación activa está toda redirigiendo al fork. La conversación viva del proyecto está en `open-gsd`. El filtro te empuja al sitio correcto si lo aplicas con la heurística del curso.

Filtro "reversible". ¿Te quedas atrapado si instalas la opción equivocada? Aquí GSD es bueno: el paquete se desinstala con un comando, los artefactos que crea son markdown legible, y migrar del original al fork solo requiere actualizar el nombre del paquete npm. El filtro pasa fácil en cualquiera de las dos opciones; pero saber esto te quita el miedo a probar antes de comprometerte definitivamente.

Filtro "compone". El fork compone igual de bien que el original (mismo código, misma arquitectura). El original también; en eso no hay diferencia.

Resultado de aplicar los cuatro: la versión recomendada es claramente el fork. Si no aplicas los filtros, sigues la recomendación literal del vídeo y te quedas en una zona insegura. Si los aplicas, llegas al sitio correcto en cinco minutos de verificación.

Por eso este curso vale más que el vídeo origen. No porque el vídeo esté mal; porque el ecosistema muta y necesitas criterio propio para navegar las mutaciones. El criterio es el M0.3.

Bloque 10 · Los cuatro filtros del M0.3 aplicados al fork

Para cerrar formalmente el bloque didáctico, los cuatro filtros aplicados a `open-gsd/get-shit-done-redux` como opción recomendada:

Viva: sí. Equipo identificado (trek-e + colaboradores), commits semanales, releases regulares (v1.1.0 hace cinco días), issues con respuesta. La situación que motivó el fork también motivó que la comunidad se organizara con responsabilidades claras de mantenimiento; eso es bonus.

Adoptada: sí, con la lectura honesta. Las estrellas absolutas son aún modestas comparadas con el original porque el fork tiene siete días de vida. Pero la conversación técnica activa, los issues respondidos, los releases publicados, y las redirecciones desde el repo original son señales claras de que la comunidad técnica viva está aquí.

Reversible: sí. Comando único de desinstalación, artefactos en markdown editable, sin lock-in. Si pruebas el fork y no te encaja, lo quitas en treinta segundos.

Compone: sí, y de forma destacable. Soporta catorce runtimes; trabaja bien con Superpowers (M2) en el sentido de que el flujo de M2 puede empujar decisiones a los artefactos de GSD para que la siguiente sesión las herede; complementa a `/review` y `/ultrareview` (M4) porque los artefactos de decisión son contexto útil para la revisión asistida. En el M7 vamos a montar el flujo combinado completo.

Cuatro de cuatro. Por eso GSD entra al catálogo del curso, y por eso lo hace con la URL del fork.

Bloque 11 · Cierre y enlace a M3.3

GSD es la skill spec-driven más adoptada del ecosistema Claude Code y una de las piezas técnicas más serias del catálogo. Cinco fases (discuss, plan, execute, verify, ship), subagentes paralelos con 200K tokens limpios, multi-runtime, auditabilidad por artefactos versionables. Eso es lo técnico, y eso no ha cambiado.

La historia entre diciembre de 2025 y mayo de 2026 (origen → explosión → desaparición de TÂCHES → rug-pull del token \$GSD → fork comunitario por trek-e bajo `open-gsd`) es la demostración viva de para qué sirven los cuatro filtros del Mo.3. Aplicar los filtros te lleva a `open-gsd/get-shit-done-redux` como opción recomendada hoy. No aplicarlos te deja en zona insegura.

El comando real verificado es `npx @opengsd/get-shit-done-redux@latest` y entra en detalle en M3.4. Antes, en M3.3, vamos a ver los artefactos concretos que la skill define: qué archivos crea en tu proyecto, qué contiene cada uno, y cómo evolucionan a lo largo de una sesión real. Sin eso, la instalación sería un comando ciego.

Glosario rápido del bloque

- **TÂCHES** — handle público del creador original de GSD. Desaparecido tras rug-pull del token `$GSD` el 1 de abril de 2026. Información pública verificable en medios cripto.
- **trek-e** — colaborador histórico del proyecto. Líder del fork comunitario `open-gsd/get-shit-done-redux` desde el 22 de mayo de 2026. También mantiene el repo original en modo transición, añadiendo notas de migración.
- **Rug-pull** — término técnico cripto. El creador de un token vende su posición concentrada de golpe y hunde el valor para los demás. Episodio documentado del token `$GSD`.
- `gsd-build/get-shit-done` — repositorio original de GSD por TÂCHES. Sigue existiendo en modo transición. **No recomendado** por este curso; usa el fork.
- `open-gsd/get-shit-done-redux` — fork comunitario bajo gobernanza abierta. **Opción recomendada** por este curso. Comando: `npx @opengsd/get-shit-done-redux@latest`.
- **Cadena de suministro de software** — el conjunto de cuentas, paquetes y repositorios que tienes que confiar para que un software llegue a tu máquina sin alteraciones. La razón técnica más fuerte para preferir el fork frente al original.

Qué viene en M3.3

Las primitivas concretas del protocolo. Los archivos que GSD mantiene en tu proyecto, qué contiene cada uno, cuándo se actualizan, y cómo evolucionan a lo largo de una sesión de trabajo real. Quince minutos para tener la mecánica de la skill clara antes de instalarla en M3.4.

M3.3 · Las primitivas clave del protocolo

Bloque 0 · Lo que vamos a ver

GSD se sostiene sobre tres archivos. No es por gusto del autor: cada uno cubre una función distinta del protocolo. Verlos por separado, entender qué contiene cada uno y cuándo se actualiza, es lo que convierte la idea abstracta del M3.2 ("protocolo de continuidad") en algo concreto que puedes leer, editar a mano, y versionar en git como cualquier otro artefacto de tu proyecto.

En este submódulo:

- Por qué tres archivos y no uno solo.
- El **plan vivo**: qué contiene, cuándo se actualiza, ejemplo.
- El **archivo de decisiones**: qué contiene, cuándo se actualiza, ejemplo.
- El **handoff**: qué contiene, cuándo se actualiza, ejemplo.
- Cómo se mueven los tres a lo largo de una sesión real. Reutilizamos la tarea de M2.5 (el endpoint de cancelación de suscripciones en .NET) para que no tengas que mantener una situación nueva en la cabeza.
- Dos detalles que no son obvios la primera vez.

Una aclaración antes de seguir: en el fork `open-gsd/get-shit-done-redux` los artefactos reales son cinco archivos markdown (`PROJECT.md` , `REQUIREMENTS.md` , `ROADMAP.md` , `STATE.md` , `CONTEXT.md`) que cubren entre todos los tres roles que vamos a ver aquí. En este submódulo enseñamos los roles porque son lo que tienes que entender para usar la skill; al final del submódulo mapeamos los roles con los cinco archivos concretos para que cuando los veas en M3.4 ya sepas qué es cada uno.

Bloque 1 · Por qué tres archivos

La primera vez que ves la descripción de GSD piensas: "¿no es lo mismo poner todo en un único archivo de estado?". Razonable. Y la respuesta es no, por tres motivos:

Las tres primitivas se actualizan en frecuencias muy distintas. El plan vivo cambia decenas de veces dentro de una sesión, conforme tareas se completan o aparecen pendientes nuevos. Las decisiones se añaden cuando se cierra una opción de diseño, una vez por tema. El

handoff se reescribe entero al final de cada sesión y al inicio de la siguiente. Mezclarlos en un solo archivo significa releer todo cada vez que se toca cualquier parte.

Las tres tienen audiencias distintas en el tiempo. El plan vivo lo lee la sesión actual constantemente. Las decisiones las lee la sesión actual cuando duda y la sesión futura cuando audita. El handoff lo lee solo la sesión que retoma. Separarlos permite que cada uno tenga el formato que mejor sirve a su lector.

El versionado se beneficia de la separación. Si todo está en un único archivo, cada cambio del plan vivo aparece en git como modificación del documento entero, mezclado con cambios de decisiones que sí merecen estar ahí. Separando, el diff de cada archivo cuenta su propia historia.

La intuición que ayuda: piensa en los tres como capas de un sistema. El **plan vivo** es el *qué estoy haciendo ahora*. Las **decisiones** son el *por qué hago lo que hago*. El **handoff** es el *qué necesita saber el que venga después*.

Bloque 2 · El plan vivo

El primer rol. El más volátil de los tres. Vive en la carpeta de artefactos del proyecto (M3.4 te enseña la ubicación concreta) y en el fork se materializa principalmente en `ROADMAP.md`.

Qué contiene:

- La lista de pendientes con suficiente contexto para que un agente nuevo entienda cada uno sin tener que mirar el código. No solo "implementar endpoint" — "implementar endpoint POST /customers/{id}/subscriptions/cancel con idempotencia explícita, lock optimista por RowVersion, y emisión de evento SubscriptionCancelled".
- La lista de completados, con marca temporal y referencia al commit donde se cerró cada uno. Esto sirve para que la sesión nueva pueda inspeccionar exactamente qué quedó hecho.
- Un campo de "en curso" con la tarea concreta en la que estaba la sesión cuando se interrumpió, y el estado al que llegó.

Cuándo se actualiza: continuamente. La skill empuja al agente a actualizar el plan cada vez que cierra una tarea, abre una nueva, o detecta un pendiente que no estaba previsto. No es tarea final; es ritmo de la sesión.

Pinta aproximada (con la tarea de M2.5 como ejemplo):

Plan vivo - cancel-subscription**## En curso**

- Test de concurrencia para CancelAsync: añadir caso de dos llamadas simultáneas

Pendientes

- [] Test de emisión de evento SubscriptionCancelled (handler dummy)
- [] Validar que el evento se persiste en la outbox antes del commit
- [] Documentar el comportamiento de idempotencia en el README del módulo

Completados

- [x] Añadir endpoint POST /customers/{id}/subscriptions/cancel (commit a3f2d1c)
- [x] Implementar CancelAsync con verificación de suscripción activa (commit b89c2e7)
- [x] Añadir campo RowVersion al modelo Subscription (commit f471a90)
- [x] Test base de cancelación de suscripción activa (commit 5d23b1e)

Tres cosas a notar:

El nivel de detalle en pendientes y en-curso es suficiente para que un agente nuevo retome sin perderse. Frases descriptivas, no etiquetas crípticas.

Los completados llevan referencia de commit. La sesión nueva puede ir al diff exacto si necesita entender qué se hizo y cómo.

No hay narrativa larga; es lista. La narrativa vive en los otros dos archivos.

Bloque 3 · El archivo de decisiones

El segundo. Cambia menos. Es el archivo más valioso a medio plazo, porque captura el *por qué* que el código solo no cuenta.

Qué contiene: una entrada por decisión de diseño cerrada. Cada entrada tiene:

- **Decisión:** qué se decidió, en una frase.
- **Contexto:** qué pregunta había sobre la mesa.
- **Alternativas consideradas:** las opciones que se descartaron y por qué.
- **Quién decidió:** si fue el humano respondiendo a Brainstorming, si fue el agente con la confirmación posterior, o si fue una decisión arrastrada de otra parte del código.

Cuándo se actualiza: cuando se cierra una decisión. Una vez por tema. No se reescribe (las decisiones no se modifican, se invalidan con una nueva entrada que apunta a la anterior).

Pinta aproximada:

Decisiones – cancel-subscription

D-001 · Idempotencia: 200 con estado actual

- **Contexto**: ¿qué pasa si el cliente cancela dos veces seguidas?
- **Alternativas**: (a) error 409 explícito, (b) 200 con estado actual.
- **Elegida**: (b). Razón: el cliente facturación llama por error a veces y un 409 le hace caer a un flujo de incidencia que no aporta. El 200 con estado actual es seguro y mantiene trazabilidad de la cancelación original.
- **Quién**: humano, respuesta a Brainstorming en sesión del 2026-05-28.

D-002 · Concurrencia: lock optimista con RowVersion

- **Contexto**: ¿hay riesgo de cancelaciones simultáneas?
- **Alternativas**: (a) lock pesimista a nivel DB, (b) lock optimista con RowVersion, (c) sin lock.
- **Elegida**: (b). Razón: (a) bloquea más de lo necesario para una tabla con tráfico bajo, (c) genera estado inconsistente sin que nadie se entere. (b) es el patrón estándar de EF Core para este caso.
- **Quién**: humano, misma sesión.

D-003 · Evento de dominio SubscriptionCancelled emitido por MediatR

- **Contexto**: ¿la cancelación notifica al resto de la app?
- **Alternativas**: (a) no notificar (asume polling externo), (b) evento de dominio MediatR.
- **Elegida**: (b). Razón: el resto de cambios de estado de la app usan MediatR, mantener la consistencia evita que facturación falle cuando el evento que esperan no llega.
- **Quién**: humano, misma sesión.

Tres cosas a notar:

Las entradas son referenciables por código (D-001 , D-002). En el commit donde se aplique la decisión, el mensaje puede mencionar `Aplica D-002`. Tres meses después, cuando alguien lea el código, sabe exactamente dónde mirar para entender el por qué.

Las decisiones no se borran ni se editan. Si una se invalida más tarde, se añade una `D-014 · Invalida D-002 – cambiamos a lock pesimista por motivo X` que apunta atrás. La trazabilidad se conserva.

El estilo es seco. Decisión, contexto, alternativas, razón, autor. Sin narrativa decorativa. Este archivo es el documento que más leerás dentro de seis meses.

Bloque 4 · El handoff

El tercero. El más narrativo de los tres, y el único que se reescribe entero (no se acumula).

Qué contiene: lo que la próxima sesión necesita saber para retomar sin preguntar. Tiene dos secciones obligatorias:

- **Estado:** dónde quedó el trabajo. Una o dos frases.
- **Pendiente inmediato:** la siguiente tarea concreta a abordar, con el contexto justo. No el plan entero (eso ya está en el archivo de plan vivo); solo la siguiente.

Y normalmente una sección tercera:

- **Trampas y avisos:** cosas que la sesión nueva debería tener en cuenta y no son obvias del plan ni del código. "El test de concurrencia que estaba escribiendo falla con SQLite en memoria pero pasa con SQLite en disco". "El campo RowVersion necesita migración nueva si esta rama se fusiona". Cosas operativas.

Cuándo se actualiza: al final de cada sesión, antes de cerrar. La skill se asegura de que el agente actualice el handoff antes de terminar, incluso si la sesión termina por agotamiento de contexto. Al inicio de la siguiente sesión, el handoff es lo primero que se lee y, una vez asumido, se reescribe limpio para la siguiente parada.

Pinta aproximada:

Handoff – cancel-subscription (sesión 2026-05-29 14:32)

Estado

La pieza está al 70%. Endpoint funcionando con idempotencia y RowVersion. Faltan los tests de concurrencia y de emisión de evento. La rama feature/cancel-subscription tiene 4 commits y el baseline de tests pasa salvo lo pendiente.

Pendiente inmediato

Implementar el test de concurrencia para CancelAsync. Caso a cubrir: dos llamadas simultáneas sobre la misma suscripción activa, esperar que una gane (200) y la otra falle con DbUpdateConcurrencyException convertido a 409.

Trampas y avisos

– El test de concurrencia falla con SQLite en memoria por una limitación conocida del provider.

Usar SQLite en archivo para este test. Patrón ya existe en tests/Helpers/SqliteFixture.cs.

– El evento SubscriptionCancelled no debe emitirse en las llamadas idempotentes (segunda

llamada sobre cancelación ya cerrada). Si lo emites, MediatR notifica dos veces a facturación.

Esta decisión está en D-001 pero el test no la cubre aún – añadirlo después del de concurrencia.

– La migración del RowVersion vive en Migrations/20260528_AddRowVersionToSubscription.cs.

Si se hace rebase contra main, comprobar que no haya conflicto de orden de migraciones.

Tres cosas a notar:

El handoff es el archivo más leído por la sesión nueva y el menos releído después. Si la pieza llega bien, este archivo deja de tener valor. Por eso se reescribe en lugar de acumularse.

Las trampas son la parte más valiosa. Es lo que ahorra a la siguiente sesión los veinte minutos de descubrir por sí misma lo que ya se descubrió.

El tono es operativo, directo. Sin frase ceremonial de saludo a la sesión nueva, sin narrativa larga. La gracia del handoff es lo que dice, no cómo lo dice.

Bloque 5 · Cómo se mueven los tres en una sesión real

Volvemos a la tarea del endpoint de cancelación de suscripciones. Sesión real, hora a hora:

Minuto 0. Arrancas la sesión. La skill carga el plan vivo, las decisiones, y el handoff. El handoff dice "implementar el test de concurrencia". La sesión nueva sabe dónde ir.

Minuto 5. El agente lee el código, los tests existentes, el patrón de `SqliteFixture` que el handoff menciona. Empieza a escribir el test.

Minuto 12. Surge una duda nueva: el test de concurrencia necesita que `CancelAsync` reciba el `RowVersion` desde el endpoint, pero ahora mismo el endpoint lo ignora. ¿Lo aceptamos como header o como query? Decisión nueva. El agente entra en Brainstorming, propone alternativas, espera tu respuesta. Eliges header. **El archivo de decisiones gana una entrada nueva, D-004.**

Minuto 18. El test pasa. El agente actualiza el plan vivo: el en-curso pasa a completado con referencia al commit. Pasa a la siguiente tarea pendiente (test de evento). **El plan vivo se modifica.**

Minuto 35. Implementación del test del evento, otra micro-decisión menor (qué nombre poner al handler dummy). Esta vez no entra al archivo de decisiones porque es operativa, no de diseño. El plan vivo absorbe el cambio cuando el test queda completado.

Minuto 47. Te toca irte. Le pides a la sesión que cierre. La skill ejecuta el protocolo de cierre: actualiza el plan vivo con el estado actual, deja una entrada nueva en el archivo de decisiones si la había pendiente, y **reescribe el handoff** con el estado para la próxima vez.

Tres días después. Sesión nueva. Misma rama. El agente lee el handoff: pendiente inmediato es "documentar el comportamiento de idempotencia en el README". Trampas: "el README usa una plantilla específica del proyecto que pide tono Q&A". El agente arranca con criterio. No te pregunta lo que ya estaba decidido tres días atrás.

Lo que esta pequeña narrativa hace ver es que los tres archivos no son ceremonia. Cada uno hace una cosa y se actualiza cuando le toca. La sesión sigue sintiéndose conversacional; el coste de mantenimiento del protocolo es bajo porque el agente lo gestiona como parte de su flujo.

Bloque 6 · Dos cosas que no son obvias la primera vez

Los artefactos viven en una carpeta dentro de tu proyecto, no en una ubicación global. Eso significa que cada proyecto tiene su propio conjunto de archivos. Y eso significa que se versionan en git con el resto del código. Decisión consciente del proyecto: si compartes el repo con un equipo, los archivos viajan con el código.

Consecuencia operativa: trata esa carpeta como código. Code review puede tocar las decisiones igual que toca un endpoint. Si una entrada del archivo de decisiones no te encaja, lo dices en code review y se ajusta.

El handoff se reescribe entero, pero el reescrito anterior no se pierde si lo necesitas. La skill mantiene un histórico de handoffs en una subcarpeta de la carpeta de artefactos. Si en algún momento necesitas ver el handoff de hace dos semanas, está ahí. No lo vas a usar a menudo, pero saber que existe quita el miedo a "lo borré sin querer". El histórico también ayuda cuando una pieza se queda en pausa larga y vuelves dos meses después: el último handoff puede haber quedado desactualizado y el histórico te muestra dónde fue cambiando el foco.

Bloque 7 · Los cinco archivos reales del fork

Toca cerrar el bucle entre la abstracción didáctica de los tres roles y los cinco archivos concretos que vas a ver en tu proyecto cuando instales GSD desde el fork. El fork `open-gsd/get-shit-done-redux` mantiene cinco artefactos persistentes que sobreviven a los límites de sesión:

- **PROJECT.md** — descripción de qué es el proyecto. El más estable de los cinco; se escribe pronto y cambia poco. Lo primero que una sesión nueva mira para entender qué tiene entre manos.
- **REQUIREMENTS.md** — los requisitos detallados de la pieza activa, derivados de la fase *discuss* y validados antes de pasar a *plan*. Estable durante una pieza, se reemplaza cuando empieza otra.
- **ROADMAP.md** — el plan vivo. Pendientes, completados, en-curso. El más volátil. Es donde se materializa el primer rol que vimos (el qué se está haciendo).
- **STATE.md** — el estado operativo de la sesión activa, equivalente cercano al handoff. Lo lee la sesión que retoma; se reescribe al cierre.

- **CONTEXT.md** — el contexto persistente del proyecto entero: decisiones de diseño cerradas, convenciones aplicadas, aprendizajes acumulados a lo largo de varias piezas. El que más vale a medio plazo, equivalente cercano al rol "decisiones" del bloque 3.

El mapeo a los tres roles no es uno-a-uno sino "varios a tres": dos archivos cubren matices del rol "qué" (**REQUIREMENTS.md** para lo planificado, **ROADMAP.md** para lo vivo), uno cubre el "por qué" con persistencia larga (**CONTEXT.md**), y otro cubre el "qué necesita saber el siguiente" con persistencia corta (**STATE.md**). **PROJECT.md** se sitúa por encima como contexto de partida.

Tres detalles que te ayudan a navegar los cinco archivos sin abrumarte:

No tienes que actualizarlos a mano. La skill empuja al agente a actualizar el que corresponda según el momento de la sesión. Tu trabajo es revisar lo que el agente escribe (los archivos son legibles) y corregir cuando algo no encaja.

El orden de lectura al retomar es fijo. La sesión nueva lee primero **STATE.md** (el handoff), después **ROADMAP.md** (qué tareas hay), después **CONTEXT.md** cuando duda sobre el por qué de algo. **PROJECT.md** y **REQUIREMENTS.md** se asumen como contexto de fondo desde el inicio.

El glosario del final del submódulo recoge cada archivo con su rol, por si lo quieres tener como tarjeta de referencia rápida.

Bloque 8 · Cierre y enlace a M3.4

GSD organiza la continuidad alrededor de tres roles con responsabilidades distintas. Plan vivo (el qué), decisiones (el por qué), handoff (el qué necesita saber la próxima sesión). En el fork esos tres roles se materializan en cinco archivos markdown reales. Mezclarlos en uno solo destruye la ventaja del protocolo; separados, cada uno se actualiza en su frecuencia natural.

La actualización es parte del flujo, no tarea final. El plan vivo se mueve constantemente; el archivo de decisiones gana entradas cuando una elección de diseño se cierra; el handoff se reescribe al final de cada sesión. El agente gestiona el ritmo; la skill empuja para que no se olvide.

Los tres archivos son markdown legible. Versionables, editables a mano, revisables en code review. Eso quita la opacidad del estado del agente y la sustituye por artefactos auditables.

En el siguiente submódulo (M3.4) entramos en lo operativo: instalación con el comando exacto del repo `open-gsd/get-shit-done-redux`, verificación de que el protocolo está activo, y el gotcha más común de la primera vez (qué hacer cuando el `gsd/` no aparece automáticamente en tu proyecto).

Glosario rápido del bloque

- **Plan vivo** — el archivo que contiene pendientes, completados y en-curso. El que más cambia. Lo lee la sesión actual constantemente.
- **Archivo de decisiones** — registro de elecciones de diseño cerradas con contexto, alternativas y razón. Inmutable, acumulativo. El más valioso a medio plazo.
- **Handoff**: el documento que la sesión activa deja para la siguiente. Se reescribe entero al cierre; se lee primero al inicio. Lo más operativo de los tres.
- `PROJECT.md`, `REQUIREMENTS.md`, `ROADMAP.md`, `STATE.md`, `CONTEXT.md` : los cinco archivos reales del fork que materializan los tres roles. Viven en una carpeta dentro de tu proyecto y se versionan en git con el resto del código.
- **D-NNN**: referencia corta a una decisión del archivo de decisiones. Útil para enlazar desde mensajes de commit y comentarios.

Qué viene en M3.4

Instalación de GSD con el comando del fork con vida `open-gsd/get-shit-done-redux`, verificación operativa de que el protocolo está activo en tu Claude Code, y el gotcha más común de la primera vez. Cinco minutos para tener GSD operativo en tu setup.

M3.4 · Instalación y configuración de GSD desde el fork

Bloque 0 · Qué cubrimos en cinco minutos

Tres cosas. La instalación con el comando real verificado del fork `open-gsd/get-shit-done-redux` (el repo que justificamos con detalle en M3.2). Una verificación operativa de que el protocolo se activa en tu próxima sesión y no solo de que el comando devolvió OK. Y el gotcha más común de la primera vez, que es la carpeta de artefactos (con los cinco archivos del M3.3) no apareciendo donde esperabas.

Si vas a seguir el módulo en directo, abre tu Claude Code en un proyecto donde quieras probar la skill. GSD trabaja por proyecto; lo siguiente se ejecuta dentro de uno.

Bloque 1 · La instalación

GSD desde el fork se instala con un comando único de npm, verificado a fecha de hoy contra el README del repo:

```
npx @opengsd/get-shit-done-redux@latest
```

Eso es todo. Tres detalles del comportamiento del comando que merece tener delante mientras corre:

Resolución del paquete y dependencias. `npx` descarga el paquete `@opengsd/get-shit-done-redux` con el tag `@latest` (a fecha de hoy, v1.1.0 del 24 de mayo de 2026) y ejecuta su instalador. El instalador escribe los archivos de skill en la ubicación que tu Claude Code espera y registra los hooks de ciclo de vida que el protocolo necesita.

No es un `/plugin install`. GSD se distribuye como paquete npm, no como plugin del marketplace de Anthropic. Eso significa dos cosas operativas: una, no necesitas tener registrado ningún marketplace comunitario previamente; el comando se autoabastece. Dos, la actualización a versiones futuras se hace volviendo a ejecutar el mismo `npx ...@latest`, no con `/plugin update`.

Avisos durante la instalación. La primera vez que lo lances, el instalador puede pedirte confirmación sobre rutas (`~/claude/skills/` o equivalente) y, si el paquete antiguo del repo original (`get-shit-done-cc`) está instalado, te ofrecerá la migración automática. Acéptala; la

migración mueve tus artefactos existentes al paquete nuevo sin romper nada y desinstala el paquete viejo.

Si vienes del paquete antiguo, el instalador del fork detecta `get-shit-done-cc` y propone el cambio a `@opengsd/get-shit-done-redux` automáticamente. La migración mantiene tus archivos `PROJECT.md`, `REQUIREMENTS.md`, `ROADMAP.md`, `STATE.md` y `CONTEXT.md` intactos; solo cambia el paquete que los gestiona.

Si el comando falla por permisos de npm, lo más probable es que tu instalación local de Node tenga el directorio global protegido. Las dos opciones razonables: usar nvm para que npm escriba en tu carpeta de usuario, o ejecutar el comando con permisos elevados solo para esa instalación. Sin sudo crónico.

Bloque 2 · Cómo saber que el protocolo está activo

Que el comando termine con "installed" no garantiza que el protocolo se enganche al ciclo de vida de tu próximo prompt. La comprobación rápida:

Abre un proyecto en Claude Code donde quieras probar la skill. Pídele algo no trivial. Por ejemplo:

"Vamos a añadir un módulo nuevo de auditoría a la API. Empezamos por planificarlo."

Si GSD está activo, **el agente no arranca a planificar directamente**. Va a hacer dos cosas primero:

1. Mencionar que va a crear (o ya tiene) los cinco archivos del protocolo en la carpeta de artefactos del proyecto. Si ya existe un `STATE.md` previo, lo lee antes de proponer nada.
2. Proponer una entrada inicial para `ROADMAP.md` con la pieza que acabas de pedirle, y preguntar si quieres acotar requirements en `REQUIREMENTS.md` antes de empezar.

Esa es la señal. Si en cambio el agente arranca a escribir un plan a saco sin mencionar los artefactos, el protocolo no está enganchado al flujo. Causas posibles:

- **La tarea es demasiado pequeña para disparar el protocolo.** GSD no se activa para "renombrar esta función"; entra para piezas que justifican el ceremonial. Asegúrate de probar con algo que lo justifique.
- **La skill se instaló pero no se cargó en la sesión actual.** Reinicia Claude Code. La mayoría de skills npm requieren un ciclo nuevo de sesión para cargarse.

- **Conflicto con otra skill que también intercepta el inicio de tareas.** Si tienes Superpowers activo a la vez (caso normal), las dos se complementan sin chocar (M2 organiza el procedimiento dentro de la sesión, GSD organiza la continuidad entre sesiones). Si tienes otra skill experimental de continuidad de sesión, ahí puede haber fricción; desactívala temporalmente para confirmar.

Cuando el agente empieza mencionando los artefactos antes de planificar, el protocolo está corriendo y el M3.3 entero cobra sentido operativo.

Bloque 3 · Gotcha — La carpeta de artefactos no aparece donde esperabas

El gotcha más común de la primera vez. Instalas GSD, lanzas la primera pieza seria, el agente menciona que está creando los artefactos, y tú no los encuentras donde mirabas. Tres causas posibles, en orden de frecuencia:

Causa uno — el agente espera tu confirmación. Algunas versiones del fork piden confirmación antes de crear los archivos dentro del proyecto. Si te suelta un *"voy a crear gsd/PROJECT.md y compañía en este proyecto, ¿sigo?"* y no respondes, no los crea. Mira el último turno del agente; probablemente la pregunta esté ahí esperando.

Causa dos — el proyecto no es repo git. GSD asume control de versiones porque los cinco artefactos están diseñados para vivir bajo git. Si tu directorio actual no tiene `.git`, algunas versiones del fork piden que lo inicialices antes. `git init` y vuelves a lanzar el prompt.

Causa tres — la ubicación es la esperada pero no donde miras. Si tu Claude Code está corriendo en un directorio padre y has lanzado el prompt sobre un proyecto en subcarpeta, los archivos pueden haberse creado en el padre. Comprueba con `ls` desde el directorio actual y desde el padre. No es bug; es que la skill resuelve "este proyecto" a partir del directorio raíz del workspace de Claude Code.

Una vez los cinco archivos aparecen, la siguiente sesión los hereda sin ceremonia. El gotcha es de la primera vez; después es transparente.

Bloque 4 · Cierre y enlace a M3.5

Cinco minutos, GSD instalado desde el fork con vida con el comando real, protocolo verificado, gotcha de la primera vez resuelto. En el siguiente submódulo (M3.5) cerramos el módulo M3 con la demo de continuidad: pieza grande iniciada en una sesión, interrumpida a propósito a mitad, y retomada en otra sesión tres días después. Comparativa con el flujo de notas markdown casero

para que se vea con datos lo que en M3.1 dejamos como diagnóstico y en M3.2 contamos como historia.

Si has llegado hasta aquí con GSD instalado y los cinco archivos ya en tu proyecto de prueba, vas con la pieza lista para la demo del siguiente submódulo.

Glosario rápido del bloque

- `npx @opengsd/get-shit-done-redux@latest` — comando único de instalación del fork con vida, verificado a fecha de hoy. Auto-resuelve dependencias y configura los hooks de ciclo de vida.
- `get-shit-done-cc` — paquete npm del repo original (no recomendado por las razones del M3.2). Si lo tienes instalado, el instalador del fork ofrece migración automática; acéptala.
- **Migración automática** — el instalador del fork detecta el paquete antiguo y mueve los artefactos existentes al paquete nuevo sin perder estado. Solo cambia el paquete que los gestiona.
- **Carpeta de artefactos** — la carpeta dentro de tu proyecto donde viven `PROJECT.md`, `REQUIREMENTS.md`, `ROADMAP.md`, `STATE.md`, `CONTEXT.md`. Se versiona en git.
- **Activación del protocolo** — la señal de que GSD funciona en tu sesión actual. El agente menciona los artefactos antes de planificar o trabajar, no después.

Qué viene en M3.5

Demo de continuidad entre sesiones. Pieza grande iniciada con GSD activo, interrupción a propósito a mitad, retomada en sesión nueva tres días después. Comparativa con el flujo de notas markdown casero: cuánto más rápido retoma la sesión nueva, qué decisiones se pierden y cuáles sobreviven, y cómo se siente el flujo cuando los cinco archivos del protocolo están haciendo su trabajo. Siete minutos que cierran el módulo M3.

M3.5 · Demo — la misma pieza retomada con GSD y sin GSD

Bloque 0 · Lo que vamos a hacer

Pieza más grande que la del M2.5. No un endpoint suelto; un módulo completo. La planteamos en dos escenarios paralelos:

- **Escenario A:** notas caseras. Al final de la sesión uno, el agente actualiza un archivo `notas-sesion.md` con lo que ha hecho y lo que queda. Tres días después, sesión dos arranca leyendo ese archivo.
- **Escenario B:** GSD activo. Los tres artefactos (plan, decisiones, handoff) viven en `gsd/`. Tres días después, sesión dos arranca con el protocolo de retomada de la skill.

Mismo proyecto, mismo punto de partida, misma interrupción a la misma altura. Lo que cambia es solo el protocolo de paso entre sesiones.

Bloque 1 · La tarea

La pieza: **añadir un módulo de auditoría a la API**. Cada cambio de estado relevante de un dominio (Customer, Subscription, Invoice) tiene que registrar una entrada de auditoría con: actor, acción, entidad afectada, valor anterior, valor nuevo, timestamp del servidor.

Por qué es buen caso de demo:

- Es grande. No se hace en una sesión cómoda. Toca al menos cuatro carpetas del proyecto: modelos, capa de servicios, middleware HTTP para capturar el actor, y migraciones de base de datos.
- Tiene decisiones de diseño no triviales: ¿qué entidades entran al alcance ahora y cuáles vienen después? ¿Cómo serializamos el "valor anterior" en cambios complejos? ¿Auditoría síncrona o vía outbox?
- Esas decisiones son perdibles. Si la sesión dos no las hereda, las vuelve a tomar (probablemente distinto).

Sesión uno (90 minutos de trabajo):

- Decisiones cerradas: alcance inicial (Customer, Subscription), serialización JSON con propiedades cambiadas, modo síncrono dentro de la transacción.

- Implementado: modelo `AuditEntry`, migración, captura de actor en middleware, integración con Customer.
- Pendiente: integración con Subscription, tests de integración, documentación.

Interrupción a propósito al minuto 90, sin tiempo para "cerrar bien". El agente apenas tiene unos segundos para escribir lo que considere importante antes de que la sesión termine.

Sesión dos, tres días después. Misma pieza. El humano la retoma con un Claude Code limpio.

Bloque 2 · Escenario A — Sesión dos con notas caseras

Sesión uno cerró así: en el último turno el agente escribió un `notas-sesion.md` razonable. Contiene lista de archivos modificados, mención al modelo `AuditEntry` y a la migración, y una frase del estilo *"queda integrar con Subscription y escribir tests"*.

Sesión dos arranca:

"Estamos con el módulo de auditoría. Lee `notas-sesion.md` y sigue por donde quedó."

El agente lee el archivo, mira el código, y propone su lectura del estado. Tarda **9 minutos** en reconstruir el contexto. Lo siguiente que pasa, paso a paso:

Minuto 12. El agente empieza a integrar la auditoría con `Subscription`. Pero hace una elección distinta a la que se tomó en sesión uno: serializa el valor anterior completo (no solo las propiedades cambiadas). En `notas-sesion.md` no estaba esa decisión documentada de forma explícita; solo "JSON con cambios". Ambiguo. El agente elige razonable pero distinto.

Minuto 24. El agente añade una llamada asíncrona para escribir la auditoría, fuera de la transacción del negocio. En sesión uno habíais decidido lo contrario (síncrono dentro de transacción) porque así la auditoría falla atómicamente con el cambio si algo va mal. El `notas-sesion.md` no lo mencionaba; la decisión vivía en la cabeza del humano.

Minuto 40. Tests escritos. Pasan. La pieza se siente terminada.

Code review humano del día siguiente:

- *"La serialización ahora guarda el objeto entero, no solo las propiedades cambiadas. ¿Por qué cambiamos?"*. El humano responde lo que recuerda. Decisión que estaba cerrada vuelve a estar abierta.
- *"La auditoría es asíncrona. ¿No la queríamos atómica con la transacción?"*. Misma situación. Hay que volver, deshacer parte del trabajo, rehacer.

- "El alcance era Customer y Subscription. Veo que también has tocado Invoice. ¿Hace falta?". Y resulta que el agente, viendo Invoice cerca, "completó" sobre la marcha. Decisión nueva no acordada.

Tiempo total de la sesión dos: **40 minutos**. Tiempo de fix posterior por inconsistencias: **30 minutos**. Total real: **70 minutos**, con tres decisiones del primer día reabiertas y al menos una decisión nueva metida sin consultar.

Bloque 3 · Escenario B — Sesión dos con GSD

Reset. Sesión uno cerró igual de mal (interrupción al minuto 90), pero esta vez GSD estaba activo. Los tres artefactos quedaron actualizados al ritmo de la sesión:

- `ROADMAP.md` : pendiente inmediato = integración con Subscription. Lista clara de los siguientes pendientes (tests, docs).
- `CONTEXT.md` : tres entradas de decisión D-001 (alcance Customer + Subscription, Invoice fuera), D-002 (serialización JSON solo propiedades cambiadas), D-003 (modo síncrono dentro de transacción).
- `STATE.md` : estado en una frase, pendiente inmediato, y una trampa que el humano dictó en sesión uno: "cuidado con Invoice, está en el alcance fase dos pero no ahora; el campo `LastModifiedBy` ya existe pero no se está capturando porque el middleware se monta en una capa que Invoice no atraviesa".

Sesión dos arranca:

"Estamos con el módulo de auditoría."

El agente, con GSD activo, ejecuta el protocolo de retomada: lee el handoff primero, luego el plan, luego las decisiones. Confirma con el humano: "Voy a integrar con Subscription respetando D-002 (JSON solo propiedades cambiadas) y D-003 (síncrono en transacción). El handoff me avisa de no tocar Invoice. ¿Confirmas?".

Tres minutos de retomada en lugar de nueve.

Minuto 4. El agente empieza la integración con Subscription. Aplica D-002 y D-003 sin replantearlas.

Minuto 18. Surge una decisión nueva no contemplada: la captura de actor para acciones de servicio (no de usuario humano), por ejemplo el cron que renueva suscripciones. ¿Qué actor poner? El agente entra en Brainstorming (Superpowers activo a la vez), propone alternativas, espera tu respuesta. Eliges una constante reservada `system`. **D-004 añadida.**

Minuto 32. Tests escritos, todos pasando, pieza integrada. El plan vivo se actualiza: pendientes = solo documentación. El handoff se reescribe para la próxima sesión.

Code review humano del día siguiente: cero rotos. Las tres decisiones de sesión uno aparecen aplicadas; la cuarta está documentada con su contexto.

Tiempo total de la sesión dos: **32 minutos**. Tiempo de fix posterior: **cero**.

Bloque 4 · La comparativa lado a lado

Dimensión	Notas caseras	GSD
Tiempo de retomada al inicio	9 minutos	3 minutos
Decisiones de sesión uno aplicadas en sesión dos	1 de 3	3 de 3
Decisiones nuevas tomadas sin documentar	2	0
Tiempo de la sesión dos hasta "terminada"	40 minutos	32 minutos
Tiempo de fix posterior por inconsistencias	30 minutos	0 minutos
Tiempo total real	70 minutos	32 minutos

Cuatro datos para llevarte:

El ahorro grande no está en la sesión dos. El ahorro grande está en lo que NO pasa después: la reunión donde se descubre la inconsistencia, los 30 minutos de fix, la conversación sobre quién decidió qué. Esa es la diferencia real, y solo se ve cuando integras code review como dimensión.

La retomada inicial es secundaria, pero también gana. Tres minutos contra nueve no parece mucho a primera vista; multiplicado por la frecuencia con que retomas piezas grandes en un proyecto activo, son horas a la semana.

Las decisiones nuevas que surgen en sesión dos también ganan. Con GSD, las que aparecen se documentan en el momento. Sin GSD, viven en la cabeza del humano hasta que se olvidan.

El humano siente la diferencia tonalmente, no solo cuantitativamente. En el escenario A, el code review del día siguiente es estresante: cosas se han movido sin tu consentimiento explícito. En el B, el code review es de ajustes finos, no de "qué ha pasado aquí". Esa parte no aparece en la tabla pero afecta cómo te relacionas con el agente a largo plazo.

Bloque 5 · Cierre del módulo M3 y enlace al M4

Hemos cerrado el módulo M3 con la evidencia que justifica los cuarenta y cinco minutos invertidos.

El problema número dos de Claude Code es de protocolo, no de capacidad. La sesión se acaba; el agente nuevo no sabe lo que el anterior decidió. Los apañes caseros (notas, memoria) funcionan a medias porque el agente no sabe qué guardar con criterio y la sesión nueva no recupera con criterio.

GSD instala ese protocolo con tres artefactos versionables: plan vivo (qué se está haciendo), decisiones (por qué), handoff (qué necesita saber la próxima sesión). Cada uno cubre una función específica y se actualiza al ritmo natural de la pieza. La opción recomendada por este curso es el fork `open-gsd/get-shit-done-redux` por las razones operativas que vimos en M3.2.

Superpowers y GSD se complementan. Superpowers cubre el procedimiento dentro de la sesión; GSD cubre la continuidad entre sesiones. En el M7 vamos a ver el flujo combinado completo. Mientras tanto, en el M4 entramos en la pieza siguiente: revisión de código asistida con `/review` y su versión extendida `/ultrareview`. Una vez tienes procedimiento (Superpowers) y continuidad (GSD), la siguiente palanca de calidad es la revisión sistemática del código que el flujo produce.

Glosario rápido del bloque

- **Sesión uno / sesión dos** — la unidad de la demo. Sesión uno hace el trabajo, sesión dos retoma tras una interrupción real.
- **Decisión reabierta** — decisión cerrada en sesión uno que la sesión dos, sin protocolo de continuidad, vuelve a tomar y a veces de forma distinta. La causa principal de inconsistencias en piezas grandes.
- **Decisión nueva no documentada** — elección que la sesión dos hace sin que el humano lo registre. Si el agente no la captura en un artefacto, se pierde.
- **Tiempo total real** — métrica honesta para comparar protocolos de continuidad. Incluye la sesión, el code review y el fix posterior.
- **Code review tonal** — la diferencia cualitativa del code review cuando no hay sorpresas. No aparece en tablas pero afecta cómo el equipo trabaja con el agente.

Qué viene en M4

Revisión de código asistida en Claude Code. Empezamos con el problema (la revisión humana no escala cuando el agente produce más volumen) y vamos por los dos comandos integrados: `/review` para revisión estándar de la rama actual y `/ultrareview` para revisión profunda con varios agentes paralelos. Cuatro submódulos: problema, `/review`, `/ultrareview`, demo comparativa.

PARTE 4

/review y /ultrareview · revisión asistida

M4.1 · Por qué la revisión humana no escala con Claude Code

Bloque 0 · La escena del cuello de botella

Llevas dos semanas con Superpowers y GSD activos. Tu velocidad de producción se ha duplicado. Donde antes cerrabas una pieza al día, ahora cierras dos largas o tres medianas. El procedimiento funciona, la continuidad entre sesiones funciona, las cosas se entregan.

Y entonces miras tu pila de pull requests pendientes de review y descubres el problema número tres. La pila crece más rápido de lo que tú o tu equipo pueden vaciarla. Cada PR del agente sigue siendo un PR que alguien tiene que revisar a mano, leer línea a línea, contrastar con la decisión de diseño, comprobar tests. Y ese alguien (tú, un colega, un revisor designado) tiene el mismo número de horas al día que antes de que el agente le doblara la productividad.

Si esto te suena ya en tu equipo, este módulo va dirigido a ti. Si todavía no te ha pasado pero estás escalando el uso de Claude Code, te va a pasar en cuestión de semanas.

Bloque 1 · El diagnóstico — el problema número tres

En M2 vimos la prisa (problema uno). En M3 vimos la sesión que se acaba (problema dos). Aquí tenemos el tercero: **la revisión humana es lineal en horas; la producción del agente no.**

Dos detalles que hacen que este problema sea distinto de los dos anteriores:

No es un problema del agente; es un problema del proceso. El agente está haciendo su trabajo bien (asumiendo M2 y M3 aplicados). El cuello aparece en el siguiente eslabón del flujo: el code review. Es la revisión la que no escala, no la producción.

El daño es asimétrico. La revisión que se hace deprisa pasa cosas que la revisión cuidadosa habría cogido. Y como el agente produce más, la revisión termina haciéndose más deprisa por presión de pila. Resultado: la calidad efectiva del código que llega a producción puede bajar precisamente cuando la calidad del código generado ha subido. Paradoja molesta.

Tres síntomas concretos que verás pronto:

La pila de PRs pendientes crece semana a semana. La gente que antes revisaba en una hora ahora necesita dos. Cuando el lunes el revisor llega con quince PRs en cola, el comportamiento por defecto es priorizar y dejar tres para "cuando haya tiempo". Esos tres acaban mergeándose con revisión superficial.

Los comentarios de code review se vuelven más cortos. No porque las piezas necesiten menos comentarios, sino porque el revisor no tiene tiempo para profundizar. "LGTM" y "ok" sustituyen al comentario sustantivo. La conversación de review se atrofia.

Los bugs encontrados en producción pasan a tener un tipo nuevo. No son los bugs típicos del código humano (lógica retorcida, off-by-one en bucles). Son los bugs típicos del código del agente que la revisión rápida no caza: casos límite no cubiertos, decisiones de diseño tomadas sobre la marcha sin documentar, integración sutil con partes del sistema que el agente no conocía bien.

Bloque 2 · Por qué "revisar más rápido" no es la solución

La primera reacción razonable es entrenar al equipo para revisar más rápido. Plantillas de checklist, herramientas de PR automatizadas, división de PRs en piezas más pequeñas. Todo eso ayuda al margen y no resuelve el problema central.

El problema central es que la revisión que hace caza bugs reales en código del agente **no es una revisión más rápida; es una revisión distinta**. Las preguntas que tienes que hacerte cuando revisas código del agente son distintas de las que te haces cuando revisas código humano:

- *"¿Esta decisión de diseño está documentada o se tomó sobre la marcha?"*. No es pregunta típica de review humano.
- *"¿El test cubre el caso límite que el agente probablemente no consideró?"*. Tampoco.
- *"¿La integración con el módulo X respeta las convenciones del módulo X o el agente se inventó las suyas?"*. Tampoco.

Para que un humano haga esa revisión distinta a mano hace falta cargar contexto que no era necesario para revisar código humano. Tiempo en cargar, tiempo en mantener cargado, tiempo en aplicar. La revisión profunda de código del agente cuesta más, no menos, que la revisión profunda de código humano.

La salida no es "más humano más rápido". Es **revisión asistida**: que el agente, o varios agentes en paralelo, hagan la primera pasada de revisión con el contexto cargado (las convenciones del proyecto, las decisiones documentadas, los patrones del módulo afectado), produzcan una lista priorizada de hallazgos, y dejen al humano la decisión final sobre lo que merece atención y lo que es ruido.

Eso es lo que `/review` y `/ultrareview` cubren.

Bloque 3 · Lo que viene en el módulo

El módulo M4 son cuatro submódulos:

- M4.1 (este): el problema que la revisión asistida aborda y por qué la respuesta no es revisar más rápido.
- M4.2: `/review`, el comando integrado de revisión estándar. Qué hace, cuándo se ejecuta, qué tipo de hallazgos devuelve.
- M4.3: `/ultrareview`, la versión extendida que lanza varios agentes en paralelo con perspectivas distintas. Cuándo entra y cuándo es desproporcionada.
- M4.4: demo comparativa. Un PR real, revisión humana sola, revisión con `/review`, y revisión con `/ultrareview`. Tabla de hallazgos cazados por cada método.

Veinticinco minutos en total. Es un módulo medio, no tan extenso como M2 o M3, pero igual de necesario en el flujo. Sin revisión asistida, la velocidad que ganaste con Superpowers y GSD se devuelve por el cuello del code review.

Aviso operativo importante: `/review` y `/ultrareview` son comandos integrados de Claude Code que NO necesitas instalar. Vienen con la herramienta. La parte interesante del módulo no es la instalación; es saber cuándo lanzar cuál, cómo leer los hallazgos sin perder tiempo, y cómo distinguir señal de ruido en los resultados.

Bloque 4 · Cierre y enlace a M4.2

El problema número tres de Claude Code no es del agente; es del proceso que rodea al agente. La revisión humana no escala porque mide en horas y la producción del agente no. La paradoja molesta es que la calidad efectiva del código en producción puede bajar precisamente cuando la calidad del código generado ha subido.

Revisar más rápido no es la solución porque la revisión que caza bugs reales en código del agente es revisión distinta, no menos profunda. Cuesta más cargar el contexto que el agente cargaba implícitamente.

La salida es revisión asistida: el agente hace la primera pasada con contexto cargado, el humano decide sobre los hallazgos. Los dos comandos integrados que cubren esto son `/review` (estándar) y `/ultrareview` (extendido). En el siguiente submódulo (M4.2) entramos en `/review`: qué hace, qué tipo de hallazgos te devuelve, y cuándo es la herramienta correcta.

Glosario rápido del bloque

- **Problema número tres** — el cuello de botella que aparece cuando la producción del agente supera la capacidad de revisión humana del flujo.
- **Revisión asistida** — pasada de revisión hecha por un agente con contexto cargado (convenciones, decisiones, patrones), que produce hallazgos priorizados para que el humano decida.
- **Comentario sustantivo** — la respuesta de code review que no es "LGTM" ni "ok". La métrica cualitativa que se atrofia cuando el revisor no tiene tiempo.
- **Calidad efectiva** — la del código que finalmente llega a producción tras pasar por revisión, distinta de la calidad del código generado. Es la que importa al usuario final.

Qué viene en M4.2

`/review`, el comando integrado de revisión estándar de Claude Code. Qué hace exactamente (revisa la rama actual contra `main`), qué hallazgos devuelve, cómo leerlos, y cuándo es la herramienta correcta. Siete minutos para tener `/review` como reflejo antes de pedir un code review humano.

M4.2 · `/review` — el comando estándar

Bloque 0 · La promesa del submódulo

`/review` es comando integrado en Claude Code. No se instala, no se configura, no necesitas autenticación adicional. Lo escribes en tu sesión y arranca. Esa simplicidad esconde la primera trampa: es tan fácil de lanzar que mucha gente lo ignora pensando que es decorativo. No lo es. Es el primer paso de la revisión asistida del M4.1 y el que más relación coste-beneficio tiene.

En este submódulo vamos a ver qué hace exactamente (más concreto que "revisa código"), qué tipo de hallazgos devuelve, cómo leer la salida sin perder tiempo, y cuándo `/review` no es suficiente y toca pasar a `/ultrareview` (M4.3).

Bloque 1 · Qué hace exactamente

`/review` toma como objeto de revisión **los cambios de tu rama actual respecto a `main`** (o respecto a la rama base que tu repo tenga configurada como tronco). No revisa toda la base de código; revisa el delta.

Tres detalles del comportamiento que merece tener delante:

Lee los archivos modificados y sus alrededores. No solo el diff. Si has tocado el método `CancelAsync` en `SubscriptionService.cs`, `/review` lee el resto del archivo para entender cómo encaja, y lee los archivos que llaman a ese método para detectar si el cambio rompe asunciones río arriba. Esa carga de contexto es la que la revisión humana acelerada no se permite.

Aplica las convenciones del proyecto que pueda inferir. Si tu proyecto tiene un `.editorconfig`, una colección de pruebas con un patrón visible, o un `CLAUDE.md` con reglas, las considera. Eso significa que dos proyectos distintos pueden recibir feedback distinto sobre los mismos cambios; lo que en uno es normal, en otro puede ser desviación.

Devuelve una lista priorizada de hallazgos, no un veredicto binario. No es "esto está bien / esto está mal"; es "aquí hay un posible bug crítico, aquí dos cuestiones de diseño con prioridad media, aquí cuatro cosas menores que tú decides si miras". Eso te permite atacar lo grave primero y descartar lo trivial con un vistazo.

El comando tarda entre uno y tres minutos según el tamaño de la rama. Para piezas pequeñas (cambios en uno o dos archivos), termina en menos de un minuto. Para ramas grandes con quince o veinte archivos modificados, dos o tres.

Bloque 2 · Qué hallazgos devuelve

Hay cuatro categorías típicas de hallazgo que `/review` produce. Conocerlas de antemano ayuda a leer la salida más rápido.

Bugs sospechados. La categoría con prioridad más alta. Casos como un null no comprobado donde la signatura sugiere que puede serlo, un caso límite no cubierto por test, una asunción de orden en una colección que el llamante no garantiza, un cierre de recurso que falta en el camino de error. Lo que un revisor humano cuidadoso encontraría leyendo despacio.

Cuestiones de diseño. Prioridad media. La función podría dividirse para que un test sea posible, el acoplamiento con `OrderService` es más fuerte que el del resto del módulo, la responsabilidad mezcla validación con persistencia. Son hallazgos opinables; algunos los aplicarás, otros no. La diferencia con los bugs es que aquí el código funciona; lo que está en cuestión es cómo está estructurado.

Inconsistencias con el proyecto. Cosas que en otro proyecto serían bien y en el tuyo no encajan. Por ejemplo: el resto del módulo usa `Result<T>` para errores esperados y tu cambio lanza excepciones; el patrón de logging del proyecto incluye un `correlationId` que tu llamada nueva no añade. La utilidad de esta categoría depende mucho de lo limpio que sea tu proyecto.

Mejoras menores. Prioridad baja. Nombres de variable más claros, documentación inline donde el código se beneficiaría, `var` vs tipo explícito. Cosas que en code review humano alguien comentaría con `nit:` delante. Mirables si te sobra tiempo; ignorables si no.

La salida típica empieza con un resumen de cuántos hallazgos hay por categoría, sigue con los críticos uno a uno, y termina con los menores agrupados. Si tu sesión es lo bastante larga, te mantiene los hallazgos referenciables hasta el final.

Bloque 3 · Cómo leer la salida sin perder tiempo

Tres reglas operativas que funcionan en la mayoría de proyectos:

Empieza por la categoría "bug sospechado". Es el único nivel donde merece la pena leer el hallazgo entero antes de decidir si actúas. Los demás los puedes hojear; los bugs no. Si hay tres bugs sospechados y los tres son ciertos, has justificado el comando entero antes de mirar el resto.

Para cuestiones de diseño, contrasta con tu archivo de decisiones de GSD si lo tienes. Si el hallazgo dice "deberías dividir esta función" pero tu D-007 documenta que la decisión consciente fue mantenerla junta por razones X, descartas con confianza. Sin el archivo

de decisiones, cada cuestión de diseño te obliga a recordar si era decisión consciente o despiste, y eso cansa.

Inconsistencias con el proyecto: aplica las del bloque "alto valor / bajo coste". Aplica las inconsistencias que son baratas de arreglar y mejoran la coherencia (añadir el `correlationId` que falta, cambiar el patrón de error). Descarta o pospón las que el agente sugiere pero costarían una refactorización mayor. El tiempo extra de la revisión asistida se gana arreglando las baratas; se pierde aceptando todas.

Mejoras menores: lee solo si vas con el tiempo holgado. Si vas justo, las ignoras. No hay penalización por no aplicarlas; son nits literales.

Una técnica que ayuda en piezas grandes: pide al final que te haga un **resumen ejecutivo de tres líneas**. *"Resume los hallazgos en tres líneas: los críticos y la conclusión general de si la rama está lista para PR humano"*. Te da el bottom line antes de hojear los detalles.

Bloque 4 · Cuando `/review` es suficiente y cuándo no

`/review` es la herramienta correcta cuando:

- **La rama es del tamaño típico de tu flujo:** cinco a veinte archivos modificados, una pieza coherente con un propósito.
- **El equipo o tú vais a hacer revisión humana después.** `/review` no la sustituye; la prepara. Saca lo grueso para que la humana se centre en lo sutil.
- **La pieza no tiene riesgo sistémico alto.** Cambios en lógica de negocio, en endpoints, en migraciones rutinarias. La mayoría de los PRs entran aquí.

`/review` empieza a quedarse corto cuando:

- **La rama toca código sensible** (autenticación, autorización, criptografía, manejo de pagos, migraciones destructivas). Una sola pasada de un agente puede pasar por alto algo que otra pasada con perspectiva distinta hubiera cogido. Aquí entra `/ultrareview` (M4.3).
- **El cambio es muy grande** (más de treinta archivos, refactorizaciones transversales). La pasada única no carga suficiente contexto del proyecto en un solo paso. Varios agentes con perspectivas distintas reparten el coste de contexto.
- **El equipo no va a hacer revisión humana después.** Si la rama va directamente a `main` sin code review humano posterior, una sola pasada es asumir más riesgo del que normalmente asumes. Sube el listón a `/ultrareview`.

Un caso especial que llega a confundir: las ramas pequeñísimas (un único archivo, dos líneas cambiadas). Para esas, `/review` puede sentirse desproporcionado. La realidad es que tarda lo

mismo en arrancar y el coste marginal es bajo; lánzalo igual y cierra el ciclo. La fricción de decidir si lanzarlo o no es más cara que la espera de un minuto.

Bloque 5 · Cierre y enlace a M4.3

`/review` es comando integrado, no skill instalable. La trampa es que la facilidad lo hace ignorable; el valor está en convertirlo en reflejo del cierre de cada rama. Lánzalo antes de pedir code review humano, no después.

Las cuatro categorías de hallazgo (bugs sospechados, diseño, inconsistencias, menores) tienen relación coste-beneficio distinta. Los bugs justifican el comando solos; lo demás depende de tiempo y contexto. Combínalo con el archivo de decisiones de GSD si lo tienes; el ahorro de "esto era decisión consciente, no despiste" es enorme.

`/review` es suficiente para la mayoría del flujo y se queda corto en código sensible, ramas muy grandes, o cuando no hay revisión humana posterior. Para esos casos, en el siguiente submódulo (M4.3) entramos en `/ultrareview`, la versión que lanza varios agentes en paralelo con perspectivas distintas y consolida los hallazgos.

Glosario rápido del bloque

- **Delta de rama** — el conjunto de cambios entre tu rama actual y `main` (o el tronco configurado). El objeto de revisión de `/review`.
- **Hallazgo priorizado** — feedback con categoría y nivel de prioridad asignados por el agente. La forma de entrega que permite atacar lo grave primero.
- **Resumen ejecutivo** — petición opcional al final del `/review` para condensar la salida en pocas líneas. Útil en piezas grandes.
- **Reflejo de cierre de rama** — la práctica de lanzar `/review` automáticamente al terminar una pieza, antes de abrir PR. El cambio de hábito que más rinde de este submódulo.

Qué viene en M4.3

`/ultrareview`, la versión extendida. Lanza varios agentes en paralelo con perspectivas distintas, consolida hallazgos, devuelve algo más cercano a una revisión de tres personas en una sola pasada. Cuándo entra, cuándo es desproporcionada, y cómo leer la salida cuando viene más cargada que la de `/review`. Siete minutos.

M4.3 · /ultrareview — la versión profunda

Bloque 0 · La promesa del submódulo

Si `/review` es la revisión asistida estándar (M4.2), `/ultrareview` es la versión pesada. Mismo objetivo, profundidad muy distinta. Lo que vamos a fijar en este submódulo es lo siguiente: cómo opera por dentro (a alto nivel), qué tres tipos de hallazgos extras te entrega que `/review` no, cuándo merece la pena el coste, cuándo no, y cómo leer la salida cargada sin perder horas en lo que no importa.

Aviso de versión: `/ultrareview` se introdujo con Claude Opus 4.7 en mayo de 2026. Si tu Claude Code es anterior, el comando puede no estar disponible o estar en una forma distinta. Comprueba la versión de tu CLI antes de seguir.

Bloque 1 · Qué hace distinto de /review

Tres diferencias operativas importantes.

Lanza varios agentes en paralelo, no uno solo. Mientras `/review` corre una pasada secuencial, `/ultrareview` arranca varios revisores simultáneos, cada uno con una perspectiva configurada distinta. Una perspectiva puede ser *correctness* (caza bugs), otra *security* (caza vectores de ataque), otra *performance* (caza cuellos), otra *consistencia con el proyecto* (caza desviaciones del patrón del módulo). El detalle exacto del número y los nombres puede afinarse versión a versión; el patrón estable es que son varios y son distintos.

Consolida los hallazgos al final. No te suelta cuatro listas separadas; combina, deduplica, y devuelve una sola lista priorizada. Cuando dos revisores cogen el mismo bug por caminos distintos, lo marca como "señal alta" (más de un revisor lo vio). Cuando uno solo lo coge, queda como hallazgo individual a revisar con criterio. Esa dimensión de consenso entre revisores es información que `/review` no puede darte.

Tarda más. Donde `/review` termina en uno o tres minutos, `/ultrareview` puede tardar entre cinco y quince. La diferencia depende del tamaño de la rama y del número de revisores activos. No es una herramienta para lanzar mientras esperas; es una para lanzar y volver a mirar después.

Lo que NO cambia respecto a `/review`:

- Sigue revisando el delta de tu rama contra `main`. No es revisión de toda la base.
- Sigue siendo comando integrado de Claude Code. No se instala.

- Sigue devolviendo hallazgos categorizados y priorizados, no veredicto binario.

Bloque 2 · Tres tipos de hallazgo que /ultrareview saca y /review no

Tres tipos de hallazgo cualitativamente distintos que aparecen en `/ultrareview` y casi nunca en `/review`:

Inconsistencias entre revisores como señal. Si dos revisores aprueban la pieza y un tercero la marca como problemática, esa discrepancia es información en sí misma. `/ultrareview` te la muestra: *"Dos revisores aprueban; uno (perspective: security) detecta posible inyección si el campo `customerName` no se sanea en `OrderReportService.cs`. Examinar."*. La discrepancia te invita a mirar lo que un revisor solo no habría llamado.

Vectores de ataque sutiles. El revisor de perspectiva *security* suele cazar cosas que la pasada de propósito general ignora: validación insuficiente en el lado de servidor, payloads que pueden venir hostiles, queries SQL con concatenación incluso cuando hay ORM (porque alguien usó `FromSqlRaw`), exposición indirecta de datos sensibles en respuestas. No es que `/review` no los detecte nunca; es que con la perspectiva específica activada, la tasa de detección sube mucho.

Conflictos de patrón a nivel arquitectura. El revisor de perspectiva *consistencia con el proyecto* compara el código nuevo con el resto del módulo a una profundidad que la pasada de propósito general no alcanza. *"El módulo `Pricing` resuelve impuestos vía `ITaxResolver` inyectado; el nuevo método `CalculateFinal` los calcula inline. Inconsistente."*. Hallazgos así requieren cargar mucho contexto del proyecto, algo que solo un revisor dedicado a esa perspectiva puede hacer con calidad.

Bloque 3 · Cuándo entra y cuándo es desproporcionada

Lánzala cuando se cumple al menos una de estas:

El código toca zonas sensibles. Autenticación, autorización, criptografía, manejo de pagos, manejo de datos personales bajo regulación, migraciones de base de datos con potencial de pérdida de datos. En estas zonas, la diferencia entre una pasada y varias paralelas es la diferencia entre tranquilidad y riesgo asumido.

La rama es grande. Treinta archivos modificados o más, refactorización transversal que toca varios módulos, integración que conecta dos partes del sistema que antes vivían separadas. Aquí varias perspectivas reparten el coste de contexto y cubren más a la vez.

No vas a tener revisión humana después. Si la pieza va a `main` sin code review humano del equipo, sube el listón. Lo asumes con `/ultrareview`; con `/review` solo no.

El equipo no conoce esta parte del código. Si nadie del equipo es propietario del módulo afectado, una pasada profunda asistida es la red de seguridad. Un humano que no conoce el módulo va a hacer code review superficial por defecto; varias perspectivas asistidas reparten el conocimiento implícito que el humano no tiene.

Es desproporcionada cuando:

La rama es pequeña: dos o tres archivos, cambios cosméticos, fix de un bug puntual. `/ultrareview` tarda diez minutos en algo que `/review` resolvería en uno. Y los revisores adicionales no encuentran nada porque no hay materia para encontrar.

Ya tienes code review humano cuidadoso planificado: en proyectos donde la cultura de code review humano es fuerte y hay tiempo asignado para hacerlo bien, `/ultrareview` puede solapar con la revisión humana sin añadir mucho. `/review` solo + humano cuidadoso suele bastar.

El equipo está saturado de notificaciones: si cada PR ya recibe dos revisores humanos + checks de CI + linters + scanners, añadir `/ultrareview` puede generar ruido que el equipo termina ignorando. La revisión que se ignora es peor que la que no se hizo.

Bloque 4 · Cómo leer la salida cuando viene cargada

`/ultrareview` puede devolver veinte, treinta, cuarenta hallazgos en piezas grandes. Si los lees todos en orden de aparición, pierdes media hora. Tres reglas operativas:

Lee primero los hallazgos marcados como consenso. Son los que más de un revisor cogió. Son los menos discutibles y los más probablemente reales. Si hay tres de consenso y los tres son ciertos, ya tienes el grueso del valor.

Filtra por perspectiva según la naturaleza de la rama. Si la rama toca autenticación, lee con prioridad los hallazgos de la perspectiva *security* aunque no estén en consenso. Si la rama es de optimización, prioriza los de *performance*. La heurística: la perspectiva que coincide con el tema del cambio gana relevancia.

Pide consolidación ejecutiva. Igual que con `/review`, al terminar la lectura pide: *"Dame los tres hallazgos que más me debería preocupar y por qué"*. La salida consolidada y reordenada por gravedad real es mucho más manejable que la salida bruta.

Una técnica adicional para piezas muy grandes: usa `/ultrareview` en pasadas iterativas. Primera pasada, atacas los críticos y el consenso. Segunda pasada (después de los fixes), el comando re-revisa solo lo que has tocado y devuelve lo que queda. Esa dinámica de "revisar, fixear, re-revisar" reduce la lista a algo manejable sin perder cobertura.

Bloque 5 · Cierre y enlace a M4.4

`/ultrareview` no es un `/review` extendido sin más. Es un patrón distinto: varios revisores paralelos con perspectivas configuradas, consolidación al final, dimensión de consenso entre revisores como señal nueva. Tarda más; aporta categorías de hallazgo que `/review` no podía darte.

Las zonas donde gana son específicas: código sensible, ramas grandes, ausencia de code review humano posterior, módulos que el equipo no conoce. Las zonas donde pierde también son específicas: piezas pequeñas, equipos saturados de notificaciones, code review humano ya cuidadoso.

Leer la salida cargada con criterio es lo que separa que `/ultrareview` aporte horas o que cueste horas. Empezar por consenso, filtrar por perspectiva relevante al cambio, y pedir consolidación ejecutiva son las tres reglas operativas que rinden más.

En el siguiente submódulo (M4.4) cerramos el módulo M4 con la demo comparativa que prometimos: un PR real, revisado por un humano solo, por `/review`, y por `/ultrareview`. Tabla de hallazgos cazados por cada método, tiempo invertido por cada uno, y qué bugs reales pasaron por cada camino y cuáles se perdieron.

Glosario rápido del bloque

- **Perspectiva configurada** — el ángulo con el que cada revisor paralelo opera (correctness, security, performance, consistencia). La diversidad de perspectivas es lo que distingue `/ultrareview` de `/review`.
- **Consenso entre revisores** — hallazgo que más de un revisor cogió. Señal de alta confianza; el primero a leer.
- **Discrepancia como señal** — el caso opuesto: la mayoría aprueba, uno detecta. La discrepancia te invita a mirar a lo que solo una perspectiva caza.
- **Pasada iterativa** — patrón de uso para ramas grandes: lanzar, arreglar críticos, re-lanzar para que el comando revise lo tocado. Útil para no leer treinta hallazgos de golpe.

Qué viene en M4.4

Demo comparativa. Un PR real con tres bugs sospechados, dos cuestiones de diseño, y una inconsistencia con el proyecto. Tres métodos: revisión humana sola, `/review`, `/ultrareview`. Tabla de hallazgos cazados por cada uno y tiempo invertido. Seis minutos para cerrar el módulo M4 con la evidencia.

M4.4 · Demo — un PR real revisado por humano, /review y /ultrareview

Bloque 0 · El experimento

Tomamos un PR de tamaño medio en el proyecto .NET que vamos arrastrando desde M2.5. Pieza: añadir un endpoint nuevo de generación de informes de facturación. Catorce archivos modificados entre controlador, servicios, query a base de datos, mappers, y tests.

En este PR sembramos a propósito seis problemas distintos, cada uno de un tipo:

1. **Bug crítico:** `NullReferenceException` posible si el cliente no tiene dirección fiscal completa.
2. **Bug sutil:** el endpoint devuelve datos paginados pero el contador total ignora el filtro aplicado (cuenta de toda la tabla, no de los filtrados).
3. **Vulnerabilidad de seguridad:** el filtro de fecha llega como string al método `FromSqlInterpolated` mal usado, generando una vía de inyección SQL.
4. **Inconsistencia con proyecto:** el servicio usa excepciones para errores esperados; el resto del módulo usa `Result<T>`.
5. **Cuestión de diseño media:** el método tiene noventa líneas y combina captura de datos, transformación y formateo.
6. **Nit menor:** una variable se llama `temp1` en un loop.

Tres métodos de revisión, mismo PR, partiendo de cero:

- **Método A:** humano cuidadoso, sin asistencia. Cronometrado.
 - **Método B:** `/review`. Comando, leer la salida, decidir.
 - **Método C:** `/ultrareview`. Comando, esperar, leer consolidado.
-

Bloque 1 · Método A — Humano cuidadoso, sin asistencia

Revisor: un dev senior con experiencia en el módulo de facturación, pero sin acceso al archivo de decisiones de la sesión donde se escribió el código.

Procedimiento: abre el PR, lee el resumen, lee los archivos uno a uno, deja comentarios.

Tiempo invertido: **42 minutos**. (Tiempo cómodo para una pieza de este tamaño; un revisor con prisa lo haría en 15 y cazaría menos.)

Hallazgos:

-  El **NullPointerException** (bug 1): cazado. El revisor vio el acceso a `customer.Address.PostalCode` sin null-check.
-  El **contador total ignora filtro** (bug 2): no cazado. El test que el agente escribió pasaba sin verificar el contador, y el revisor leyó el código asumiendo que el contador era correcto.
-  La **inyección SQL** (vulnerabilidad 3): no cazada. El revisor no es especialista en seguridad y `FromSqlInterpolated` parece seguro a primera vista.
-  La **inconsistencia con `Result<T>`** (4): cazada. El revisor conoce el patrón del módulo.
-  El **método de noventa líneas** (5): cazado. Comentario sugerido: "considera dividir en tres".
-  El **`temp1` en el loop** (6): no cazado. Probablemente lo vio y lo dejó pasar; los nits no son lo que el ojo entrena en piezas medianas.

Cobertura: 3 de 6. Tiempo: 42 minutos.

Bloque 2 · Método B — /review

Lanza el comando. Espera dos minutos. Lee la salida.

Hallazgos devueltos:

-  Bug 1 (NullPointerException): cazado, marcado como crítico.
-  Bug 2 (contador total): cazado. Esta es la diferencia clave con el método humano. El comando leyó el código de la query, leyó el código del contador, comparó ambos, y detectó que el filtro se aplicaba en uno y no en el otro. Justo lo que un humano cansado pasa por alto.
-  Vulnerabilidad 3 (inyección SQL): mencionada como "posible problema de seguridad", prioridad media. La detectó pero no le dio la prioridad máxima.
-  Inconsistencia 4 (Result vs excepción): cazada, prioridad media.
-  Cuestión 5 (método largo): cazada, prioridad baja.
-  Nit 6 (`temp1`): cazado, prioridad muy baja. Mencionado al final con otros nits.

Cobertura: 6 de 6. Tiempo: 2 minutos de espera + 4 de lectura = 6 minutos total.

Lo importante de esta fila no es solo que cazó los seis. Es que el bug 2 (el contador total), que el humano cuidadoso no cazó, lo cogió por la vía de "leer ambos lados de la lógica y comparar", que es el tipo de revisión que la fatiga humana hace mal.

Bloque 3 · Método C — /ultrareview

Lanza el comando. Espera once minutos. Lee la salida consolidada.

Hallazgos devueltos:

-  Bug 1: cazado, **consenso de tres revisores**. Señal de alta confianza.
-  Bug 2: cazado, **consenso de dos revisores** (correctness + consistencia con proyecto, este último porque el resto de endpoints paginados sí cuentan con filtro).
-  Vulnerabilidad 3: cazada por el revisor de **perspectiva security**, marcada como **crítica**. Aquí está el cambio frente a `/review`: el revisor de perspectiva security le da la prioridad correcta, mientras que la pasada de propósito general la había dejado en media.
-  Inconsistencia 4: cazada, marcada como cuestión arquitectónica.
-  Cuestión 5: cazada por **dos revisores** con sugerencias de división diferentes (uno propone tres métodos, otro propone introducir una clase). Discrepancia útil; te invita a decidir cuál encaja mejor en el proyecto.
-  Nit 6: cazado.

Además, **un hallazgo extra que ningún otro método cazó**:

-  **Hallazgo nuevo de la perspectiva performance**: la query de informes carga el grafo completo de la entidad `Customer` cuando solo se usan dos campos. Si el cliente tiene muchos `Order` históricos, esto puede degradar bastante el endpoint en producción. No es bug funcional; es deuda de rendimiento que en producción real importa.

Cobertura: 6 de 6 + 1 extra. Tiempo: 11 minutos de espera + 8 de lectura = 19 minutos total.

Bloque 4 · La comparativa lado a lado

Dimensión	Humano cuidadoso	/review	/ultrareview
Bug crítico (NullRef)	✓	✓ crítico	✓ crítico, consenso 3
Bug sutil (contador)	✗ no cazado	✓ cazado	✓ cazado, consenso 2
Vulnerabilidad SQL	✗ no cazado	⚠ prioridad media	✓ prioridad crítica
Inconsistencia patrón	✓	✓ media	✓ arquitectónica
Método largo	✓ baja	✓ baja	✓ con discrepancia útil
Nit <code>temp1</code>	✗	⚠ muy baja	⚠ muy baja
Hallazgo extra (perf)	—	—	✓
Tiempo invertido	42 min	6 min	19 min
Cobertura	3 / 6	6 / 6	7 / 6

Cuatro datos para llevarte:

/review solo ya gana mucho terreno. Seis minutos contra cuarenta y dos del humano, con mejor cobertura. El argumento del M4.1 se sostiene con datos: la revisión humana no escala porque mide en horas y la asistida no.

/ultrareview gana en código sensible y deuda oculta. La diferencia clave no son los seis primeros hallazgos (los caza /review también), sino la prioridad correcta de la vulnerabilidad SQL y el hallazgo extra de perspectiva performance. En código que va a manejar dinero, datos personales, o pagos, esa diferencia justifica el coste de los trece minutos extra sobre /review.

El humano cuidadoso no es inútil; es complementario. La inconsistencia 4 (el patrón `Result<T>` vs excepción) la cazó porque conoce el módulo. Los métodos asistidos también la cazaron, pero la lectura humana de la consistencia con el ámbito sigue siendo un diferenciador en revisión final. La práctica que funciona en equipos serios: asistida primero, humano cuidadoso después con la lista asistida delante. El humano se centra en lo opinable; lo gordo ya está cazado.

Los nits son ruido en casi todos los métodos. Tanto /review como /ultrareview cazan el `temp1` y lo marcan como muy bajo. El humano lo dejó pasar conscientemente. Los nits son menú; los bugs son obligación. No reescribas tu flujo para los nits.

Bloque 5 · Cierre del módulo M4 y enlace al M5

Hemos cerrado el módulo M4 con la evidencia que justifica los veinticinco minutos invertidos.

La revisión humana es lineal en horas; la producción del agente no. El cuello aparece en code review y degrada la calidad efectiva del código que llega a producción. La salida no es revisar más rápido; es revisar distinto, con asistencia que cargue contexto que el humano cansado no se permite cargar.

`/review` es el reflejo a instalar en el cierre de cada rama. Comando integrado, dos a tres minutos, mejor cobertura que la revisión humana acelerada. Combínalo con el archivo de decisiones de GSD si lo tienes; te ahorra discutir cuestiones que ya estaban cerradas.

`/ultrareview` entra en código sensible, ramas grandes, o cuando no hay revisión humana posterior. Aporta dimensión de consenso entre revisores, perspectivas específicas (security, performance, consistencia), y hallazgos extra que la pasada de propósito general no daría. Coste mayor; cobertura cualitativamente distinta.

En el módulo M5 entramos en una skill de naturaleza distinta a las cuatro anteriores: **Context Mode**, la pieza que comprime tu contexto de sesión para que dure más antes de saturarse. Si Superpowers cambia el procedimiento, GSD da continuidad entre sesiones, y `/review - /ultrareview` arreglan el cuello del code review, Context Mode ataca el cuello técnico subyacente: el límite del contexto del agente. Cuatro submódulos en M5.

Glosario rápido del bloque

- **Bug sembrado** — problema introducido a propósito en el PR de demo para medir capacidad de detección de cada método. Permite comparar cobertura sin depender de la suerte.
- **Cobertura efectiva** — el ratio de problemas reales detectados sobre los presentes. Métrica honesta de calidad de revisión.
- **Hallazgo extra** — problema real cazado por un método que ningún otro detectó. La señal de que el método aporta capacidad cualitativamente distinta.
- **Asistida primero, humano después** — la práctica de equipo que funciona en proyectos serios. La asistida caza lo gordo; la humana se concentra en lo opinable.

Qué viene en M5

Context Mode, una skill que comprime tu contexto de sesión hasta diez o más veces sin perder información operativa. Cuatro submódulos: el problema (la saturación del contexto en sesiones largas), qué es Context Mode operativamente, cómo se instala y se usa, y demo de antes/después en una sesión real. Veinticinco minutos.

PARTE 5

Context Mode · comprimir para durar más

M5.1 · El problema de la saturación del contexto

Bloque 0 · La escena del agente que se hace tonto

Llevas una hora larga en una sesión productiva. La pieza es grande, hay decisiones tomadas, código escrito, tests pasando. El agente está respondiendo bien.

Y entonces lo notas. Le pides algo que en la primera media hora habría resuelto en treinta segundos, y tarda. Le señalas un fragmento del código y te pregunta dónde está, cuando lo escribió él hace cuarenta minutos. Le recuerdas una decisión cerrada hace una hora y la repropone como si fuera nueva. Le pides un cambio en un archivo y el archivo aparece editado, sí, pero también un archivo distinto que no le pediste tocar.

Si pruebas a reiniciar la sesión y rehacer la última hora desde cero, el agente vuelve a estar afilado. No es problema del agente; es problema de la sesión.

Si esto te ha pasado más de cinco veces, este módulo va dirigido a ti. Y si ya has empezado a dividir tareas grandes en sesiones pequeñas para evitarlo, estás compensando con tu tiempo lo que la herramienta debería gestionar mejor.

Bloque 1 · El diagnóstico — el problema número cuatro

En M2 vimos la prisa. En M3, el corte entre sesiones. En M4, el cuello del code review. Aquí va el cuarto, técnicamente el más sutil de los cuatro: **dentro de una misma sesión, el contexto del agente se satura y el comportamiento se degrada de formas que no anuncia.**

Tres detalles que distinguen este problema de los anteriores:

No es comportamiento; es física del contexto. El agente tiene una ventana de contexto finita. Cuando el historial de la sesión se acerca al límite, cosas viejas empiezan a desplazarse fuera o a recibir menos atención efectiva. No hay decisión consciente del agente; es la dinámica subyacente del modelo.

La degradación no anuncia su llegada. La sesión sigue funcionando. Las respuestas tienen forma normal. El código compila. Lo que cambia es la calidad sutil: el agente confunde dos archivos parecidos, olvida una decisión que ya estaba cerrada, repite trabajo que ya hizo. Cada error individual parece despiste; sumados, descubres que hay un patrón.

El problema empeora con la productividad. Cuanto mejor aplicas M2 (procedimiento), más turnos genera la sesión, más contexto se acumula, más rápido aparece la saturación. Paradoja molesta: la skill que más te ayuda a hacer las cosas bien también es la que más rápido satura el contexto del agente.

Para distinguirlo bien del problema del M3:

M3 era el problema de la sesión que termina. Cierras el chat, abres uno nuevo, hay un corte real. Lo que se pierde es el contexto entero, y la skill (GSD) instala protocolo de paso entre sesiones.

M5 es el problema de la sesión que sigue viva pero se degrada. No hay corte; hay decadencia. Lo que se pierde es parte del contexto, gradualmente, dentro de la misma sesión. Aquí no hace falta protocolo de paso; hace falta **comprimir** el contexto para que más quepa dentro del límite.

Los dos problemas son vecinos y se confunden con frecuencia, pero las soluciones son distintas. GSD no sirve para esto; Context Mode no sirve para lo del M3. Se complementan; no se sustituyen.

Bloque 2 · Por qué "vaciar y reiniciar" no es la solución

La primera reacción razonable cuando notas la saturación es cerrar la sesión y abrir una nueva limpia. Si tienes GSD instalado, además, el handoff de M3 te ayuda a retomar con menos coste. Y esto funciona... a medias.

Tres problemas concretos del enfoque "cerrar y reiniciar":

El handoff no captura todo el contexto operativo de la sesión activa. GSD captura las decisiones de diseño y los pendientes; no captura el estado de trabajo en curso del tipo "estaba trazando un bug a través de tres archivos y llevaba media exploración". Esa exploración es contexto valioso que tendrías que reconstruir.

La fricción de cierre y reapertura es alta cuando es frecuente. Si pasa una vez por sesión grande, asumible. Si pasa tres o cuatro veces, la fricción acumulada hace que evites las piezas grandes. Cambias tu trabajo para no chocar con el límite en lugar de quitar el límite.

No siempre puedes cerrar. A veces estás en mitad de algo que no admite pausa cómoda: una serie de cambios que tienen que ir juntos, un debug que requiere mantener el modelo mental cargado. Reiniciar cuesta más que aguantar al agente degradado.

La salida que de verdad funciona es **comprimir el contexto sin cerrarlo**. La idea: el contenido viejo de la sesión se sustituye por una versión más densa que ocupa una fracción del espacio pero retiene la información operativa. El agente sigue viendo "el contexto entero", pero ese "entero" es ahora más compacto, y por tanto cabe más antes de saturar.

Eso es lo que Context Mode hace. En el siguiente submódulo (M5.2) entramos en qué es operativamente y cómo opera por dentro.

Bloque 3 · Lo que viene en el módulo

El módulo M5 son cuatro submódulos:

- M5.1 (este): el problema que la compresión de contexto aborda y la diferencia con el problema del M3.
- M5.2: qué es Context Mode operativamente. Repositorio (`mksglu/context-mode`), namespace `/contextmode:` , almacenamiento SQLite local, ratio típico de compresión (alrededor de sesenta veces a fecha de mayo de 2026, con la variabilidad esperada según el contenido).
- M5.3: cómo se instala, cómo se invoca dentro de una sesión, qué hacer cuando notas la primera saturación.
- M5.4: demo antes y después en una sesión real. Mismo trabajo, una sesión sin compresión y otra con; comparativa de cuánto dura el agente afilado.

Veinticinco minutos en total. Es un módulo medio; no tan extenso como M2 ni M3, pero igual de necesario en el flujo de quien usa Claude Code para piezas largas.

Una nota honesta antes de seguir: Context Mode es la skill más joven del catálogo de este curso. Lleva pocos meses en circulación, y eso significa dos cosas. Una buena: el desarrollo es activo y las cifras de adopción crecen rápido. Una a vigilar: las versiones cambian de forma frecuente, los comandos pueden variar entre releases, y la R5/15.1 del curso aplica con énfasis especial aquí. Cualquier comando concreto que veas en este módulo merece verificarse contra el README del repositorio en el momento que lo instales.

Bloque 4 · Cierre y enlace a M5.2

El problema número cuatro de Claude Code no es de comportamiento ni de protocolo; es de física del contexto. La ventana es finita y se satura. La degradación no anuncia su llegada; se nota cuando lo sutil empieza a fallar.

El problema vecino al del M3 pero distinto. M3 atiende el corte entre sesiones con protocolo de paso (GSD); M5 atiende la decadencia dentro de la misma sesión con compresión. Las dos skills se complementan; ninguna sustituye a la otra.

"Cerrar y reiniciar" es solución a medias porque pierde contexto operativo no capturado por handoff, porque la fricción acumulada cambia lo que trabajas para no chocar con el límite, y porque no siempre puedes pausar. La salida es comprimir sin cerrar. Eso es Context Mode, y en el siguiente submódulo (M5.2) entramos en qué es operativamente.

Glosario rápido del bloque

- **Saturación de contexto** — el efecto de aproximarse al límite de la ventana del agente. La degradación es gradual y no se anuncia.
- **Context rot** — sinónimo coloquial común en el ecosistema. La degradación de calidad de las sesiones largas.
- **Compresión de contexto** — sustituir el contenido viejo de la sesión por una versión más densa que retiene la información operativa con menos tokens. La solución que Context Mode implementa.
- **M3 vs M5** — atajo mental para no confundirlos: M3 es el corte entre sesiones (GSD); M5 es la decadencia dentro de la misma sesión (Context Mode).

Qué viene en M5.2

Qué es Context Mode operativamente. Repositorio, namespace, almacenamiento local, ratio de compresión típico, qué tipo de contenido comprime mejor y qué tipo peor. Diez minutos para tener el modelo mental claro antes de instalarlo en M5.3.

M5.2 · Qué es Context Mode operativamente

Bloque 0 · La promesa del submódulo

En M5.1 dejamos el diagnóstico: el contexto del agente se satura dentro de la sesión viva, la degradación no anuncia, "cerrar y reiniciar" es solución a medias. Context Mode es la skill que aborda el problema con compresión en lugar de paso entre sesiones.

En este submódulo vamos a fijar lo siguiente: qué es operativamente, cómo opera por dentro a alto nivel, qué tipos de contenido comprime bien y cuáles peor (lo que decide si te aporta o te frustra), y cuál es su lectura contra los cuatro filtros del Mo.3. Sin esto, la instalación del M5.3 sería un comando ciego.

Aviso de versión: Context Mode lleva pocos meses en circulación a fecha de mayo de 2026. Los detalles operativos del repo `mksglu/context-mode` (comandos exactos, parámetros, ratios reportados) pueden afinarse en versiones siguientes. El README del repositorio manda sobre lo que veas aquí si hay discrepancia.

Bloque 1 · Qué es operativamente

Context Mode es una **skill instalable en Claude Code** que añade un namespace nuevo (`/contextmode:`) con comandos para gestionar la compresión del contexto vivo de tu sesión.

Tres detalles operativos importantes:

El namespace es propio. Cuando invocas `/contextmode:` algo, Claude Code sabe que va dirigido a la skill, no a un comando integrado. Eso significa que la skill convive con `/review`, `/ultrareview`, y cualquier otro comando integrado sin conflicto. Los comandos típicos que verás cuando la instales:

- `/contextmode:compress` — comprime el contexto vivo actual.
- `/contextmode:status` — muestra el estado del contexto y el ratio de compresión actual.
- `/contextmode:restore` — recupera un contexto comprimido previamente.

El nombre exacto de cada comando puede afinarse versión a versión; lo estable es el patrón namespace + verbo.

El almacenamiento es local. Context Mode usa una base de datos SQLite dentro de tu directorio de configuración de Claude Code. No envía nada a un servicio externo, no requiere

cuenta, no necesita autenticación. La base de datos guarda las compresiones que vas generando, etiquetadas para que las puedas recuperar después.

La compresión es activa, no pasiva. La skill no comprime automáticamente sin que tú lo pidas (eso podría romper sesiones en momentos delicados). El comando lo lanzas tú cuando notas saturación o cuando proactivamente quieres alargar la sesión. Eso te da control y un poco más de responsabilidad: si nunca lanzas el comando, la skill no te ayuda.

Para tu modelo mental: piensa en Context Mode como compresor del estado vivo, no como repositorio. Lo que comprime es lo que tu sesión tiene ahora mismo en el contexto; lo que persiste en SQLite es una versión consultable, pero el valor está en lo que pasa en la sesión activa.

Bloque 2 · La idea técnica detrás

A alto nivel, lo que Context Mode hace cuando le pides comprimir:

Identifica lo que el contexto vivo contiene. Mensajes del usuario, respuestas del agente, archivos cargados, resultados de comandos. Cada bloque tiene un coste en tokens reconocible.

Detecta lo redundante y lo verboso. Si has hecho cinco rondas de iteración sobre un mismo archivo, el contexto vivo contiene cinco versiones del archivo, cinco rondas de "este cambio, mira, lo aplico" del agente, cinco rondas de tu confirmación. Solo la última versión del archivo es realmente operativa para lo que viene; las anteriores son histórico. Mismo patrón con búsquedas, exploraciones, hipótesis descartadas.

Sustituye lo redundante por un resumen estructurado. El resumen captura la información operativa: qué archivos están en el estado actual, qué decisiones se tomaron, qué pendientes hay. Pierde la verbosidad: las cinco rondas se vuelven una entrada del tipo *"Iteramos sobre `SubscriptionService.cs` hasta llegar a la versión actual. Decisión clave: idempotencia con 200 + estado actual. Test cubre tres casos."* Lo histórico no operativo se pierde; lo operativo se preserva.

Reporta el ratio de compresión. La skill te dice cuánto ocupaba el contexto antes y cuánto después. Los ratios reportados en sesiones reales rondan factor sesenta (sesenta veces menos espacio para preservar la misma información operativa) a fecha de mayo de 2026, con variabilidad alta según el tipo de contenido. Para piezas de exploración intensa el ratio puede ser mayor; para piezas más densas en código nuevo, el ratio baja.

Lo que la skill NO comprime:

- **El prompt actual del usuario.** Lo que acabas de escribir no se toca.

- **Los últimos turnos productivos.** La skill mantiene los últimos N turnos sin tocar para no romper la dinámica de la conversación.
- **Los archivos clave que la sesión está usando activamente.** Si estás iterando sobre un archivo, ese archivo se queda en su versión actual sin compresión adicional.

Esto importa para tu modelo mental: la compresión es del histórico, no del presente. Lo que estás haciendo ahora mismo está intacto; lo que se comprime es lo que llevaste hasta llegar a este punto.

Bloque 3 · Qué comprime bien y qué peor

Tres tipos de contenido donde Context Mode rinde mucho:

Exploración con muchos archivos leídos pero pocos modificados. Cuando le pides al agente que entienda una zona de la base de código antes de tocarla, el agente lee veinte, treinta archivos. Pocos se quedan en el estado final; la mayoría son referencia que el agente cargó y digirió. Aquí la compresión es enorme, porque el resumen es del tipo *"Exploré el módulo `Billing` para entender el patrón de cálculo de IVA. Implementación clave en `TaxResolver.cs`. Convención: las strategies viven en `Billing/Strategies/`".* Cinco líneas en lugar de quince mil tokens de archivos.

Iteración larga sobre un mismo problema. Cuando un cambio se ha intentado de tres formas distintas antes de llegar a la cuarta que funciona, las tres primeras son historia. La compresión se queda con el resumen de "se probó A y B, no funcionaron por X razón; la solución es C". El espacio que ocupaban A, B, y sus rondas de discusión se libera.

Sesiones de debugging con muchas hipótesis descartadas. El bug se buscó en cinco capas distintas; al final aparece en la sexta. Las cinco descartadas son ruido para lo que viene. Comprimirlas a "se descartó hipótesis en Auth, Cache, DB, Mapper, Logger" preserva lo útil (no volver a buscar allí) y libera espacio.

Dos tipos donde la compresión rinde menos:

Trabajo concentrado en un único archivo grande con muchos cambios en sucesión. Si has hecho doscientas líneas de modificación en un solo archivo, todas relevantes, la versión actual del archivo ya contiene la información. La compresión se queda con lo mismo, dando ratio modesto. Sigue compensando lanzarla, pero no esperes los grandes números aquí.

Sesiones donde acabas de cargar muchos archivos nuevos. Si la sesión empieza con una carga grande para tener todo el contexto, comprimir antes de haber trabajado con esos archivos

sustituye texto que el agente necesitaba completo por un resumen que podría perder matices. Espera a haber trabajado un rato; entonces comprime.

La heurística: comprime cuando notes saturación o cuando una pieza grande lleve más de una hora y tengas otra hora por delante. Para piezas cortas, no compensa la fricción.

Bloque 4 · Los cuatro filtros del M0.3 aplicados

Viva: con asterisco. El repo tiene actividad reciente y commits frecuentes a fecha de mayo de 2026, pero lleva pocos meses en circulación. La cuestión a vigilar es si ese ritmo se sostiene en seis u ocho meses. Si te lo planteas para producción seria, mira el ritmo de commits y de issues respondidos antes de comprometerte. Por ahora, el filtro se cumple con la precaución de un proyecto joven.

Adoptada: cumple con la lectura honesta. Las cifras absolutas de estrellas son modestas comparadas con Superpowers o GSD, pero el crecimiento es sostenido y la comunidad activa. El factor diferencial es que aborda un problema que pocas skills atacan; eso le da espacio.

Reversible: sí, claramente. Skill estándar, namespace propio, base de datos local en tu directorio de configuración. Si la quitas, la base SQLite queda sin uso pero no rompe nada; puedes borrarla a mano. Las sesiones que comprimiste con la skill seguían siendo sesiones de Claude Code normales; el histórico operativo está en tu memoria y en GSD si lo usas.

Compone: sí, y de forma destacable con dos skills del catálogo. Con Superpowers se complementa (Superpowers genera sesiones más densas que se benefician de compresión). Con GSD se solapa parcialmente sin chocar (GSD captura entre sesiones, Context Mode dentro de la misma); las dos se usan juntas sin fricción. Con `/review` y `/ultrareview` es independiente; cualquier sesión, comprimida o no, los puede invocar igual.

Cumple los cuatro con un matiz de juventud en viva. Por eso entra al catálogo y por eso este módulo es el más sensible al énfasis R5/15.1: verifica versiones antes de comprometerte.

Bloque 5 · Cierre y enlace a M5.3

Context Mode es compresor del estado vivo, con namespace propio, almacenamiento SQLite local, activación manual. La compresión es del histórico de la sesión; el presente queda intacto. Eso te da control y te exige acordarte de lanzar el comando.

La idea técnica es sustituir lo redundante y verboso por un resumen estructurado que preserva la información operativa. Los ratios reportados rondan factor sesenta a fecha de mayo de 2026, con

variabilidad alta según el tipo de contenido. Comprime bien la exploración con muchos archivos leídos, las iteraciones largas, y las sesiones de debugging con hipótesis descartadas. Comprime peor el trabajo concentrado en un archivo grande y las sesiones recién cargadas.

Pasa los cuatro filtros del M0.3 con un asterisco en viva por la juventud del proyecto. Por eso entra al catálogo y por eso aplica con énfasis la R5/15.1: comprueba el README y el ritmo del repositorio al instalar.

En el siguiente submódulo (M5.3) entramos en lo operativo: instalación, primer comando, y qué hacer cuando notas la primera saturación. Cinco minutos para tener Context Mode listo y la mecánica de uso interiorizada.

Glosario rápido del bloque

- **Namespace** `/contextmode:` — el prefijo de los comandos de la skill. Convive con los comandos integrados sin chocar.
- **Compresión activa** — la skill no comprime automáticamente; tú lanzas el comando. Decisión consciente del proyecto para no romper sesiones en momentos delicados.
- **Ratio de compresión** — la relación entre tamaño del contexto antes y después. Rondas reportadas: factor sesenta con variabilidad. Mira el reporte del comando para el ratio de tu sesión concreta.
- **Información operativa** — lo que la sesión necesita preservar tras la compresión: archivos en estado actual, decisiones tomadas, pendientes. Lo que se pierde: verbosidad histórica, rondas de iteración, hipótesis descartadas.

Qué viene en M5.3

Instalación con el comando exacto del README, primer uso (`/contextmode:status` antes de comprimir, `/contextmode:compress` cuando notas saturación, `/contextmode:status` después para ver el ratio), y qué hacer si la compresión te pierde un matiz que necesitabas. Cinco minutos.

M5.3 · Instalación y primer uso de Context Mode

Bloque 0 · Qué cubrimos en cinco minutos

Tres cosas: instalación con el comando del repo `mksglu/context-mode`, primer uso (status antes, compresión, status después), y qué hacer en el escenario menos común pero importante de que la compresión te haya perdido un matiz que necesitabas seguir usando.

Si vas a seguir el módulo en directo, abre tu Claude Code en una sesión nueva. Vamos a generar saturación a propósito para probar la skill sobre material real.

Bloque 1 · La instalación

El repositorio del proyecto es `mksglu/context-mode`. La forma de instalación más común para skills de este tipo es la de marketplace comunitario:

```
/plugin marketplace add mksglu/context-mode
/plugin install context-mode@context-mode
```

Algunas versiones del proyecto distribuyen también la skill como clon directo en `~/.claude/skills/context-mode`. Si el README del repo apunta a esa vía, sigue el clon manual y reinicia Claude Code.

Cosas a tener delante:

La instalación es ligera. Pocos archivos, pocas dependencias. Cuestión de segundos.

Tras instalar, lanza `/contextmode:status` en una sesión cualquiera para confirmar que la skill responde. Si responde con un mensaje del estilo *"Contexto actual: X tokens. Sin compresiones previas"*, la skill está activa y conectada al ciclo de tu sesión. Si no responde, revisa con `/skills list` o equivalente que la skill aparece en la lista.

La base de datos SQLite se crea sola. La primera vez que comprimes algo, la skill crea su SQLite en tu directorio de configuración. No tienes que prepararla.

Si la instalación falla por permisos, `/permissions` y vuelves a intentarlo.

Bloque 2 · El primer uso, paso a paso

Vamos a generar la saturación a propósito para probar el ciclo completo. En una sesión nueva con Context Mode instalado:

Paso 1 — Genera contexto suficiente. Pídele al agente que explore una carpeta razonablemente grande de tu proyecto. Algo del tipo:

"Léete los archivos de la carpeta `src/Services/` y dame un resumen de cómo está organizada la capa de servicios."

El agente lee diez, quince archivos, te devuelve un resumen. Tu contexto ahora tiene volumen.

Paso 2 — Mira el status. Lanza `/contextmode:status`. Te devuelve algo del estilo:

"Contexto actual: 47.300 tokens (de 200.000 disponibles). 23% de uso. Sin compresiones previas."

Anota mentalmente el número de tokens. Es la línea de base.

Paso 3 — Comprime. Lanza `/contextmode:compress`. La skill tarda unos segundos. Te devuelve un reporte:

"Compresión aplicada. Antes: 47.300 tokens. Después: 800 tokens. Ratio: 59x. Información operativa preservada: estructura de la capa de servicios, convenciones detectadas, dependencias clave entre módulos."

Lo que ha hecho la skill es sustituir los quince archivos cargados verbatim por un resumen estructurado que captura lo que la sesión va a usar después.

Paso 4 — Verifica. Lanza otro `/contextmode:status` y comprueba que el número de tokens ha bajado al nivel reportado.

Paso 5 — Sigue trabajando normalmente. Esta es la parte importante. El agente sigue teniendo "memoria" de la exploración, pero esa memoria es ahora densa. Le puedes pedir tareas que dependan del trabajo anterior:

"Basándote en lo que viste de `src/Services/`, propón dónde añadir el servicio nuevo de auditoría que vamos a empezar."

Y el agente responde con criterio, usando la información operativa preservada. No tiene los archivos verbatim cargados, pero sí lo que la sesión necesita.

Si el agente responde como si hubiera olvidado la exploración, la compresión perdió algo. Pasa al bloque siguiente.

Bloque 3 · Qué hacer cuando la compresión pierde un matiz

Es el escenario menos común pero importante de cubrir. La compresión es densa pero no es perfecta: a veces resume una decisión con menos contexto del que la sesión va a necesitar después, o agrupa cosas que en realidad había que mantener separadas.

Tres maneras de detectarlo:

- El agente responde "no recuerdo bien ese detalle" después de la compresión, cuando antes lo manejaba sin problema.
- El agente toma una decisión que contradice algo discutido antes de comprimir.
- El agente pide que le recargues un archivo que ya había leído en la sesión.

Tres salidas posibles, en orden de menos a más invasivo:

Vuelve a darle el matiz a mano. Si lo que falta es una decisión concreta, le puedes recordar en un turno: *"Quedó cerrado que la auditoría va síncrona dentro de transacción, no asíncrona; mantén eso."* Es un turno corto, no rompe la dinámica.

Recarga el archivo concreto que el agente había olvidado. Si lo que falta es un archivo de referencia, pídele al agente que lo lea de nuevo. La compresión liberó el espacio; cargarlo otra vez cuesta tokens pero ahora cabe.

Restaura el contexto previo. Si la compresión fue demasiado agresiva y has perdido material crítico, `/contextmode:restore` recupera la versión guardada antes de la última compresión. Es la red de seguridad: la compresión es reversible si la activas dentro de la misma sesión.

La heurística de uso: en la mayoría de sesiones la compresión funciona limpia y no necesitas ni recordatorios ni restauración. Cuando notes que algo se perdió, el remedio menos invasivo casi siempre basta. `/contextmode:restore` queda como red de seguridad por si la compresión va mal en una pieza delicada.

Bloque 4 · Cierre y enlace a M5.4

Cinco minutos, Context Mode instalado, un ciclo completo de compresión hecho, y el remedio para el escenario raro de matiz perdido en la mochila.

En el siguiente submódulo (M5.4) cerramos el módulo M5 con la demo de antes y después en una sesión real: misma pieza grande, una sesión sin comprimir nada, otra con dos compresiones a lo largo. Comparativa de cuánto tiempo dura el agente afilado, cuántas piezas se cierran en una sesión, y qué pasa con la calidad en los últimos turnos.

Glosario rápido del bloque

- `/contextmode:status` — comando que reporta el estado actual del contexto y compresiones aplicadas. Útil antes y después de comprimir para ver el ratio.
- `/contextmode:compress` — comando que aplica la compresión sobre el contexto vivo. Mantiene presente intacto, comprime histórico.
- `/contextmode:restore` — comando que recupera la versión del contexto anterior a la última compresión. Red de seguridad si la compresión pierde algo importante.
- **Recordar el matiz a mano** — remedio mínimo cuando la compresión deja fuera algo que la sesión va a necesitar. Un turno corto suele bastar.

Qué viene en M5.4

Demo antes y después. Misma sesión, mismo trabajo, dos escenarios: una pasada sin compresión hasta saturar, otra con dos compresiones a tiempo. Comparativa de cuántas piezas se cierran, cuándo aparece la degradación en cada caso, y qué le cuesta a tu reloj cada uno. Siete minutos que cierran el módulo M5.

M5.4 · Demo — la misma sesión con y sin compresión

Bloque 0 · El experimento

Tomamos una pieza grande del proyecto .NET: añadir un módulo completo de notificaciones (email, SMS, push) que se enchufa al sistema de eventos de dominio que ya tenemos. Cinco subsistemas a tocar, decisiones de diseño en cascada, tests para cada canal.

El experimento tiene dos pasadas paralelas:

- **Pasada A** — sesión normal, sin Context Mode. Trabajamos hasta que el agente se degrada y cuantificamos hasta dónde llegamos.
- **Pasada B** — misma sesión, mismo trabajo, con Context Mode activo y dos compresiones aplicadas en momentos elegidos.

Mismo punto de partida en las dos. Misma duración objetivo: tres horas de trabajo. Lo que cambia es solo el uso o no de la skill.

Bloque 1 · Pasada A — Sin compresión

Hora 0. Arrancas la sesión limpia. El agente está afilado. Pides exploración inicial de los módulos `Customers`, `Subscriptions`, `Billing` para que tenga el contexto de cómo se emiten eventos. Quince archivos leídos. Decisión cerrada sobre el patrón de notificación que vais a usar.

`/contextmode:status` simulado: 38.000 tokens.

Hora 1. Implementación del primer canal (email). Decisiones de diseño tomadas: usar `INotificationChannel` como abstracción, una clase por canal, tests con un fake. Diecisiete archivos modificados o creados. Todo va fluido.

`/contextmode:status` simulado: 92.000 tokens (46%).

Hora 2. Implementación del segundo canal (SMS). Empieza a aparecer el primer síntoma: cuando le pides al agente que mire el patrón de email para replicarlo en SMS, tarda más de la cuenta y termina pidiendo que recargue el archivo de email. Lo cargas. Continúa, ahora más lento.

A la hora dos y veinte minutos, el agente toma una decisión que contradice la del minuto cuarenta: cambia ligeramente la firma de `INotificationChannel` sin proponerlo, lo cual rompe el

canal email implementado en hora uno. El test del email falla. Tú detectas, le señalas, el agente ajusta. Veinte minutos perdidos.

`/contextmode:status` simulado: 138.000 tokens (69%).

Hora 2:40. Empezando el tercer canal (push). El agente confunde dos archivos parecidos (`PushNotificationChannel.cs` y `PushNotificationOptions.cs`) y aplica un cambio en el equivocado. Tú detectas, deshaces, ajustas. Quince minutos más perdidos.

Hora 3. El tercer canal a medio implementar. Decisión consciente de parar: el agente está claramente degradado, la pieza no va a cerrar bien en lo que queda de sesión. Cierras.

Resultado de la pasada A:

- Pieza al 65% (dos canales completos, tercero a medias).
- Tres incidentes de degradación que costaron 35 minutos de fix.
- Decisión de cierre por degradación, no por terminación.
- Para continuar, sesión nueva mañana con todo el coste de retomada.

Bloque 2 · Pasada B — Con compresión a tiempo

Reset. Mismo trabajo, mismo agente, Context Mode activo.

Hora 0. Igual que en A. Exploración inicial, decisión cerrada, agente afilado. 38.000 tokens.

Hora 1. Igual que en A. Email implementado, tests verdes, todo fluido. 92.000 tokens.

Hora 1 — momento clave. Lanzas `/contextmode:compress`. La skill comprime el histórico de la exploración inicial y de la implementación de email, manteniendo el estado actual (archivos modificados, decisión cerrada sobre `INotificationChannel`, patrón establecido).

Reporte: "Antes 92.000 tokens, después 4.500. Ratio 20x. Preservado: estructura de capa de servicios, patrón `INotificationChannel` con su decisión D-001, estado actual del canal email completado."

`/contextmode:status` : 4.500 tokens (2%).

Hora 2. Implementación de SMS. Como el contexto está limpio, el agente trabaja con la misma agilidad que en hora cero. Mira los archivos de email cuando los necesita (recargándolos sale barato porque hay espacio), aplica el patrón a SMS sin desviarse, cierra el canal. Sin incidentes.

`/contextmode:status` : 58.000 tokens.

Hora 2:30 — segunda compresión. Antes de empezar el tercer canal, comprimes. Reporte: "Antes 58.000, después 6.200. Ratio 9x.". La sesión vuelve a estar ligera.

Hora 3. Tercer canal (push) implementado entero, tests pasando, decisiones documentadas. La pieza cierra a las tres horas con los tres canales completos y la integración con el sistema de eventos hecha.

Resultado de la pasada B:

- Pieza al 100%.
- Cero incidentes de degradación.
- Decisión de cierre por terminación, no por degradación.
- Sin coste de retomada para mañana porque no hay retomada que hacer.

Bloque 3 · La comparativa lado a lado

Dimensión	Sin Context Mode	Con Context Mode
Pieza completada al final de la sesión	65%	100%
Incidentes de degradación	3	0
Tiempo perdido en fix de degradación	35 minutos	0 minutos
Calidad del último turno productivo	degradada	igual que el primero
Necesidad de sesión adicional para terminar	sí	no
Coste mental del cierre	"no lo cerré bien"	"cerrado limpio"

Cuatro datos para llevarte:

La compresión a tiempo es la palanca. La pasada B no es mejor porque tenga compresión; es mejor porque la compresión llegó antes de que el agente entrara en zona degradada. Si hubieras esperado a la hora 2:30 para la primera compresión, parte del daño ya estaría hecho. La heurística que funciona: comprime cuando llegas al 40% del contexto, no cuando llegas al 80%.

La degradación cuesta tiempo doble. Los 35 minutos de fix no son solo los 35 minutos; son la fricción mental de descubrir que el agente se rompió, dudar de qué más se ha roto, y volver a confiar en lo que entrega después. Esa carga emocional no aparece en la tabla pero afecta cómo terminas la sesión.

Las compresiones también ahorran sesiones siguientes. En la pasada A tendrías que abrir sesión mañana solo para terminar el tercer canal. En la B, mañana empiezas otra pieza. La ganancia se compone: cada sesión que no necesitas abrir es media hora de retomada que ahorras.

El coste de Context Mode es muy bajo cuando lo conviertes en reflejo. Dos comandos en tres horas de sesión. Veinte segundos de fricción total. La ganancia es desproporcionada respecto al coste de uso.

Bloque 4 · Cierre del módulo M5 y enlace al M6

Hemos cerrado el módulo M5.

El problema número cuatro de Claude Code es de física del contexto, no de comportamiento ni de protocolo. La sesión sigue viva pero se degrada de formas que no anuncia. La salida no es cerrar y reiniciar; es comprimir sin cerrar.

Context Mode comprime el histórico de la sesión preservando lo operativo. Ratios reportados de orden sesenta. Comprime mejor la exploración con muchos archivos leídos y la iteración larga; comprime peor el trabajo concentrado en archivos grandes ya en estado final. Activación manual, almacenamiento local, namespace propio. Pasa los cuatro filtros del Mo.3 con el asterisco de juventud del proyecto.

La heurística de uso que rinde: comprime cuando llegas al 40% del contexto, no esperes al 80%. Convierte el `/contextmode:status` en chequeo cada media hora. Las compresiones bien colocadas convierten una sesión de tres horas en una sesión que cierra la pieza, en lugar de una que cierra al 65% y obliga a abrir otra mañana.

En el módulo M6 entramos en la sexta skill del catálogo: **claude-mem**. Aborda un problema vecino al de Context Mode pero distinto: no la compresión del contexto vivo de una sesión, sino la **memoria persistente y consultable entre proyectos**. Si Context Mode te alarga la sesión, claude-mem te da memoria a largo plazo de patrones, decisiones, y aprendizajes que valen a través de proyectos distintos. Cinco submódulos.

Glosario rápido del bloque

- **Saturación gradual** — la degradación que aparece sin avisar dentro de una sesión larga. Síntomas: olvidos, contradicciones, confusión entre archivos parecidos.
- **Compresión a tiempo** — la práctica de comprimir antes de que el agente entre en zona degradada. Regla útil: al 40% del contexto, no al 80%.

- **Coste mental del cierre** — la dimensión cualitativa que diferencia cerrar una pieza al 100% de cerrar al 65% por agotamiento. No aparece en métricas pero pesa en cómo terminas la jornada.
- **Reflejo de status** — la práctica de lanzar `/contextmode:status` periódicamente (por ejemplo cada media hora de sesión activa) para decidir cuándo comprimir. Equivalente al reflejo de `/review` antes de PR.

Qué viene en M6

claude-mem, la skill de memoria persistente. Cinco submódulos: el problema (lo que sabes en un proyecto no se transfiere a otro), qué es claude-mem operativamente (plugin no MCP, SQLite + Chroma, `npx claude-mem install`), instalación, primeros usos, y demo de cómo cambia el ritmo cuando arrancas un proyecto nuevo con memoria previa cargada. Treinta minutos en total.

PARTE 6

claude-mem · memoria persistente entre proyectos

M6.1 · El problema de la memoria que no se transfiere entre proyectos

Bloque 0 · La escena del proyecto nuevo donde el agente no te conoce

Llevas seis meses trabajando con Claude Code en un proyecto serio. Has tomado cientos de decisiones de diseño, has aprendido cómo el agente responde mejor en tu estilo de proyecto, has descubierto qué patrones del proyecto valen y cuáles no. Si tienes GSD activo (M3), todo eso vive en los artefactos del proyecto y la sesión nueva del mismo proyecto los hereda.

Hoy arrancas un proyecto nuevo. Repo recién creado, carpeta vacía, dependencias por configurar. Abres Claude Code y le pides la primera tarea.

Y entonces lo notas. El agente no sabe **nada**. No sabe que tu equipo prefiere `Result<T>` a excepciones para errores esperados. No sabe que en tus proyectos los tests viven al lado del código y no en una carpeta hermana. No sabe que en producción te has llevado bofetadas con SQLite en memoria y prefieres ficheros para los integration tests. No sabe que los nombres de variables tipo `temp1` te ponen de los nervios.

Empiezas de cero. Cada decisión que ya tenías cerrada en el proyecto anterior, la tienes que volver a tomar. Cada patrón que ya estaba bien resuelto, lo vuelves a explicar. La primera semana del proyecto nuevo se siente como retroceder seis meses en relación con el agente.

Si esta escena te ha pasado dos o tres veces, este módulo va dirigido a ti. Y si has empezado a ralentizar el arranque de proyectos nuevos para evitar la fricción, estás pagando tiempo por una limitación de la herramienta que sí tiene solución.

Bloque 1 · El diagnóstico — el problema número cinco

En M2 vimos la prisa. En M3, la sesión que se acaba dentro de un proyecto. En M4, el cuello del code review. En M5, la saturación de la sesión activa. Aquí va el quinto: **lo que el agente aprende contigo en un proyecto no se traslada a otro proyecto distinto**.

Tres detalles que distinguen este problema de los anteriores:

Es ortogonal a los cuatro anteriores. Puedes tener Superpowers, GSD, `/review` y Context Mode todos activos y este problema seguir intacto. Las cuatro skills cubren cosas dentro de un proyecto o dentro de una sesión; ninguna cubre lo que sabes cuando cambias de carpeta.

El daño es invisible al principio. La primera semana del proyecto nuevo se siente lenta pero asumes que es normal. La segunda semana sigues sintiéndola lenta y empiezas a pensar que el agente "ya no es como antes". No es el agente; es que arrancó sin nada.

Cuanto más maduro es tu uso de Claude Code, más sufres. Un usuario casual no tiene base de conocimiento implícita que transferir; arranca igual de novato siempre. Un usuario maduro tiene meses de aprendizajes y preferencias destiladas; perderlas al cambiar de proyecto duele en proporción a lo que había acumulado.

La distinción con M3 importa: en M3 el problema era el corte temporal dentro del mismo proyecto. Aquí el problema es el corte espacial: proyectos distintos, en carpetas distintas, posiblemente en repos distintos, pero del mismo dev (tú) con los mismos patrones de trabajo. GSD captura dentro del proyecto y no atraviesa proyectos. Context Mode comprime dentro de la sesión y no captura nada entre sesiones de proyectos distintos. Hace falta una pieza nueva.

Bloque 2 · Por qué los apaños caseros no llegan

La primera reacción razonable es montar una carpeta global del estilo `~/dev-notes/` con tus decisiones y preferencias destiladas. Al empezar un proyecto nuevo, le pides al agente que la lea. Tres problemas concretos del enfoque:

Tú no escribes esas notas con criterio. Cuando intentas destilar "lo que el agente debería saber sobre cómo trabajo", te das cuenta de que la mayor parte vive como hábito implícito que no sabes verbalizar. La nota termina siendo una lista superficial de cinco preferencias que se queda muy corta.

El agente nuevo lee las notas y arranca, pero no las aplica con criterio. Igual que en M3 con el archivo de notas casero: la sesión nueva lee, procesa, y luego no relaciona la nota con la decisión concreta del momento. La nota dice "prefiero `Result<T>` a excepciones para errores esperados", el agente arranca a escribir excepciones porque la pieza concreta del momento no le activa la asociación.

No hay continuidad bidireccional. Lo que aprendes en el proyecto nuevo no se vuelca de vuelta a la base global. Si en el proyecto nuevo descubres que un patrón mejor reemplaza a uno viejo, tu carpeta global no se entera. Pasa a estar desactualizada sin que lo notes.

La salida que funciona es **memoria persistente y consultable**: una capa que captura tus decisiones, patrones, errores y aprendizajes a lo largo de todas tus sesiones de Claude Code (no solo de un proyecto), las indexa con búsqueda semántica, y las inyecta al inicio de cada sesión nueva con criterio basado en el contexto del proyecto del momento.

Eso es **claude-mem**.

Bloque 3 · Lo que viene en el módulo

Aviso operativo importante antes de seguir: la skill se llama **claude-mem** (con guion). El vídeo origen del curso la transcribe como "cloud-mem" por error de transcripción automática del audio. Ten clara la grafía correcta para no perderte buscando.

El módulo M6 son cinco submódulos:

- M6.1 (este): el problema que claude-mem aborda y la diferencia con los problemas vecinos del M3 y M5.
- M6.2: qué es claude-mem operativamente. Plugin (no MCP server, como muchos asumen), almacenamiento local con SQLite + Chroma, captura continua, inyección al inicio.
- M6.3: la mecánica de captura y de inyección. Qué se guarda, cómo se decide qué inyectar al empezar, ratio de ahorro de tokens.
- M6.4: instalación con el comando real verificado y verificación operativa.
- M6.5: demo del cambio de ritmo al arrancar un proyecto nuevo con memoria previa cargada vs sin ella. Treinta minutos en total.

Una nota honesta: claude-mem es la skill más reciente del catálogo y la que más rápido está cambiando entre releases. La R5/15.1 del curso aplica con énfasis especial: cualquier comando concreto merece verificarse contra `docs.claude-mem.ai` en el momento que lo instales.

Bloque 4 · Cierre y enlace a M6.2

El problema número cinco de Claude Code es de transferencia, no de captura. Tu conocimiento de cómo trabajas con el agente existe; lo que no existe es la capa que lo lleva del proyecto viejo al nuevo. Las cuatro skills anteriores del catálogo no cubren esto; hace falta una pieza nueva.

La distinción con M3 y M5 es lo que más confunde a la gente al principio. M3 era continuidad dentro del proyecto entre sesiones (GSD). M5 era saturación dentro de la sesión activa (Context Mode). M6 es transferencia entre proyectos distintos del mismo dev. Las tres son problemas de memoria, pero cada una atiende un eje distinto.

Los apaños caseros (carpeta global de notas) fallan por la misma razón que fallaban en M3: el agente lee sin criterio y la base global no se actualiza bidireccionalmente. La salida es memoria persistente, indexada y consultable; claude-mem es la pieza del catálogo que la implementa. En el siguiente submódulo (M6.2) entramos en qué es operativamente.

Glosario rápido del bloque

- **Problema número cinco** — la imposibilidad de transferir lo aprendido en un proyecto al siguiente sin intervención manual. Lo que claude-mem aborda.
- **M3 vs M5 vs M6** — atajo mental: M3 es corte temporal dentro de un proyecto (GSD). M5 es decadencia dentro de la sesión (Context Mode). M6 es corte espacial entre proyectos (claude-mem).
- **Memoria persistente y consultable** — capa que captura decisiones y patrones a lo largo de todas las sesiones, los indexa con búsqueda semántica, y los inyecta al inicio de cada sesión nueva.
- **claude-mem** — nombre correcto de la skill (con guion). El vídeo origen lo transcribe mal como "cloud-mem".

Qué viene en M6.2

Qué es claude-mem operativamente. Plugin, no MCP server (a pesar de lo que mucha gente asume). Almacenamiento local con SQLite y Chroma. Captura continua a lo largo de la sesión. Inyección al inicio de cada sesión nueva. Diez minutos para tener el modelo mental antes de la mecánica detallada del M6.3.

M6.2 · Qué es claude-mem operativamente

Bloque 0 · La promesa del submódulo

En M6.1 dejamos el diagnóstico: el problema número cinco es la transferencia de conocimiento entre proyectos distintos del mismo dev. claude-mem es la skill que lo aborda. En este submódulo:

- Qué es operativamente (plugin, captura continua, almacenamiento local).
- Por qué **plugin y no MCP server**, distinción que mucha gente del nicho confunde y que cambia cómo se usa la skill.
- Qué hace SQLite + Chroma por dentro a alto nivel y por qué la combinación importa.
- Los cuatro filtros del Mo.3 aplicados.

Sin todo esto, la instalación del M6.4 sería un comando ciego y la primera sesión te confundirías al ver que las cosas pasan automáticamente cuando esperabas controlar manualmente cada paso.

Bloque 1 · Qué es operativamente

claude-mem es un **plugin de Claude Code** que añade tres capacidades coordinadas al ciclo de vida de tus sesiones:

Captura continua del trabajo activo. Mientras trabajas con el agente, el plugin va guardando en una base local todo lo relevante: archivos que editas, decisiones que tomas, comandos que ejecutas, errores que aparecen y cómo se resolvieron. No es captura agresiva del estilo "guarda el chat entero literal"; el plugin destila y resume sobre la marcha para que la base no crezca sin control.

Indexación con búsqueda híbrida. La base no es una pila de notas planas; es un índice con dos modos de búsqueda. Texto completo (palabras exactas) y semántica (significado). Eso permite que cuando la siguiente sesión necesite "lo que aprendiste sobre concurrencia en EF Core", el plugin encuentre el contenido relevante aunque ahora lo expreses con palabras distintas a las que usaste hace tres meses.

Inyección al inicio de cada sesión nueva. Cuando abres Claude Code en cualquier proyecto, el plugin lee el contexto (carpeta actual, primer prompt) y selecciona de la base lo que es probable que necesites. No te inyecta toda la memoria de los seis meses anteriores; te inyecta el

subconjunto relevante para lo que vas a hacer. Eso es lo que diferencia "memoria útil" de "ruido de historial".

Los ratios reportados en uso real rondan **factor diez de ahorro de tokens** comparado con recargar contexto a mano. La cifra varía mucho según el tipo de trabajo (proyectos con patrones bien definidos ahorran más; proyectos exploratorios ahorran menos), pero el orden de magnitud es estable.

Bloque 2 · Por qué plugin y no MCP server

Esta es la confusión más común con claude-mem y merece atención. En el ecosistema Claude Code hay dos formas técnicas distintas de añadir capacidades al agente: **plugins** y **MCP servers**. La diferencia importa porque cambia cómo se instala, cómo se invoca, y qué garantías ofrece cada uno.

Un MCP server es un servicio externo (suele correr en un proceso aparte) al que el agente llama explícitamente cuando necesita una capacidad concreta. Es invocación bajo demanda, con protocolo definido y tooling propio. La interacción típica: tú lanzas un prompt, el agente decide que necesita una capacidad MCP, llama al servidor, recibe la respuesta, sigue. Los MCP servers son la pieza para capacidades discretas (consultar una base externa, llamar a una API, etc.).

Un plugin es código que se ejecuta automáticamente en eventos del ciclo de vida del agente: al iniciar la sesión, al cerrar la sesión, al cambiar de turno, al detectar ciertas acciones. No espera a que el agente lo llame; se dispara por los hooks que tiene registrados.

claude-mem es plugin. La razón es coherente con lo que hace: necesita capturar continuamente (lo que requiere hook a cada turno) e inyectar al inicio (lo que requiere hook al arranque de sesión). Como MCP server tendría que ser el agente quien decidiera llamarlo, y la decisión de "ahora me interesaría recordar algo" es justamente la que el agente no está bien posicionado para tomar; precisamente le falta saberlo.

Esto tiene consecuencias operativas relevantes para ti:

El vídeo origen del curso dice "install cloud-mem mcp". Esto es doble error: el nombre real es `claude-mem` (con guion, no "cloud-") y la naturaleza es plugin, no MCP. Si intentas instalarlo como MCP server, no funciona. El comando real lo veremos en M6.4.

No tienes que invocar claude-mem por nombre nunca. A diferencia de Skill Creator (que invocas con `/skill-creator`) o de Context Mode (que invocas con `/contextmode:compress`), claude-mem trabaja en segundo plano. Tú no le hablas; él captura y inyecta cuando le toca por hook.

La gestión de fallos es distinta. Si claude-mem falla en una captura, la sesión continúa funcionando normalmente; pierdes esa pieza de memoria pero el flujo no se rompe. Si fuera MCP server, un fallo de invocación detendría al agente en el turno.

Bloque 3 · SQLite + Chroma — qué hace cada uno

La capa técnica de almacenamiento son dos bases coordinadas: **SQLite** y **Chroma**. La combinación no es por gusto; cada una resuelve un problema distinto.

SQLite mantiene el índice de **texto completo**. Es el motor de búsqueda relacional clásico, optimizado con FTS5 para que encuentre palabras exactas y combinaciones booleanas rápido. Cuando buscas "DbUpdateConcurrencyException", SQLite te devuelve cualquier entrada de la memoria que contenga ese término literal. Es rápido, predecible, y barato en recursos. La base SQLite vive como archivo único en tu directorio de configuración; puedes abrirlo con cualquier herramienta SQLite estándar si quieres inspeccionarlo a mano.

Chroma mantiene el índice **semántico**. Cada entrada de memoria se convierte en un vector (representación numérica del significado) y se almacena en Chroma. Cuando buscas "cómo manejábamos los choques en escrituras concurrentes", Chroma te encuentra la entrada de `DbUpdateConcurrencyException` aunque no compartas las palabras exactas. La base Chroma también vive localmente; no hay servicio externo.

¿Por qué las dos a la vez en lugar de una?

SQLite gana cuando sabes lo que buscas. Si recuerdas el nombre exacto del término, FTS5 es más rápido y más preciso que la búsqueda semántica.

Chroma gana cuando solo recuerdas el concepto. Si han pasado tres meses y solo recuerdas "habíamos resuelto un problema de concurrencia con algo de EF", la búsqueda semántica te encuentra las entradas relevantes aunque hayas olvidado la terminología exacta.

El plugin combina ambas en cada consulta y devuelve resultados ordenados por relevancia agregada. Tú no tienes que decidir qué tipo de búsqueda usas; el plugin lo orquesta. Lo importante para tu modelo mental es saber que las dos están ahí, que viven en local, y que puedes abrirlas si necesitas auditar qué recuerda el plugin sobre ti.

Bloque 4 · Los cuatro filtros del M0.3 aplicados

Viva: sí, con la precaución de proyecto reciente. El repo `thedotmack/claude-mem` tiene actividad de commits frecuente, releases regulares, y documentación dedicada en `docs.claude-mem.ai`. El

proyecto es más joven que Superpowers o GSD; mira el ritmo de commits y issues respondidos antes de comprometerte para producción seria.

Adoptada: sí, con crecimiento sostenido. Las cifras absolutas son aún modestas comparadas con las skills más establecidas, pero la conversación técnica viva en el nicho es notable. Aborda un problema que pocas piezas atacan; eso le da espacio.

Reversible: sí, claramente. Plugin estándar, almacenamiento local en tu directorio de configuración. Si lo quitas, las bases SQLite y Chroma quedan en disco sin uso; puedes borrarlas a mano. Las sesiones que tuviste con claude-mem activo seguían siendo sesiones normales de Claude Code; lo que el plugin capturó ya cumplió su función en el momento.

Compone: sí, y de forma destacable con el resto del catálogo. Con Superpowers se complementa (M2 genera sesiones más densas que claude-mem captura mejor). Con GSD no se solapa (M3 da continuidad dentro del proyecto; M6 da transferencia entre proyectos). Con Context Mode se solapa parcialmente sin chocar (M5 comprime dentro de la sesión; M6 captura entre sesiones). Soporta múltiples clientes agente además de Claude Code (OpenCode, Codex, Gemini, Hermes, Copilot), bonus al filtro.

Cuatro de cuatro con el matiz de juventud. Por eso entra al catálogo del curso.

Bloque 5 · Cierre y enlace a M6.3

claude-mem es plugin, no MCP server. La distinción importa porque el plugin trabaja en segundo plano por hooks de ciclo de vida; tú no lo invocas, él captura e inyecta automáticamente. Si intentas instalarlo como MCP (porque el vídeo origen lo dice así), no funciona.

El almacenamiento local combina SQLite (búsqueda por texto exacto) con Chroma (búsqueda semántica por significado). La combinación es lo que permite encontrar cosas tanto cuando recuerdas el nombre exacto como cuando solo recuerdas el concepto. Las dos viven en tu directorio de configuración y son auditable a mano si quieres ver qué recuerda de ti.

Pasa los cuatro filtros del M0.3 con el asterisco de juventud del proyecto. En el siguiente submódulo (M6.3) entramos en la mecánica detallada: qué captura el plugin exactamente, cómo decide qué inyectar al inicio de una sesión nueva, y por qué el ratio de ahorro de tokens ronda factor diez. Diez minutos para tener la mecánica clara antes de la instalación del M6.4.

Glosario rápido del bloque

- **Plugin (Claude Code)** — código que se dispara por hooks de ciclo de vida (inicio de sesión, cambio de turno, cierre). Trabaja en segundo plano sin invocación explícita del agente.
- **MCP server** — servicio externo al que el agente llama bajo demanda. Naturaleza distinta a la del plugin; relevante para descartar confusiones.
- **Captura continua** — el plugin guarda decisiones, archivos editados y errores resueltos a lo largo de la sesión, no como tarea final.
- **SQLite con FTS5** — almacén relacional con índice de texto completo. Gana cuando buscas palabras exactas.
- **Chroma** — almacén vectorial con búsqueda semántica. Gana cuando solo recuerdas el concepto.
- **Búsqueda híbrida** — combinación de SQLite + Chroma en una consulta única, con resultados ordenados por relevancia agregada. Lo que claude-mem hace cada vez que decide qué inyectar.

Qué viene en M6.3

La mecánica detallada del plugin. Qué captura exactamente (archivos editados, decisiones cerradas, errores resueltos, patrones detectados). Cómo decide qué inyectar al inicio de una sesión nueva (contexto del proyecto + primer prompt como pistas). Por qué el ratio de ahorro de tokens ronda factor diez. Diez minutos para tener la mecánica clara antes de la instalación.

M6.3 · La mecánica de captura e inyección

Bloque 0 · La promesa del submódulo

En M6.2 dejamos el modelo mental: plugin, almacenamiento local, búsqueda híbrida. Ahora vamos a los dos momentos operativos donde se decide si claude-mem te aporta valor o si te frustra.

Momento uno: la captura. Lo que el plugin guarda mientras trabajas con el agente. Si captura poco, no tendrá nada útil que recordar más tarde. Si captura demasiado, la base se llena de ruido y la inyección posterior se vuelve impredecible.

Momento dos: la inyección. Lo que el plugin te suelta al inicio de la siguiente sesión, en otro proyecto, semanas después. Aquí está la diferencia entre "qué bien, el agente ya sabe lo que aprendimos en agosto" y "el agente está hablando de cosas que no tienen que ver con lo que estoy haciendo".

En este submódulo:

- Qué decide guardar el plugin durante la captura, y qué descarta.
- Cómo decide qué inyectar al arrancar una sesión nueva.
- Por qué el ratio de ahorro de tokens ronda factor diez en lugar de "memoria infinita".
- Dos comportamientos no obvios que aparecen la primera vez y te confunden si no los esperas.

Bloque 1 · La captura — qué se guarda y qué se descarta

claude-mem no guarda la conversación literal. Si lo hiciera, la base crecería sin control y la búsqueda semántica se ahogaría en texto verbatim. Lo que hace es **destilación continua**: durante la sesión, el plugin identifica y guarda cuatro tipos de contenido.

Decisiones cerradas. Cuando una elección de diseño se cierra (sea por respuesta a Brainstorming si tienes Superpowers, sea por confirmación explícita en el chat), el plugin la captura como entrada con su contexto. Ejemplo: si en una sesión decides que "para errores esperados usamos `Result<T>` en lugar de excepciones porque las excepciones del runtime confunden el panel de errores de Sentry", esa decisión se guarda con la pieza completa: la regla, el porqué, el ámbito (proyecto X, módulo Y).

Patrones aplicados. Cuando el agente aplica un patrón de código que tú confirmas con un commit (o cuando tú aplicas uno a mano y el agente lo lee después), el plugin lo captura como

ejemplo destilado. No guarda el código verbatim; guarda la estructura del patrón ("para tests de concurrencia con EF Core usamos SQLiteFixture en disco, no en memoria") y un puntero al archivo donde se ve aplicado.

Errores resueltos. Cuando una sesión empieza con un error y termina con la solución, el plugin captura el par: síntoma del error, causa raíz identificada, solución aplicada. Esto vale especialmente entre proyectos: la próxima vez que aparezca un error similar en otro proyecto, claude-mem tiene la solución a mano.

Convenciones detectadas del proyecto. El plugin observa decisiones repetidas a lo largo de una sesión y las generaliza. Si tres veces seguidas confirmas que los tests viven al lado del código en lugar de en una carpeta hermana, el plugin lo captura como convención del proyecto, no como tres decisiones puntuales.

¿Qué se descarta?

- **Conversación verbatim larga.** Si has tenido una discusión de quince turnos sobre una pieza, el plugin no guarda los quince turnos; guarda el resultado destilado.
- **Lecturas exploratorias sin decisión.** Si el agente lee diez archivos para entender una zona del código y al final no se cierra nada concreto, esas lecturas no se guardan. La base no es un log; es un destilado.
- **Información sensible detectada.** El plugin tiene heurísticas para no guardar lo que parecen credenciales, claves API o secretos. Mira la documentación para los detalles exactos; si manejas datos sensibles, pásate por la sección de privacidad antes de la primera sesión.

La captura es **silenciosa**. No te interrumpe la conversación; no te pide confirmación de cada entrada. Si quieres revisar lo que ha capturado, lo haces consultando la base directamente (las dos bases SQLite y Chroma del M6.2 son abribles a mano).

Bloque 2 · La inyección — qué se devuelve y por qué eso

Aquí está el momento donde claude-mem decide si te aporta. Cuando arrancas una sesión nueva en cualquier proyecto, el plugin **selecciona del archivo destilado lo que probablemente vas a necesitar** y lo inyecta al inicio.

La selección no es "todo lo que tengo en memoria". Sería ruido y rompería la sesión nueva por saturación inmediata del contexto. La selección es por relevancia, usando dos pistas:

El contexto del proyecto actual. Carpeta donde arrancaste Claude Code, lenguaje principal detectado, dependencias del `package.json` o el `.csproj`. Esto le dice al plugin qué subconjunto

de la memoria es plausiblemente relevante. Si arrancas un proyecto Python, el plugin no te inyecta convenciones de .NET; si arrancas un proyecto .NET, no te inyecta patrones de Node.js.

Tu primer prompt. Esta es la pista más fina. Si tu primer prompt es "vamos a añadir autenticación al endpoint de usuarios", el plugin lanza búsqueda híbrida en la base con esos términos. SQLite caza coincidencias literales ("autenticación", "endpoint", "usuarios"), Chroma caza coincidencias semánticas ("login", "auth", "API user management"). El resultado: un puñado de entradas previas relevantes (decisiones de auth, patrones de endpoint que ya usabas, errores típicos resueltos).

Lo que el plugin te inyecta es ese puñado, no toda la memoria. Típicamente entre cinco y quince entradas. No las ves como mensaje del agente; las ves como contexto cargado al inicio. El agente las trata como conocimiento de fondo y arranca con criterio aplicado.

Tres consecuencias operativas:

El primer prompt importa mucho. Si arrancas con un prompt vago ("hola"), el plugin tiene poco con qué decidir. Si arrancas con un prompt específico que describe lo que vas a hacer, la selección es buena. Práctica que funciona: trata el primer prompt como un mini-brief de la pieza.

La calidad mejora con uso. Las primeras semanas con claude-mem activo, la inyección puede sentirse pobre (poca memoria capturada, baja confianza en la relevancia). A las tres o cuatro semanas, cuando hay base suficiente, la inyección empieza a sentirse claramente útil. Hay un periodo de calentamiento.

La inyección es invisible si no la buscas. El agente no te dice "inyecté tres entradas de memoria". Aparecen como contexto y se aplican. Si quieres ver qué te inyectó, la documentación oficial tiene un comando para listar lo cargado en la sesión actual.

Bloque 3 · Por qué factor diez y no infinito

El ratio que se reporta sobre claude-mem es **alrededor de factor diez de ahorro de tokens** al cargar contexto histórico. Es buen número, pero merece entender por qué no es más alto y qué lo limita.

El factor diez es comparado con recargar contexto a mano. La comparación natural: imagina que sin claude-mem tienes que abrir tu carpeta global de notas, copiar los párrafos relevantes, y pegarlos al inicio de la sesión nueva. Eso costaría X tokens de contexto. Con claude-mem, la inyección destilada cuesta aproximadamente X/10.

No es más alto por dos razones. Una: la inyección tiene que ser legible para el agente, no comprimida al límite. Si la comprimes demasiado, el agente no puede aplicarla. Dos: la inyección

incluye contexto suficiente para que cada entrada tenga sentido por sí sola (qué proyecto, cuándo, en qué situación), y ese contexto cuesta tokens.

Hay un límite duro impuesto por el modelo. Por debajo de cierto número de tokens, una entrada de memoria deja de tener información útil. El plugin no comprime por debajo de ese umbral.

La conclusión práctica: factor diez no es "memoria infinita". Es "memoria suficientemente densa para que valga la pena cargarla en lugar de ignorarla". Ese es el punto óptimo de la skill; intentar empujar el ratio más allá rompería la utilidad.

Bloque 4 · Dos comportamientos no obvios la primera vez

Comportamiento uno — el agente te corrige sobre algo que no recuerdas haber dicho. Pasa así: arrancas un proyecto nuevo, le pides al agente que escriba una clase, el agente propone una estructura, tú dices "perfecto", y el agente responde "ojo, en proyectos anteriores decidiste que esa estructura no encaja con tu patrón de tests; te propongo X en su lugar". Tu primera reacción es desconcertante: ¿de dónde saca eso? Sale de la inyección de memoria. El plugin cargó la decisión que tú tomaste hace dos meses en otro proyecto y el agente la aplicó. Una vez entiendes la mecánica, deja de ser desconcertante; pasa a ser exactamente el valor que buscabas.

Comportamiento dos — la inyección puede inducir a aplicar algo que ya no aplicas. Las preferencias cambian. Una decisión que tomaste hace seis meses puede haber quedado obsoleta. Si el plugin la inyecta y el agente la aplica sin que tú la corrijas, te encuentras con una pieza nueva siguiendo un patrón viejo que ya no te encaja. El remedio: cuando notes que el agente está aplicando algo que ya no haces, corrígelo explícitamente. El plugin captura la corrección como nueva entrada y, con el tiempo, la decisión nueva pesa más que la vieja al seleccionar qué inyectar. La memoria evoluciona si tú la corriges; si no la corriges, fosiliza.

Bloque 5 · Cierre y enlace a M6.4

La captura es por destilación, no por log verbatim. El plugin guarda decisiones cerradas, patrones aplicados, errores resueltos y convenciones detectadas. Descarta conversación larga, lecturas exploratorias sin cierre, e información sensible. La base no crece descontrolada; crece selectivamente.

La inyección usa el contexto del proyecto + el primer prompt como pistas para seleccionar un puñado relevante de la memoria. El primer prompt importa mucho; trátalo como mini-brief. Hay

un periodo de calentamiento de varias semanas antes de que la inyección se sienta claramente útil.

El factor diez de ahorro de tokens es el punto óptimo entre densidad y legibilidad. No es memoria infinita; es memoria suficientemente densa para valer la pena. Y la memoria evoluciona si tú la corriges; si no lo haces, fosiliza decisiones viejas.

En el siguiente submódulo (M6.4) entramos en lo operativo: instalación con el comando real verificado, verificación de que el plugin está activo, y los dos gotchas más comunes de la primera vez. Cinco minutos para tener claude-mem operativo en tu setup.

Glosario rápido del bloque

- **Destilación** — el método de captura. El plugin no guarda la conversación literal; guarda decisiones, patrones, errores y convenciones identificadas.
- **Inyección** — la entrega al inicio de la sesión nueva. Selección por relevancia usando contexto del proyecto y primer prompt como pistas.
- **Periodo de calentamiento** — las primeras semanas con el plugin activo, antes de que haya base suficiente para que la inyección se sienta claramente útil.
- **Memoria que fosiliza** — el efecto de no corregir decisiones obsoletas. La inyección las seguiría aplicando hasta que las contradigas explícitamente.
- **Factor diez** — ratio típico de ahorro de tokens al cargar contexto histórico con la skill comparado con recargar a mano. Punto óptimo entre densidad y legibilidad.

Qué viene en M6.4

Instalación con el comando real `npx claude-mem install`, verificación operativa de que el plugin está activo, y los dos gotchas más comunes (la base aparece vacía la primera semana y eso es normal; la inyección al inicio de la sesión es invisible si no la buscas). Cinco minutos.

M6.4 · Instalación y configuración de claude-mem

Bloque 0 · Qué cubrimos en cinco minutos

Tres cosas y se cierra. Instalación con el comando real verificado del repo `thedotmack/claude-mem`. Verificación de que el plugin se enganchó al ciclo de vida de tus sesiones. Y los dos gotchas más comunes de la primera semana: la base parece vacía y eso es normal; la inyección al inicio es invisible si no sabes mirar.

Si vas a seguir el módulo en directo, abre tu Claude Code en cualquier proyecto. El plugin se instala una vez y trabaja a partir de ahí en todas tus sesiones.

Bloque 1 · La instalación

claude-mem se instala con un comando único de npm:

```
npx claude-mem install
```

Sin namespace de plugin marketplace, sin `/plugin install`. Es paquete npm que ejecuta su propio instalador. Tres detalles del comportamiento mientras corre:

Crea las bases locales SQLite + Chroma. La primera instalación inicializa los archivos de base en tu directorio de configuración. Vacíos al principio; se llenarán a medida que trabajes con el agente. La ubicación exacta depende de tu sistema; el instalador te la reporta al terminar.

Registra los hooks de ciclo de vida. El plugin queda enganchado a los eventos que necesita (inicio de sesión, cierre de sesión, cambios relevantes durante la sesión). Esto es lo que hace que la captura y la inyección ocurran automáticamente sin que tengas que invocarlas.

Funciona en sesiones siguientes, no en la actual. Si tienes Claude Code abierto cuando ejecutas el comando, los hooks se cargan para la próxima sesión. Cierra y reabre Claude Code después de instalar para que la captura empiece.

Si el comando falla por permisos de npm, mira las opciones razonables del M3.4 (instalar con nvm para tener npm en directorio de usuario, o ejecutar solo esa instalación con permisos elevados). Sin sudo crónico.

Si vienes del paquete erróneamente buscado "cloud-mem" (transcripción del vídeo origen) o de un intento de instalación como MCP server (también error del vídeo), no hay configuración

previa que deshacer; simplemente esos intentos no instalaron nada funcional.

Bloque 2 · Cómo saber que está activo

Que el instalador termine con OK no garantiza que el plugin esté capturando en tu próxima sesión. La comprobación rápida:

Abre Claude Code en un proyecto cualquiera (es indiferente cuál; el plugin trabaja global, no por proyecto como GSD). Trabaja una pieza pequeña: pídele al agente que escriba una función trivial, confírmala, ciérrala.

Cierra la sesión.

Abre una sesión nueva inmediatamente, en el **mismo proyecto**, y pregúntale al agente algo del estilo:

"¿Qué hicimos en la sesión anterior?"

Si claude-mem está activo, el agente responde con un resumen de lo que se trabajó la sesión anterior, sin que tú le hayas pasado nada explícitamente. Ese resumen viene de la base que claude-mem ha capturado entre sesiones. Si en cambio el agente responde *"no tengo contexto de sesiones anteriores"*, el plugin no está enganchado.

Tres causas posibles si la verificación falla:

- **No reiniciaste Claude Code después de instalar.** Los hooks se cargan al inicio del proceso; con sesión vieja siguen sin estar activos.
- **Permisos de escritura en el directorio de configuración.** El plugin no pudo crear las bases SQLite o Chroma. Mira en la salida del instalador si hubo aviso al respecto.
- **Conflicto con otra skill que también engancha hooks de ciclo de vida.** Si tienes otra skill de memoria experimental, puede haber colisión. Desactívala para confirmar.

Cuando la verificación funciona, ya está. El plugin captura desde ese momento; no tienes que hacer nada más en el día a día.

Bloque 3 · Los dos gotchas de la primera semana

Gotcha uno — la base parece vacía y todo se siente igual que antes. La primera semana con claude-mem activo, la inyección al inicio de sesiones nuevas no aporta visiblemente. Tu reacción esperada: *"esto no hace nada"*. La realidad: la base está en periodo de calentamiento.

Necesita acumular suficiente material destilado para que la inyección encuentre algo relevante que devolver. Una semana de uso normal suele bastar; dos semanas garantizan que la inyección empiece a sentirse útil.

Qué hacer: paciencia. No instalar la skill y desinstalarla a los tres días pensando que no funciona. La curva de valor del primer mes es claramente ascendente.

Gotcha dos — la inyección es invisible y crees que no está pasando. Cuando claude-mem inyecta al inicio de una sesión nueva, no te lo dice. No verás un mensaje del estilo *"He cargado tres entradas de tu memoria sobre EF Core."* La memoria aparece como contexto cargado y el agente la aplica de fondo. Tu reacción esperada la primera vez que aparece: *"vaya, este agente está más afilado que normalmente"*. Pero como no asocias eso con el plugin, no te das cuenta de que es lo que está funcionando.

Qué hacer: cuando notes que el agente arranca con criterio sobre algo que no le acabas de explicar, asume que es claude-mem inyectando. Si quieres confirmarlo de forma explícita, la documentación oficial tiene un comando para listar qué se cargó en la sesión actual. No es necesario para el día a día; basta saber que está ahí.

Bloque 4 · Cierre y enlace a M6.5

Cinco minutos, claude-mem instalado, plugin verificado, gotchas de la primera semana en la mochila. En el siguiente submódulo (M6.5) cerramos el módulo M6 con la demo del cambio de ritmo al arrancar un proyecto nuevo. Comparativa con y sin memoria previa cargada: cuánto tiempo se ahorra en la primera semana del proyecto, qué decisiones aparecen aplicadas sin tener que reexplicarlas, y dónde la inyección no llega y tienes que ayudar.

Si has llegado hasta aquí con claude-mem instalado y la prueba de verificación pasada, vas con la pieza lista para la demo del siguiente submódulo.

Glosario rápido del bloque

- `npm claude-mem install` — comando real único de instalación, verificado a fecha de hoy. Paquete npm con instalador propio.
- **Hooks de ciclo de vida** — los eventos a los que el plugin se engancha (inicio, cierre, cambios relevantes). Lo que hace que la captura y la inyección ocurran automáticamente.
- **Periodo de calentamiento** — la primera semana o dos con la skill activa, antes de que haya material destilado suficiente para que la inyección sea claramente útil.

- **Inyección invisible** — el comportamiento por defecto del plugin al inicio de una sesión nueva. No te avisa; carga la memoria como contexto y el agente la aplica de fondo.

Qué viene en M6.5

Demo del cambio de ritmo al arrancar un proyecto nuevo. Misma situación, dos escenarios: agente sin memoria previa cargada y agente con cinco semanas de claude-mem activo en proyectos anteriores. Cuánto tiempo se gana en la primera semana, qué decisiones aparecen aplicadas, y dónde la inyección no llega. Siete minutos que cierran el módulo M6.

M6.5 · Demo — el arranque del mismo proyecto nuevo con y sin claude-mem

Bloque 0 · El experimento

Proyecto nuevo. Sin código previo. La idea: una pequeña API de gestión de incidencias internas para un equipo de soporte. Stack obligatorio: .NET 10, ASP.NET Core, EF Core sobre SQLite, xUnit v3, mismo perfil que los demás demos del curso.

Dos escenarios:

- **Escenario A** — Claude Code limpio. Sin claude-mem activo, o con claude-mem activo pero base totalmente vacía. El agente arranca sin saber nada de tu forma de trabajar.
- **Escenario B** — Mismo Claude Code, con claude-mem activo y cinco semanas de uso previo en otros tres proyectos .NET de tu equipo (los del M2.5, M3.5 y M4.4 que vamos arrastrando en el curso).

Mismo prompt inicial. Misma pieza objetivo (primera semana del proyecto). Mismo nivel de exigencia de calidad. Lo que cambia es solo la presencia o no de memoria capturada de tu trabajo previo.

Bloque 1 · El prompt de arranque

El mismo en los dos escenarios:

"Empezamos un proyecto nuevo. Una API de gestión de incidencias internas para soporte. Stack: .NET 10, ASP.NET Core, EF Core con SQLite, xUnit v3. La primera semana vamos a sacar el modelo de dominio (Ticket , User , Priority , Status), el endpoint de creación de tickets, y los tests de integración."

Vamos a ver qué hace el agente en cada escenario.

Bloque 2 · Escenario A — Sin memoria previa

El agente lee el prompt y propone empezar. Pero como no tiene ni una entrada de memoria sobre tu forma de trabajar, las preguntas y decisiones que aparecen son las del primer día con cualquier

dev nuevo:

Hora 0:10. El agente pregunta: *"¿Prefieres organizar el código por capas (Controllers, Services, Repositories) o por features (Tickets/, Users/)?"*. Tú respondes "features". Decisión cerrada.

Hora 0:25. *"¿Errores esperados como excepciones o como `Result<T>`?"*. Respondes `Result<T>`.

Hora 0:50. *"Los tests, ¿al lado del código o en una carpeta hermana de tests?"*. Respondes "al lado".

Hora 1:30. *"Para el test de integración, ¿SQLite en memoria o en archivo?"*. Aquí tú has aprendido en otros proyectos que en memoria tiene limitaciones con concurrencia, pero el agente lo propone igual. Tú lo corriges: "en archivo, usando el patrón `SqliteFixture` que solemos usar". Decisión cerrada y patrón mencionado por ti.

Hora 2:00. Implementación del endpoint y los tests. El agente propone un middleware de manejo de errores que captura excepciones; tú lo corriges porque va contra la decisión de las 0:25 (`Result<T>`, no excepciones). El agente ajusta.

Hora 3:00 a 5:00. Sigue la implementación. Tres correcciones más sobre decisiones que tú ya tenías cerradas en proyectos anteriores: cómo nombrar las clases de DTOs, qué tipo de logging usar, qué validar en el controller y qué en el servicio.

Día 2 al 5. La semana avanza. Cada tarea nueva trae al menos una decisión que tú ya tenías cerrada en otro proyecto y que tienes que volver a explicar. Para el viernes, la pieza está terminada, pero te has pasado la semana sintiendo que el agente aprendía cosas tuyas que ya debería haber sabido.

Tiempo total de la semana: ocho horas trabajadas (a razón de hora y media por día), de las cuales un cálculo razonable estima **dos horas largas dedicadas a reexplicar decisiones**.

Bloque 3 · Escenario B — Con cinco semanas de claude-mem activo

Reset. Mismo proyecto greenfield, mismo prompt. Lo que cambia: claude-mem lleva cinco semanas capturando en otros proyectos .NET de tu equipo. La base tiene material destilado sobre tus decisiones, patrones, y errores resueltos en proyectos anteriores.

Minuto 0. Lanzas el prompt. Antes de que el agente responda, claude-mem inyecta del fondo el contexto relevante: tus convenciones de proyectos .NET, el patrón `Result<T>` que usas, las

preferencias de testing, el patrón `SqliteFixture` que ya documentaste. Tú no ves esta inyección como mensaje; aparece como contexto cargado.

Minuto 1. El agente responde con una propuesta de arranque que **ya aplica tus convenciones**: organización por features, mención explícita de `Result<T>` para errores esperados, tests al lado del código, y propuesta de usar `SqliteFixture` con archivo para el test de integración. No te pregunta las cuatro cosas que en el escenario A te preguntaba a las 0:10, 0:25, 0:50 y 1:30; las asume aplicando lo que ya tiene en memoria. Te pide confirmación en bloque: *"Voy con este perfil de proyecto. ¿Algo distinto en este caso?"*. Confirmas.

Hora 0:30. Implementación arrancada con tus convenciones desde el primer commit. No hay middleware de excepciones que corregir; el agente lo monta directamente con `Result<T>`.

Hora 2:30. Aparece una decisión nueva genuinamente nueva (algo específico de la API de incidencias que no estaba en proyectos anteriores: política de notificaciones cuando se cierra un ticket). Esta sí es para Brainstorming y la cierras. La memoria captura la decisión nueva.

Día 2 al 5. La semana avanza fluida. Las decisiones cerradas en otros proyectos se aplican; las decisiones nuevas se discuten y cierran en su momento; la memoria se actualiza con las nuevas. Cero correcciones por decisiones que el agente debería haber sabido.

Tiempo total de la semana: cinco horas trabajadas. Pieza terminada con menos esfuerzo y con la sensación de que el agente ya te conocía.

Bloque 4 · La comparativa lado a lado

Dimensión	Sin claude-mem	Con claude-mem
Decisiones cerradas en otros proyectos aplicadas en el arranque	0	6 de 6
Preguntas que el agente hace al inicio	4 grandes + correcciones a lo largo de la semana	1 de confirmación en bloque
Correcciones por decisiones que ya tenías cerradas	~7 a lo largo de la semana	0
Tiempo total de la primera semana	8 horas	5 horas
Sensación operativa	"le tengo que reexplicar todo"	"ya me conocía"
Memoria al final de la semana	nada nuevo capturado	una decisión nueva (notificaciones) añadida a la base

Cuatro datos para llevarte:

El ahorro está donde más cuesta. La primera semana de un proyecto nuevo es la más cara en tiempo y en fricción mental. Cualquier ahorro ahí pesa más que un ahorro equivalente en la quinta semana del proyecto. claude-mem ataca precisamente ese momento.

El ahorro escala con el número de proyectos nuevos. Si arrancas un proyecto nuevo al año, la diferencia es marginal. Si arrancas cinco al año (consultoría, agencia, equipo con muchos clientes), las tres horas ahorradas por proyecto se suman a quince horas anuales. La adopción de claude-mem rinde más cuanto más diverso es tu portfolio.

El periodo de calentamiento es real pero corto. Las cifras del escenario B asumen que claude-mem llevaba cinco semanas capturando. Las primeras dos semanas con la skill activa no rinde así; necesita acumular material. Pero a partir de la tercera semana, cada proyecto nuevo se beneficia del trabajo previo.

La memoria captura del proyecto nuevo también vale para el siguiente. La decisión nueva sobre notificaciones que tomaste en este proyecto pasa a estar en la base. Cuando arranques el séptimo proyecto, esa decisión también se inyectará si es relevante. La memoria es acumulativa; cada proyecto que cierras alimenta los siguientes.

Bloque 5 · Cierre del módulo M6 y enlace al M7

Hemos cerrado el módulo M6.

El problema número cinco de Claude Code es la falta de transferencia entre proyectos. Lo que aprendes en un proyecto no viaja al siguiente sin intervención manual. Los apaños caseros (carpeta global de notas) fallan por la misma razón que en M3: el agente lee sin criterio y la base no se actualiza bidireccionalmente.

claude-mem instala la pieza que arregla el problema. Plugin (no MCP server), almacenamiento local con SQLite + Chroma, búsqueda híbrida, captura por destilación, inyección selectiva al inicio de cada sesión usando el contexto del proyecto y el primer prompt como pistas. Ratio típico de ahorro: factor diez en tokens cargados; ratio práctico en horas: tres horas ahorradas por semana de proyecto nuevo cuando la base lleva varias semanas de uso.

La memoria es acumulativa y evoluciona si tú la corriges. No corregir decisiones obsoletas fosiliza la base; corregir cuando algo deja de aplicar mantiene la inyección fresca. La calidad de la skill no es estática; depende de cómo la cuides.

En el módulo M7 cerramos el curso. Vamos a integrar las seis skills en un flujo combinado completo, ver una demo end-to-end de un proyecto entero (desde el primer prompt hasta el

merge a `main`), y dejarte con la guía para que decidas qué skill personalizada construir como siguiente paso. Cinco submódulos.

Glosario rápido del bloque

- **Decisión cerrada en otro proyecto** — la elección de diseño que tomaste antes y que sin claude-mem te toca volver a tomar en el proyecto nuevo. La unidad de medida útil para evaluar el valor de la memoria entre proyectos.
- **Confirmación en bloque** — la pregunta del agente con claude-mem activo al inicio del proyecto. En lugar de preguntar cuatro cosas separadas, propone un perfil completo y pide validación.
- **Periodo de calentamiento** — las primeras dos semanas con la skill activa, antes de que haya base suficiente para que el escenario B se materialice. Tener esto presente evita desinstalar la skill antes de tiempo.
- **Memoria acumulativa** — el efecto de que cada proyecto que cierras alimenta los siguientes. La curva de valor es ascendente a lo largo de meses.

Qué viene en M7

Cierre del curso. Cinco submódulos: el flujo combinado de las seis skills (M7.1), demo end-to-end de un proyecto entero con todas activas (M7.2), cómo decidir qué skill personalizada construir tú a continuación (M7.3), errores comunes al adoptar el catálogo entero (M7.4), y un cierre con la guía de próximos pasos a tres meses (M7.5).

PARTE 7

Tu flujo y tu próxima skill

M7.1 · El flujo combinado de las seis skills

Bloque 0 · La promesa del submódulo

A lo largo del curso has visto seis skills, una por módulo: Skill Creator (M1), Superpowers (M2), GSD (M3), `/review` y `/ultrareview` (M4), Context Mode (M5), claude-mem (M6). Si las miras como menú, son seis opciones a elegir. Si las miras como sistema, son la respuesta coordinada a cinco problemas distintos de Claude Code en producción.

En este submódulo vamos a fijar tres cosas. Una: cómo se mapea cada skill al problema que resuelve, para que no las confundas y para que sepas cuál te falta cuando aparezca el síntoma. Dos: el orden de adopción que funciona, porque instalar las seis a la vez es peor que ir incorporándolas por capas. Tres: dónde se solapan y dónde no, con honestidad para que no esperes redundancia donde no la hay.

Bloque 1 · Las seis skills mapeadas a los cinco problemas

A lo largo del curso te he insistido en numerar problemas. La razón es esta:

Problema	Síntoma	Skill que lo aborda
1 — La prisa	El agente entrega rápido y mal por defecto	Superpowers (M2)
2 — La sesión que se acaba	Cortes en piezas largas, decisiones perdidas entre sesiones del mismo proyecto	GSD (M3)
3 — La revisión humana no escala	La pila de PRs crece más rápido de lo que el equipo revisa	<code>/review</code> + <code>/ultrareview</code> (M4)
4 — La saturación del contexto	El agente se degrada dentro de la misma sesión	Context Mode (M5)
5 — La memoria que no se transfiere	Cada proyecto nuevo arranca de cero	claude-mem (M6)

Skill Creator (M1) no aparece en la tabla porque no resuelve un problema directo del catálogo; es el meta-skill que te permite construir nuevas piezas para problemas que aparezcan en tu flujo y no estén cubiertos por las cinco anteriores. Es la sexta porque sin las cinco antes no sabrías ni qué te falta para crear.

La tabla anterior es la primera herramienta de diagnóstico. Cuando aparezca un síntoma en tu día a día, mira la columna del síntoma; la skill que toca está enfrente. Y a la inversa: si alguien te pregunta "¿para qué sirve esta skill?", tienes la respuesta en una frase.

Bloque 2 · El orden de adopción recomendado

Aquí va una recomendación clara: **no instales las seis a la vez**. Es peor que ir por capas. La razón: si las metes todas el lunes, no sabes cuál de los cinco cambios que notas el viernes viene de qué skill, y si algo va mal no tienes forma de aislar.

El orden que funciona en la mayoría de devs intermedios:

Semana 1: Superpowers (M2). El cambio más grande de los cinco. Cambia el comportamiento por defecto del agente. Vas a sentir fricción los primeros tres días (M2.1) y velocidad real a partir del cuarto. No metas nada más esa semana.

Semana 2: GSD desde el fork (M3). Una vez Superpowers está integrado y trabajas con piezas más estructuradas, GSD captura las decisiones y el progreso de esas piezas. Tiene sentido en este orden: el flujo de Superpowers produce material denso que GSD organiza bien.

Semana 3: `/review` integrado en tu hábito (M4). No hace falta instalación porque es comando integrado. Lo que hace falta es convertir `/review` en reflejo antes de pedir code review humano. Una semana de hábito basta para fijarlo.

Semana 4: Context Mode (M5). Cuando llevas tres semanas con piezas largas y has notado al menos una vez la degradación dentro de una sesión, Context Mode tiene sentido. Antes, no urge.

Semana 5 en adelante: claude-mem (M6). La última de las cinco operativas porque su valor depende de la base capturada. Cuanto antes la instales, antes empieza a acumular, pero su rendimiento solo se nota a partir de la tercera semana de captura. Si la instalas la semana 5 del curso, en la semana 8 empieza a brillar.

Cuando notes el primer hueco: Skill Creator (M1). No la instales hasta que tengas un problema concreto del que decir "esto no lo cubre ninguna de las cinco". Entonces M1 te ayuda a construir la solución.

Esta cadencia no es dogma; es la que funciona para la mayoría. Si tu situación es peculiar (por ejemplo, ya tienes equipo grande con cuello en code review desde el primer día), puedes mover el orden. Lo importante es no instalar las seis a la vez.

Bloque 3 · Cómo encajan en un día de trabajo típico

Un día normal con las seis skills activas se siente así, ordenadas por momento:

Al abrir Claude Code en un proyecto. claude-mem inyecta del fondo el contexto relevante de tus proyectos anteriores. Si el proyecto tiene GSD activo, también se cargan los cinco archivos (`PROJECT.md` , `REQUIREMENTS.md` , `ROADMAP.md` , `STATE.md` , `CONTEXT.md`). El agente arranca con criterio doble: el de tu trabajo histórico (M6) y el de la pieza concreta en este proyecto (M3).

Al empezar una pieza nueva. Superpowers entra: Brainstorming hace preguntas hasta cerrar el diseño, Writing-Plans descompone, Subagent-Driven-Development ejecuta con TDD y revisión en dos fases. GSD captura las decisiones cerradas y el plan en sus archivos.

A media tarde, si la sesión llega al 40% de contexto. Context Mode entra con `/contextmode:compress` . El contexto se densifica; la sesión sigue afilada hasta el final.

Al cerrar la pieza, antes de abrir PR. `/review` se lanza como reflejo. Si la pieza toca código sensible o no va a haber revisión humana después, `/ultrareview` en su lugar. La revisión asistida saca lo gordo; tú o un colega revisa lo sutil.

Al final del día, al cerrar la sesión. GSD reescribe el handoff (`STATE.md`) para la siguiente sesión. claude-mem captura las decisiones nuevas, los patrones aplicados, y los errores resueltos del día.

Cuando aparece un problema que ninguna skill cubre. Skill Creator (M1) entra: defines la skill personalizada que necesitas, la testeas, la incorporas al flujo. La sexta es la que escala el sistema en el tiempo.

Bloque 4 · Solapamientos honestos

Hay tres pares de skills que pueden parecer redundantes y merece aclararlos:

M3 (GSD) y M6 (claude-mem). Las dos hacen "memoria". La diferencia: GSD captura **dentro de un proyecto** entre sesiones del mismo proyecto. claude-mem captura **entre proyectos** distintos. Si solo tienes uno, GSD basta. Si trabajas con varios, las dos. Cuando trabajas con las dos, no se pisan; cada una cubre su eje.

M3 (GSD) y M5 (Context Mode). Las dos pueden parecer "gestión de contexto". La diferencia: GSD organiza el estado del proyecto en artefactos persistentes que el agente lee al inicio. Context Mode comprime el contexto vivo de una sesión activa. Una persiste, la otra comprime. Trabajan en momentos distintos y no se solapan operativamente.

M5 (Context Mode) y M6 (claude-mem). Las dos pueden parecer "ahorro de tokens". La diferencia: Context Mode ahorra dentro de la sesión activa al densificar lo que ya se vio. claude-

mem ahorra al inicio de la sesión nueva al cargar lo relevante en lugar de obligarte a recargar manualmente. Una ahorra contra la sesión vieja, la otra contra el contexto histórico.

El que sí queda sin par directo es `/review` y `/ultrareview` (M4): no hay otra skill del catálogo que haga revisión asistida. Es pieza única en su rol.

Bloque 5 · Cierre y enlace a M7.2

Las seis skills no son menú; son sistema. Cinco resuelven cinco problemas distintos y nombrados (la prisa, la sesión que se acaba, la revisión humana, la saturación del contexto, la memoria que no se transfiere). La sexta (Skill Creator) es el meta-skill que escala el sistema cuando aparece un problema sin cubrir.

El orden de adopción importa. Una skill por semana, empezando por Superpowers, terminando por claude-mem (las operativas), y Skill Creator cuando aparece el primer hueco. Instalar las seis a la vez es peor que ir por capas.

Los solapamientos aparentes (memoria, contexto, ahorro de tokens) son explicables y no son redundancias. Cada skill ataca un eje específico que las demás no cubren. Trabajan juntas sin pisarse.

En el siguiente submódulo (M7.2) entramos en la demo end-to-end: un proyecto real desde el primer prompt hasta el merge a `main`, con las seis skills activas. Diez minutos para ver el flujo combinado en vivo sobre la misma pieza .NET que vamos arrastrando en el curso.

Glosario rápido del bloque

- **Pila coordinada** — la forma de mirar las seis skills cuando entiendes que se complementan en lugar de duplicarse. Diferencia operativa frente a "menú de seis opciones".
- **Orden de adopción** — la secuencia recomendada por semana para incorporar las skills sin perder el control. Superpowers primero, Skill Creator al final cuando aparece el primer hueco.
- **Solapamiento aparente** — los pares de skills que parecen redundantes (M3 y M6 por memoria; M3 y M5 por contexto; M5 y M6 por ahorro). Aclarado en el bloque 4 para que no las descartes pensando que sobran.
- **Meta-skill** — Skill Creator (M1) en la lectura del módulo final. La que te permite construir piezas nuevas cuando las cinco operativas no llegan.

Qué viene en M7.2

Demo end-to-end. Pieza real (cierre de un sprint en el proyecto .NET de incidencias del M6.5) con las seis skills activas a lo largo del proceso: desde el primer prompt de la mañana del lunes hasta el merge a `main` del viernes. Diez minutos donde vas a ver dónde entra cada skill en vivo.

M7.2 · Demo end-to-end — un sprint con las seis skills activas

Bloque 0 · El experimento

Proyecto: la API de incidencias internas del M6.5 (.NET 10, ASP.NET Core, EF Core sobre SQLite, xUnit v3). Lleva ya tres semanas en marcha; el modelo de dominio y los endpoints básicos están terminados. La pieza de esta semana: añadir un sistema de notificaciones que avise a los usuarios cuando un ticket cambia de estado.

Configuración de partida:

- **Superpowers**: activo desde la semana 1 del proyecto.
- **GSD** (fork redux): activo desde la semana 1; los cinco archivos están al día.
- **/review**: hábito instalado, reflejo automático antes de PR.
- **Context Mode**: activo, regla personal de comprimir al llegar al 40%.
- **claude-mem**: lleva ocho semanas capturando en cuatro proyectos .NET del equipo. Base con material denso.
- **Skill Creator**: no se invoca esta semana; no hay hueco nuevo que cubrir.

Cinco días de trabajo, hora y media por día. Vamos a verlo día a día.

Bloque 1 · Lunes — Arranque de la pieza

09:00. Abres Claude Code en la carpeta del proyecto. claude-mem inyecta del fondo lo relevante: tu convención de `Result<T>` para errores, el patrón `SqliteFixture` para tests de concurrencia, decisiones de proyectos anteriores sobre cómo manejar notificaciones (en un proyecto anterior decidiste usar MediatR con outbox; eso entra como contexto). GSD carga los cinco archivos del proyecto: el `STATE.md` te dice que el último ticket pendiente del sprint anterior quedó cerrado el viernes, y el plan vivo (`ROADMAP.md`) ya tiene un hueco reservado para la pieza de notificaciones.

09:05. Lanzas el prompt: *"Vamos con notificaciones. Cuando un ticket cambia de estado, notificar al usuario asignado por email."*

Superpowers entra. Brainstorming hace preguntas con criterio ya orientado: *"Veo que en `CONTEXT.md` tienes la decisión D-007 de usar MediatR con outbox para eventos de dominio. ¿Aplicamos el mismo patrón aquí, o esta pieza requiere algo distinto? Y tres preguntas más concretas: ¿quién es el remitente del email? ¿La plantilla viene de fuera o la generamos? ¿Y si la notificación falla, reintentamos o solo loggeamos?"*.

Respondes las cuatro preguntas en bloque. Brainstorming guarda el documento de diseño.

09:15. Using-Git-Worktrees crea la rama `feature/ticket-notifications` en worktree aislado. Baseline de tests en verde, confirmado.

09:20. Writing-Plans descompone en dieciocho micro-tareas de 2-5 minutos cada una. Las apruebas.

09:25 - 10:30. Subagent-Driven-Development ejecuta. Subagentes frescos por tarea, TDD red-verde, revisión en dos fases tras cada tarea. Cuando termina, ocho tareas completas: el handler base, el sender abstracto, el adaptador SMTP, los tres tests unitarios principales.

10:30. Cierras el día con la pieza al 45%. GSD reescribe `STATE.md` con pendiente inmediato para mañana. claude-mem captura las decisiones cerradas del día.

Bloque 2 · Martes — La saturación del contexto y la primera comprensión

09:00. Abres sesión nueva. GSD carga el estado; el agente sabe exactamente por dónde retomar. Brainstorming no entra al inicio porque la pieza ya estaba en plan; pasa directo a la siguiente micro-tarea.

09:00 - 10:30. Cinco micro-tareas más completadas. Adaptador para SMS (segundo canal), tests de integración del handler completo, persistencia de la notificación enviada en la outbox.

10:30. Lanzas `/contextmode:status` por reflejo. Resultado: el contexto está al 38%. Decides comprimir para tener la tarde holgada.

10:31. `/contextmode:compress`. La skill devuelve: *"Antes 76.000 tokens, después 4.200. Ratio 18x. Información operativa preservada: estructura del handler, decisión D-018 sobre outbox, estado de tests."*

10:32 - 12:00. Tres micro-tareas más con el contexto fresco. Adaptador para push notification (tercer canal), test de integración multi-canal, manejo de fallo parcial cuando un canal cae y los otros no.

12:00. Cierras. Pieza al 70%. GSD actualiza, claude-mem captura.

Bloque 3 · Miércoles — La decisión nueva y el documento que se va al

`CONTEXT.md`

09:00. Sesión nueva. Pendiente inmediato: documentar el comportamiento de fallo parcial cuando un canal falla y los otros pasan.

09:30. Surge una decisión nueva que no estaba prevista. El equipo de producto pregunta si el usuario debería poder elegir qué canales reciben notificaciones (preferencias por usuario) o si todos los canales activos notifican siempre. Decisión de producto, no técnica, pero afecta al diseño.

Brainstorming entra para discutir las dos opciones, con sus implicaciones. Decides: preferencias por usuario, con default "todos los canales". GSD añade la decisión nueva al `CONTEXT.md` (D-019). claude-mem la captura para futuros proyectos.

09:45 - 11:00. Implementación de preferencias de usuario. Cuatro micro-tareas más. Tests pasan.

11:00 - 12:00. Documentación del fallo parcial y de las preferencias en el README del módulo.

12:00. Pieza al 90%. Cierras.

Bloque 4 · Jueves — Revisión asistida y los hallazgos consolidados

09:00. Sesión nueva. Pendiente inmediato del `STATE.md`: cerrar las dos micro-tareas finales (test end-to-end y limpieza de comentarios) y abrir PR.

09:00 - 10:00. Cerradas las dos tareas finales.

10:00. Reflejo de cierre de rama: `/review`. Tarda dos minutos. Devuelve siete hallazgos:

- Dos bugs sospechados (uno crítico: el handler de SMS no maneja el caso de número de teléfono nulo; otro medio: el contador de notificaciones enviadas no es transaccional con la outbox).
- Dos cuestiones de diseño (acoplamiento más fuerte de lo necesario con `MailKit`; el adaptador SMTP no implementa una abstracción que facilitaría mock en tests).
- Una inconsistencia con proyecto (un método devuelve `Task<bool>` cuando el resto del proyecto usa `Result<T>`).
- Dos nits menores.

10:15. Como la pieza toca notificaciones que pueden afectar a clientes reales, decides hacer también `/ultrareview`. Tarda once minutos.

10:30. `/ultrareview` confirma los siete hallazgos de `/review` con consenso entre revisores. Añade un hallazgo extra de la perspectiva *security*: la plantilla de email acepta interpolación de

campos del ticket; si el campo `description` viene con HTML del cliente y el sender no escapa, puede haber XSS por email. Se marca como crítico.

10:30 - 11:30. Aplicas los fixes. El bug crítico de SMS, el del contador, el de XSS por email, el del `Result<T>`. Los demás (cuestiones de diseño y nits) los dejas para el code review humano del viernes.

11:30. Lanzas `/review` otra vez para confirmar que los fixes están limpios. Solo quedan los hallazgos opinables. Listo para PR.

11:35. Abres PR con descripción auto-generada que enlaza al documento de diseño de Brainstorming, al plan de Writing-Plans, y a las decisiones D-007, D-018, D-019.

12:00. Cierras. claude-mem captura los hallazgos extra que aparecieron y cómo se resolvieron, especialmente el patrón de XSS por email (vale para futuros proyectos que envíen emails).

Bloque 5 · Viernes — Code review humano y merge

09:00. Te llega un comentario en el PR: un colega te pide un par de ajustes en las cuestiones de diseño que dejaste pendientes. Aplicas. Lanzas `/review` para verificar.

10:00. El colega aprueba. Merge a `main`.

10:30. GSD reescribe `STATE.md` con la pieza marcada como completada en `ROADMAP.md` y `CONTEXT.md` con las tres decisiones nuevas. claude-mem captura el cierre.

11:00. Pieza siguiente del sprint. Brainstorming arranca con el ciclo de la semana siguiente.

Bloque 6 · La comparativa contra el escenario sin skills

Dimensión	Sin skills	Con las seis skills
Tiempo total de la semana	~12-14 horas reales (con interrupciones por context rot, decisiones reabiertas, fixes posteriores)	~7.5 horas reales
Bugs cazados antes de PR	aleatorio según fatiga	8 hallazgos, 4 críticos arreglados
Decisiones nuevas documentadas	en la cabeza, perdidas a las tres semanas	3 entradas en <code>CONTEXT.md</code> , capturadas también por claude-mem
Estado al final del viernes	"creo que está bien, veremos"	"merged, documentado, base actualizada"
Material aprovechable para próximo proyecto	el código nuevo; nada más	el código + decisiones + patrón XSS-por-email + base de claude-mem actualizada

Cuatro datos para llevarte.

La ganancia mayor no es de tiempo, es de calidad sostenida. Las seis skills no te hacen escribir más código; te hacen que el código que sale sea más fiable y que las decisiones queden documentadas. Eso compone a lo largo de meses.

El flujo combinado se siente conversacional cuando lo integras. Las primeras semanas con todo activo cuesta acordarse de los reflejos (`/contextmode:status`, `/review`, el repaso del `STATE.md` al cerrar). A las cuatro semanas, lo haces sin pensar.

Los huecos donde aplicar Skill Creator se hacen visibles cuando el resto está integrado. Hasta que no tienes flujo, no ves dónde hace falta una pieza nueva. Cuando lo tienes, los huecos saltan y entonces M1 entra con criterio.

El sistema completo es ortogonal a tu lenguaje y stack. Esta demo es en .NET 10, pero la mecánica del flujo es la misma en Node, Python, Go, o cualquier otro stack. La única diferencia son los ejemplos concretos.

Bloque 7 · Cierre y enlace a M7.3

La demo end-to-end deja ver que las seis skills no son seis skills; son una sola pila operativa. Cada una entra en su momento, hace su trabajo, y deja paso a la siguiente. El día se siente conversacional aunque hay seis sistemas trabajando en segundo plano.

La ganancia de tiempo importa pero la ganancia mayor es de calidad sostenida y de material que vale para piezas futuras. La base de claude-mem, los `CONTEXT.md` de GSD, y los patrones que aparecen en `/ultrareview` se acumulan. Cada semana añade material a la base que sirve para la siguiente.

Los huecos donde Skill Creator entra solo aparecen cuando el resto está integrado. En el siguiente submódulo (M7.3) entramos en cómo decidir qué skill personalizada construir tú a continuación. No es Skill Creator otra vez; es la guía para que decidas qué falta en tu flujo y merece construirse.

Glosario rápido del bloque

- **Pila operativa** — las seis skills funcionando coordinadas en una sesión real. Diferencia operativa frente a "seis skills sueltas".
- **Reflejo de skill** — el hábito de invocar `/contextmode:status` o `/review` en momentos clave sin tener que decidir conscientemente. Se interioriza en las primeras semanas.
- **Material aprovechable** — lo que queda al final de una pieza que vale para piezas futuras. Decisiones documentadas, patrones detectados, errores resueltos capturados.
- **Calidad sostenida** — la métrica honesta del flujo combinado a medio plazo. Más fiable que "tiempo ahorrado en una pieza".

Qué viene en M7.3

Cómo decidir qué skill personalizada construir tú a continuación. No es Skill Creator otra vez; es la guía para identificar el hueco en tu flujo que ninguna de las seis cubre, validar que merece una skill propia, y planificar su construcción. Siete minutos.

M7.3 · Cómo decidir qué skill personalizada construir tú a continuación

Bloque 0 · Por qué este submódulo no es "vuelve a M1"

En M1 vimos qué es Skill Creator y cómo se usa. La tentación al terminar M1 era buena: "ya sé crear skills, ¿qué construyo?". Aquí está la honestidad del curso: **lo que construir no se decide en M1**. Se decide después de varias semanas usando las cinco operativas y observando dónde el flujo combinado del M7.2 no llega.

Construir skills propias antes de tener flujo es la receta para esfuerzo desperdiciado. Construyes algo que parece útil un viernes por la tarde y descubres dos semanas después que cubre un problema que ya cubría Superpowers o que no aplica a tu día a día real. Skill Creator es herramienta poderosa; la utilidad de lo que crees depende de la calidad del problema que ataques.

En este submódulo:

- Cómo identificar un hueco real en tu flujo combinado.
 - Tres preguntas para validar que el hueco merece skill propia y no una mejora del prompt.
 - El método de arranque de la skill nueva en cinco pasos.
 - Los dos errores más comunes al construir tu primera skill personalizada.
-

Bloque 1 · Cómo identificar un hueco real

Un hueco real cumple tres condiciones simultáneas. Si falta alguna, probablemente no es hueco; es algo que el flujo ya cubre o que no merece la inversión.

Condición uno: es un patrón que se repite, no una situación puntual. Si te ha pasado una vez, no es hueco; es la primera vez. Si te ha pasado cinco veces en un mes con casuísticas distintas pero la misma raíz, ahí hay patrón.

Cómo detectarlo: lleva una nota mental (o explícita) durante las primeras cuatro semanas con el flujo combinado activo de los momentos donde te encuentras pensando "*esto debería estar automatizado*". Cuando una misma idea aparece tres veces en tu nota, candidato a skill.

Condición dos: ninguna de las cinco operativas lo cubre. Antes de pensar en skill propia, pasa por el filtro de las cinco:

- ¿Es un problema de procedimiento del agente? → Superpowers ya lo cubre.
- ¿Es un problema de continuidad dentro del proyecto? → GSD ya lo cubre.
- ¿Es un problema de revisión? → `/review` o `/ultrareview` ya lo cubren.
- ¿Es un problema de contexto saturado? → Context Mode ya lo cubre.
- ¿Es un problema de memoria entre proyectos? → claude-mem ya lo cubre.

Si la respuesta a las cinco es "no exactamente", tienes un hueco.

Condición tres: es resoluble con prompt + lógica, no requiere infraestructura externa. Skill Creator construye skills de Claude Code; no construye servicios externos, ni MCP servers complejos, ni integraciones que requieran arquitectura propia. Si tu hueco necesita una API externa con autenticación delegada, persistencia compleja o coordinación distribuida, lo que necesitas no es skill; es servicio. Una skill cubre lo que se resuelve dentro del propio agente con un protocolo bien definido.

Cuando las tres condiciones se cumplen, tienes hueco real y candidato a skill propia.

Bloque 2 · Tres preguntas para validar el candidato

Antes de invertir el día en construir la skill, tres preguntas que te ahorran tiempo si la respuesta a alguna no encaja.

Pregunta uno: ¿se resuelve con tres líneas de prompt mejor escritas?. Muchas necesidades aparentes son problemas de redacción de prompt. Si la solución a tu hueco es "le digo al agente con más detalle qué hacer", no es skill; es mejor prompt. Una skill aporta cuando hay procedimiento estructurado, no cuando lo que falta es claridad textual.

Cómo distinguir: si puedes escribir un único prompt de cinco líneas que resuelva el problema y se lo puedes pasar a cualquier dev del equipo sin perder valor, no necesitas skill. Si el problema requiere múltiples pasos coordinados, captura de estado, o hooks de ciclo de vida, es skill.

Pregunta dos: ¿va a usarlo más gente que tú?. Una skill que solo tú usas tiene un valor distinto a una que va a usar el equipo. No es que la individual sea mala (la mayoría empieza así); es que la valoración del coste de construcción es distinta. Skill solo para ti: invierte solo si la usarás más de dos veces por semana. Skill para tu equipo: invierte cuando al menos dos personas más la pidan.

Pregunta tres: ¿la solución es estable en el tiempo?. Si lo que estás resolviendo cambia cada dos meses (porque depende de una herramienta externa en versión alfa, de una API en cambio constante, o de un patrón que aún no está claro), la skill va a quedarse obsoleta rápido.

Las buenas skills capturan patrones estables; las malas codifican preferencias temporales que cambian antes de que la skill rinda.

Si las tres respuestas son "sí, esto pasa por skill" / "sí, varias personas la usarán" / "sí, lo que codifica es estable", el candidato vale. Si alguna es "no claramente", reconsidera.

Bloque 3 · Método de arranque en cinco pasos

Una vez decidido que la skill se construye, el método para arrancar bien:

Paso uno: nombra la skill con su responsabilidad en una frase. Antes de tocar Skill Creator, escribe en un párrafo qué hace la skill, cuándo se invoca, y qué deja al terminar. Si no puedes escribir ese párrafo en cinco minutos, la responsabilidad no está clara y la skill saldrá borrosa.

Paso dos: identifica los inputs y outputs explícitos. Una skill bien diseñada tiene contrato: recibe X, devuelve Y. Si los inputs son vagos ("contexto de la sesión") y los outputs también ("sugerencias"), la skill será impredecible. Si son concretos ("recibe el path del PR y la rama base; devuelve lista de cuestiones por categoría"), la skill será testeable.

Paso tres: invoca Skill Creator en modo Create. Aplica lo aprendido en M1.3. Le pasas el párrafo del paso uno y el contrato del paso dos. Skill Creator genera el esqueleto.

Paso cuatro: define al menos cinco casos de prueba antes de probar la skill en real. Casos concretos donde la skill debería entregar resultados claros. Esto es la red de seguridad: si la skill pasa los cinco casos, probablemente funciona; si no, sabes qué ajustar antes de meterla en tu flujo real.

Paso cinco: úsala una semana antes de iterar. La tentación es ajustar la skill después de la primera invocación que no convence. Aguanta. Una semana de uso te da datos reales de cuándo encaja y cuándo no. A partir de ahí, Skill Creator en modo Improve toma los datos y propone mejoras concretas.

Bloque 4 · Los dos errores más comunes al construir tu primera skill

Error uno — construir una skill que duplica algo que ya hace una de las seis. La mayoría de huecos aparentes la primera vez son ilusiones. El dev intermedio nuevo en el catálogo cree haber detectado un hueco y resulta que era cobertura existente de Superpowers o de GSD que aún no había experimentado a fondo. Antes de construir tu primera skill, pasa el candidato por el filtro del bloque 1 con honestidad. Si dudas, pregunta a alguien que conozca bien las seis.

Error dos — construir una skill muy específica de un proyecto que termina sin viajar. Una skill diseñada para una situación que solo aparece en tu proyecto actual difícilmente vale en otros proyectos. Para que la skill rinda, su responsabilidad debe ser abstracta: "genera notas de release a partir de los commits de la rama" vale; "genera notas de release con el formato exacto que usa el equipo de marketing de mi empresa en su intranet" no vale en otros proyectos. Diseña por nivel de abstracción razonable.

Bloque 5 · Cierre y enlace a M7.4

Construir skills personalizadas no es paso siguiente automático tras instalar las seis; es decisión que se toma cuando aparece un hueco real y validado en tu flujo combinado. Construir antes es esfuerzo desperdiciado en la mayoría de casos.

Un hueco real cumple tres condiciones: es patrón repetido, no lo cubre ninguna de las cinco operativas, y se resuelve con prompt + lógica sin requerir infraestructura externa. Las tres preguntas de validación (¿se resuelve con prompt? ¿lo usará más gente? ¿es estable?) afinan el candidato antes de invertir.

El método de arranque tiene cinco pasos cortos: nombrar la responsabilidad, definir contrato, generar esqueleto con Skill Creator, escribir cinco casos de prueba, usar una semana antes de iterar. Los dos errores comunes son construir duplicados de las seis y construir cosas demasiado específicas para que viajen.

En el siguiente submódulo (M7.4) vamos a los errores comunes al adoptar el catálogo entero. No los errores individuales de cada skill (ya los cubrimos en cada módulo); los errores que aparecen cuando intentas adoptar las seis al mismo tiempo. Cinco minutos.

Glosario rápido del bloque

- **Hueco real** — situación recurrente en tu flujo que ninguna de las cinco operativas cubre y se resuelve con prompt + lógica. La unidad de decisión para construir skills personalizadas.
- **Tres condiciones del hueco** — patrón repetido, no cubierto por las cinco, resoluble con prompt + lógica. Si falta alguna, probablemente no es hueco.
- **Contrato de skill** — la definición explícita de inputs y outputs antes de construir. La diferencia entre skill predecible y skill borrosa.
- **Casos de prueba** — los cinco escenarios concretos donde la skill debe entregar resultado claro. Red de seguridad antes de meterla en tu flujo real.

- **Semana antes de iterar** — la regla anti-impulso. Aguantar el uso una semana antes de ajustar evita las iteraciones reactivas que llevan a ningún sitio.

Qué viene en M7.4

Errores comunes al adoptar el catálogo entero. No los errores específicos de cada skill; los del flujo combinado. Cinco minutos con los cuatro patrones de mal uso más frecuentes y cómo corregirlos.

M7.4 · Errores comunes al adoptar el catálogo entero

Bloque 0 · La promesa del submódulo

A lo largo del curso te he insistido en una cosa: las skills son herramientas y las herramientas se interiorizan con uso, no con instalación. Este submódulo recoge los cuatro errores más frecuentes que veo en devs intermedios cuando intentan adoptar el catálogo entero después de un curso como este.

No son errores graves. No te van a romper nada. Son errores que te restan el valor que la adopción ordenada te daba. Si conoces los cuatro antes de empezar la adopción de verdad, los esquivas y la curva de aprendizaje se hace más corta.

Bloque 1 · Error uno — Instalarlas todas el mismo día

El error más común y el más natural: terminas el curso, te apetece probar todo, y el sábado por la mañana instalas las seis. El lunes tu Claude Code se siente raro, no sabes qué viene de qué skill, y a la semana siguiente desinstalas tres porque no las entiendes.

Por qué pasa: cada skill cambia algo en el comportamiento del agente o en tu flujo de trabajo. Si cambian seis cosas a la vez, no puedes aislar cuál de los cinco efectos que notas viene de cuál.

Cómo evitarlo: el orden de adopción del M7.1. Una skill por semana. Empieza por Superpowers. Cuando lleves tres días con ella y notes que se siente fluido, mete la siguiente. Si no se siente fluido todavía, espera una semana más. La adopción no tiene fecha límite; lo único que arruina la curva es la prisa.

Bloque 2 · Error dos — Abandonar tras la primera fricción

Cada una de las cinco operativas tiene un periodo de calentamiento. Superpowers se siente más lento los primeros tres días. Context Mode requiere que recuerdes lanzar el comando hasta que se vuelve reflejo. claude-mem no aporta visiblemente las primeras dos semanas. GSD pide ceremonial que al principio molesta.

El error: probar una skill, sentir la fricción del periodo de calentamiento, asumir que la skill "no funciona", y desinstalarla.

Por qué pasa: la curva de valor de las skills no es lineal. Las dos primeras semanas el coste sube (aprender la mecánica, integrar el reflejo) sin que el valor sea aún claramente visible. A partir de la tercera semana, el valor sube rápido. Si abandonas antes de la tercera semana, te quedas con todo el coste y nada del valor.

Cómo evitarlo: compromiso explícito de tres semanas de prueba por cada skill. Si después de tres semanas sigue sin aportar, desinstala. Si después de tres semanas notas que cuando la apagas un día echas en falta algo, ya es parte de tu flujo.

Bloque 3 · Error tres — Perseguir las cifras y olvidar el procedimiento

Una vez instaladas las skills, aparecen métricas visibles: el ratio de compresión de Context Mode, los hallazgos de `/ultrareview`, las estrellas de los repos, el factor diez de ahorro de claude-mem. La tentación es valorar la adopción por las métricas.

El error: pasar de usar las skills a optimizar las métricas que las skills generan. Persigues el ratio de compresión más alto en lugar de comprimir cuando hace falta. Persigues más hallazgos en `/ultrareview` en lugar de aplicar los que importan. Hablas en el equipo de las estrellas en lugar de del procedimiento.

Por qué pasa: las métricas son visibles y comparables; el procedimiento es invisible y subjetivo. Es natural deslizar la atención hacia lo medible. Pero las skills no se adoptaron por las métricas; se adoptaron por el procedimiento. Las métricas son síntoma, no causa.

Cómo evitarlo: cuando hables del catálogo con alguien (o cuando lo pienses para ti), describe **qué cambió en cómo trabajas**, no qué cambió en los números. Si solo puedes describir números, el procedimiento aún no está integrado.

Bloque 4 · Error cuatro — Querer construir skill propia antes de tiempo

El M7.3 te dejaba el método para decidir qué skill construir. El error que ataca este bloque es saltarse el método y construir antes de tiempo, normalmente la primera o segunda semana del curso.

Por qué pasa: Skill Creator (M1) es la primera del catálogo y la única que invita explícitamente a "crear cosas tú". El reflejo natural al terminar M1 es "voy a crear algo". Si lo haces antes de tener flujo combinado, lo más probable es que construyas duplicado de algo que cubre Superpowers o crees algo demasiado específico para que viaje.

Cómo evitarlo: la regla del M7.3. No construyes skill propia hasta que llevas un mes con las cinco operativas integradas y has identificado un hueco que cumple las tres condiciones (patrón repetido, no cubierto por las cinco, resoluble con prompt + lógica). Antes de ese momento, lo que necesitas es interiorizar lo que ya hay, no añadir lo nuevo.

Bloque 5 · Cierre y enlace a M7.5

Los cuatro errores comunes no son técnicos; son de impaciencia. Instalar todo a la vez, abandonar tras la primera fricción, perseguir cifras en lugar de procedimiento, construir skill propia antes de tener flujo. Conocerlos por adelantado los esquiva.

La curva de valor de las skills no es lineal. Las dos primeras semanas el coste sube sin que el valor sea visible. A partir de la tercera semana, el valor sube rápido. Comprometerse a tres semanas por skill es la diferencia entre adoptar el catálogo y descartarlo prematuramente.

El procedimiento es el objetivo, no las métricas. Cuando hables del catálogo (a ti mismo o a otros), describe qué cambió en cómo trabajas, no qué cambió en los números. Si solo puedes describir números, el procedimiento aún no está integrado.

En el siguiente submódulo (M7.5) cerramos el curso entero con la guía de próximos pasos a tres meses: qué hacer la primera semana después de terminar el vídeo, qué a las cuatro semanas, qué a los tres meses. Y un cierre honesto sobre lo que el curso no ha cubierto y dónde mirar a continuación.

Glosario rápido del bloque

- **Periodo de calentamiento** — el tiempo necesario para que una skill empiece a aportar valor visible. Varía por skill (de tres días en Superpowers a tres semanas en claude-mem) pero existe en todas.
- **Curva de valor no lineal** — la realidad operativa de la adopción: el coste se paga al principio, el valor llega más tarde. Abandonar antes de que llegue es desperdiciar el coste pagado.
- **Métrica como síntoma** — los números que las skills generan son consecuencia del procedimiento, no causa. Optimizar el número directamente desvía la atención del cambio real.
- **Hueco real validado** — el único motivo legítimo para construir skill propia. Definido en M7.3 con sus tres condiciones.

Qué viene en M7.5

Cierre del curso. Guía de próximos pasos en tres horizontes: la primera semana después de este vídeo, las primeras cuatro semanas, los primeros tres meses. Qué hacer en cada uno y qué dejar para después. Y un apartado honesto sobre lo que TOPSKL no cubre y dónde mirar para seguir aprendiendo.

M7.5 · Guía de próximos pasos a tres meses

Bloque 0 · Lo que ha cubierto el curso

Antes del cierre, una mirada hacia atrás breve. En estas cuatro horas y media has visto:

- **El criterio de selección** (M0): cuatro filtros para evaluar cualquier skill antes de incorporarla, la distinción entre skills generales y personalizadas, y los avisos honestos sobre el ecosistema.
- **Las cinco skills operativas** del catálogo (M1 a M6): Skill Creator, Superpowers, GSD desde el fork, `/review` con `/ultrareview`, Context Mode, y claude-mem. Cada una con su problema mapeado, su mecánica explicada, su instalación verificada, y su demo.
- **El flujo combinado** (M7.1 y M7.2): cómo encajan las seis en una semana de trabajo real.
- **La guía de extensión** (M7.3): cómo decidir qué skill personalizada construir tú a continuación.
- **Los errores comunes** (M7.4): qué patrones evitar al adoptar el catálogo.

Lo que cierra el curso es lo que viene ahora: qué hacer cuando termine este vídeo. Los tres horizontes a continuación son guía operativa, no inspiración.

Bloque 1 · La primera semana

Días uno a tres. No instales nada. Repasa los módulos M0 y M2 si quieres tenerlos frescos, y observa tu propio uso actual de Claude Code en estos tres días. Apunta dos o tres momentos donde sentiste alguno de los cinco problemas nombrados en el curso: la prisa, la sesión que se acaba, el cuello del code review, la saturación del contexto, la falta de memoria entre proyectos. Tener tus síntomas concretos en la mano hace que las skills tengan sentido cuando las instales.

Día cuatro. Instala Superpowers con el comando del M2.4. Es el cambio más grande del catálogo y el que más vale interiorizar primero.

Días cinco a siete. Trabaja con Superpowers en una pieza real. Acepta la fricción de los primeros días (M2.1). No instales nada más esta semana.

Al final de la primera semana habrás interiorizado el procedimiento básico. Es la base sobre la que descansará todo lo demás.

Bloque 2 · Las primeras cuatro semanas

Semana dos: instala GSD desde el fork (M3.4). Trabaja un proyecto entero con Superpowers + GSD. Familiarízate con los cinco archivos del fork. Al final de la semana, tu sesión nueva en el mismo proyecto debería retomar sin que tengas que explicar nada.

Semana tres: integra `/review` como reflejo antes de cada PR (M4.2). Lánzalo aunque la pieza sea pequeña. La fricción de decidir cuándo lanzarlo es más cara que lanzarlo siempre. A los cinco días el reflejo está fijado.

Semana cuatro: instala Context Mode (M5.3). Empieza con la regla del 40%: cuando `/contextmode:status` te diga 40%, comprime. Al final de la semana sabrás cuándo te aporta más y cuándo no.

Al final de las cuatro semanas tienes el flujo operativo de cuatro skills integradas. claud-mem y Skill Creator se quedan para el horizonte siguiente porque su valor depende de tener flujo previo.

Bloque 3 · Los primeros tres meses

Semana cinco: instala claud-mem (M6.4). A partir de aquí trabaja con normalidad. Las dos primeras semanas no notarás la diferencia (M6.4 sobre el periodo de calentamiento). A partir de la semana siete u ocho, empezarás a ver inyecciones útiles al abrir proyectos viejos.

Semanas seis a diez. Trabajo normal con las cinco operativas. Observa los huecos que aparezcan: situaciones donde piensas "esto debería estar automatizado" pero ninguna de las cinco lo cubre. Lleva nota mental o explícita. Aplica el filtro de las tres condiciones del M7.3 cuando un patrón se repita tres veces.

Semana once o doce, si aparece un hueco real validado. Instala Skill Creator (M1.2) y construye tu primera skill personalizada siguiendo los cinco pasos del M7.3. Si no hay hueco real, no instales Skill Creator; espera a que aparezca.

A los tres meses. Reevaluar. ¿Cuáles de las cinco operativas se quedaron? ¿Cuáles construiste tú? ¿Qué problemas siguen sin cubrir? ¿Qué del ecosistema ha cambiado en estos tres meses (porque seguro que algo ha cambiado)?

Bloque 4 · Lo que TOPSKL no ha cubierto, con honestidad

Tres áreas que el curso no toca y merece saber dónde mirar:

Skills específicas de stack o lenguaje. El catálogo de las seis es agnóstico al stack; las demos son en .NET por coherencia del vault, pero la mecánica vale igual para Node, Python, Go, Rust, etc. Hay skills específicas para cada stack (libs, frameworks concretos, patrones de un ecosistema). No las cubrimos porque el curso es de catálogo curado, no enciclopedia. Si trabajas mucho con un stack específico, busca en el ecosistema skills concretas para ese stack una vez tengas el flujo base.

MCP servers. Hemos hablado de ellos en M6.2 solo para distinguirlos de plugins. No los cubrimos como tema porque resuelven una clase distinta de problemas (capacidades externas bajo demanda) y merecerían un curso propio. Cuando empieces a necesitar capacidades que no resuelven las skills (consultar bases externas, llamar a APIs específicas con autenticación delegada), mira el ecosistema MCP.

Optimización fina de prompts y de configuración del modelo. Hay otra capa entera de afinado debajo de las skills: cómo escribir mejor tus prompts, cómo configurar parámetros del modelo, cómo elegir entre Sonnet y Opus según la tarea. Es una capa importante pero ortogonal al catálogo curado de TOPSKL. Recursos buenos para esto son la documentación oficial de Anthropic y guías comunitarias del nicho.

Lo importante: TOPSKL te deja el cimiento. Las tres áreas anteriores son extensiones que toman sentido cuando el cimiento está integrado.

Bloque 5 · Cómo seguir afilando criterio cuando el ecosistema cambie

La historia de GSD del M3.2 es la prueba de que el ecosistema se mueve rápido. Cualquier skill del catálogo puede sufrir el tipo de cambio que el vídeo origen no podía anticipar. Por eso este curso tiene fecha (29 de mayo de 2026) y por eso los frontmatters de cada submódulo llevan campo `fecha-verificacion`.

Tres prácticas que te ayudan a mantener tu criterio afilado en los próximos meses:

Revisión trimestral del catálogo. Cada tres meses, una hora para verificar las seis skills contra sus repos: ¿siguen vivas? ¿hay versiones nuevas con cambios relevantes? ¿algún incidente público que cambie la recomendación? Aplica los cuatro filtros del M0.3 con honestidad. Si una skill se cae, busca reemplazo o decide vivir sin ella.

Suscripción ligera al nicho. No necesitas leer cada artículo. Una vez al mes, treinta minutos en blogs como blog.fsck.com (Jesse Vincent), docs de los proyectos del catálogo, y discusiones en GitHub de los repos que usas. Te llega lo importante sin que te coma la vida.

Confiar en tu propio criterio sobre el de tutoriales recientes. Cuando veas un curso o vídeo nuevo recomendando una skill, aplica los cuatro filtros del Mo.3 antes de instalarla. Los tutoriales pueden estar obsoletos como lo está el vídeo origen de TOPSKL en cuanto a GSD. El criterio se queda; las recomendaciones envejecen.

Bloque 6 · Cierre del curso

Cuatro horas y media de vídeo. Treinta y dos submódulos. Seis skills explicadas con todo el detalle que te permite usarlas con criterio en lugar de seguir tutoriales. Y un método (los cuatro filtros del Mo.3) para evaluar lo que aparezca después del curso.

Si te llevas una sola cosa, llévate esta: **la calidad de tu flujo con Claude Code no depende de las skills que instales, sino del criterio con que las elijas, del orden con que las adoptes, y del procedimiento que dejes integrado a las tres semanas de uso.** Las seis del catálogo son buenas hoy. Algunas seguirán siendo buenas dentro de tres meses; otras serán reemplazadas. Tu criterio para decidir cuál es cuál es lo que permanece.

Gracias por llegar hasta el final. Si decides empezar la adopción ordenada del bloque 2 esta semana, en tres meses tu trabajo con Claude Code será cualitativamente distinto al de hoy. No porque tengas más herramientas, sino porque tienes un sistema.

Glosario rápido del bloque (y del curso entero)

- **Los tres horizontes** — la primera semana, las cuatro semanas, los tres meses. Cada uno con tareas concretas y diferenciadas.
- **Periodo de observación** — los días uno a tres del horizonte de una semana. Observar tus propios síntomas antes de instalar nada.
- **Reevaluación trimestral** — la práctica de verificar el catálogo cada tres meses contra los cuatro filtros del Mo.3. La forma de mantener el criterio vivo.
- **Suscripción ligera** — la atención periódica al ecosistema sin caer en saturación informativa. Treinta minutos al mes basta.
- **El sistema sobre las herramientas** — la idea clave del cierre. El valor a medio plazo está en el procedimiento integrado, no en la cantidad de herramientas instaladas.

Fin del curso

Si has llegado a este último glosario, has terminado TOPSKL. Esta línea es el final formal del temario. Lo que sigue depende de ti: la guía operativa del bloque 2 es el siguiente paso natural.

Nos vemos en tres meses con tu flujo integrado y, si aparece, con tu primera skill personalizada construida.